

The background is a solid purple color with a network of black, irregular cracks running across it, particularly concentrated on the right side. The text is centered and has a distressed, stencil-like appearance.

TECHNICAL INTERVIEW HANDBOOK

Technical Interview Handbook

Target Audience: React Native Developers, Web Developers (MERN), Undergraduates

Long Story Short;

I made this because I want printable material to refresh the knowledge before going to interviews.

If you find useful, share this with your friends 😊

Table Of Content

Chapter 1: Object-Oriented Programming (OOP).....	1
1.1 The Four Pillars	2
Encapsulation.....	2
Inheritance.....	2
Polymorphism	2
Abstraction.....	3
1.2 Interfaces vs Abstract Classes	4
1.3 Method Overloading vs Overriding	4
1.4 Advanced Concepts	4
Composition vs Inheritance	5
Association, Aggregation, Composition.....	5
1.5 SOLID Principles	5
S - Single Responsibility	5
O - Open/Closed	5
L - Liskov Substitution	6
I - Interface Segregation	6
D - Dependency Inversion.....	6
1.6 Interview Questions (50+)	6
1.7 Practice Problems	9
Chapter 2: Java Core Fundamentals	9
2.1 JVM Architecture	9
2.2 String, StringBuilder, StringBuffer	10
2.3 final, static, this, super	11
2.4 Wrapper Classes & Autoboxing	12
2.5 Exception Handling	12
2.6 Collections Framework	13
List.....	13
Set	13
Map	13
Queue	14
2.7 Multithreading.....	14

2.8 Java 8+ Features	16
Lambda Expressions.....	16
Functional Interfaces	16
Stream API	16
Optional	17
Method References	17
Date/Time API.....	18
2.9 Interview Questions (100+)	18
Chapter 3: Design Patterns	23
3.1 Creational Patterns.....	23
Singleton	23
Factory Method	24
Abstract Factory.....	24
Builder	25
Prototype	25
3.2 Structural Patterns	25
Adapter	26
Decorator	26
Facade	26
Proxy	27
Composite	27
3.3 Behavioral Patterns	28
Observer	28
Strategy.....	28
Command.....	29
Template Method	29
State	29
Chain of Responsibility.....	30
Iterator	30
Mediator	31
Memento	31
Visitor	32

3.4 Interview Questions (50+)	32
Chapter 4: Coding Challenges	34
4.1 String Problems (20)	34
Reverse String	34
Palindrome Check	35
Anagram Check	35
First Non-Repeating Character	35
Longest Substring Without Repeating	35
Valid Parentheses	36
String Compression	36
Longest Common Prefix	36
Count and Say	37
Group Anagrams	37
4.2 Array Problems (25)	37
Two Sum	37
Maximum Subarray (Kadane's)	38
Merge Sorted Arrays	38
Remove Duplicates from Sorted Array	38
Rotate Array	38
Product of Array Except Self	39
Find Missing Number	39
Move Zeroes	39
Container With Most Water	39
3Sum	40
4.3 Math Problems (15)	40
Reverse Integer	40
Palindrome Number	41
Power of Two	41
Fizz Buzz	41
Count Primes (Sieve of Eratosthenes)	41
Happy Number	42
Factorial	42

GCD	42
LCM	42
Excel Sheet Column Number	43
4.4 Recursion Problems (20)	43
Fibonacci	43
Climbing Stairs	43
Generate Parentheses	44
Letter Combinations of Phone Number	44
Permutations	44
Subsets	45
Combination Sum	45
Word Search	46
N-Queens	46
Sudoku Solver	47
4.5 Bit Manipulation (10)	47
Single Number	48
Number of 1 Bits	48
Reverse Bits	48
Missing Number (XOR)	48
Power of Four	48
Chapter 5: Data Structures & Algorithms	49
5.1 Arrays & Hashing (15 problems)	49
Contains Duplicate	49
Valid Anagram	49
Top K Frequent Elements	49
Encode and Decode Strings	50
Longest Consecutive Sequence	50
5.2 Two Pointers (10 problems)	50
Valid Palindrome	51
Two Sum II (Sorted)	51
3Sum Closest	51
Trapping Rain Water	51

5.3 Sliding Window (8 problems)	52
Best Time to Buy and Sell Stock	52
Longest Repeating Character Replacement	52
Minimum Window Substring	53
5.4 Stack (10 problems)	53
Valid Parentheses	53
Min Stack	53
Evaluate Reverse Polish Notation	54
Daily Temperatures	54
Largest Rectangle in Histogram	55
5.5 Binary Search (10 problems)	55
Binary Search	55
Search in Rotated Sorted Array	55
Find Minimum in Rotated Sorted Array	56
Search a 2D Matrix	56
Koko Eating Bananas	56
5.6 Linked List (12 problems)	57
Reverse Linked List	57
Merge Two Sorted Lists	57
Linked List Cycle	57
Remove Nth Node From End	58
Reorder List	58
Add Two Numbers	59
Copy List with Random Pointer	59
LRU Cache	59
5.7 Trees (15 problems)	60
Maximum Depth of Binary Tree	60
Same Tree	60
Invert Binary Tree	61
Binary Tree Level Order Traversal	61
Validate Binary Search Tree	61
Lowest Common Ancestor of BST	62

Binary Tree Right Side View	62
Kth Smallest Element in BST	62
Serialize and Deserialize Binary Tree	62
5.8 Tries (3 problems)	63
Implement Trie	63
Word Search II	64
5.9 Heaps / Priority Queue (5 problems)	65
Kth Largest Element	65
Merge K Sorted Lists	65
Find Median from Data Stream	65
5.10 Graphs (12 problems)	66
Number of Islands	66
Clone Graph	66
Pacific Atlantic Water Flow	66
Course Schedule	67
Course Schedule II	68
Chapter 6: Backend Engineering	68
6.1 Node.js Fundamentals	68
Event Loop	68
Event Emitter	69
Streams	69
Buffer	69
Cluster Module	69
6.2 Express.js	70
Basic Server	70
Middleware	70
Routing	70
File Upload	71
6.3 REST API Design	71
CRUD Operations	71
Pagination & Filtering	72
Error Handling	72

6.4 Authentication & Authorization	72
JWT Authentication	73
Session-based Auth.....	73
OAuth 2.0.....	74
6.5 Database - SQL (PostgreSQL).....	74
Basic Queries.....	74
Joins	74
Aggregations	75
Indexes.....	75
Transactions.....	75
6.6 Prisma ORM.....	75
Schema Definition.....	75
CRUD Operations	76
Relations	76
Filtering & Sorting.....	77
6.7 MongoDB & Mongoose	77
Schema Definition.....	77
CRUD Operations	77
Queries.....	78
Population (Joins)	78
6.8 Caching (Redis)	78
Basic Operations	78
Caching Middleware	79
Redis Data Structures	79
6.9 WebSockets	79
Socket.IO Server.....	79
Socket.IO Client.....	80
Rooms	80
6.10 Interview Questions (50+)	80
Chapter 7.5: React Native	82
7.5.1 React Native Core Concepts	82
Platform-Specific Code	82

Dimensions & Responsive Design.....	83
Flexbox Layout	83
SafeAreaView.....	83
StatusBar	83
KeyboardAvoidingView	84
ScrollView vs FlatList.....	84
SectionList	84
Image	84
ImageBackground	85
TouchableOpacity vs Pressable	85
Modal.....	85
ActivityIndicator.....	86
RefreshControl.....	86
7.5.2 React Navigation.....	86
Installation & Setup	86
Stack Navigator	86
Navigation Props.....	87
Tab Navigator.....	87
Drawer Navigator.....	88
Nested Navigation	88
useNavigation Hook.....	88
useFocusEffect	89
7.5.3 Zustand State Management	89
Installation	89
Basic Store.....	89
Async Actions.....	90
Selectors.....	90
Persist Middleware	90
Immer Middleware	90
Multiple Stores.....	91
7.5.4 Expo & EAS	91
Expo CLI Setup	91

Expo Config (app.json)	91
Expo Location.....	92
Expo Camera	92
Expo ImagePicker.....	92
Expo Notifications	93
Expo SecureStore	93
Expo FileSystem	93
EAS Build	94
EAS Update	94
eas.json	94
7.5.5 AsyncStorage	94
Installation	94
Basic Operations	94
Custom Hook	95
7.5.6 Animations.....	96
Animated API	96
Spring Animation	96
Parallel Animations	96
Sequence Animations	96
Loop Animation.....	97
LayoutAnimation.....	97
7.5.7 Gestures (React Native Gesture Handler)	97
Installation	97
TapGestureHandler.....	97
PanGestureHandler	98
Swipeable.....	98
7.5.8 API Integration.....	99
Fetch API	99
Axios.....	99
React Query with RN.....	100
7.5.9 Performance Optimization	101
FlatList Optimization	101

React.memo for List Items	101
useCallback for Handlers	101
Image Optimization	101
Hermes Engine	101
Native Driver	102
7.5.10 Debugging & Development Tools.....	102
React Native Debugger	102
Flipper	102
Console Logs	102
Remote Debugging	102
Performance Monitor	103
React DevTools	103
7.5.11 Testing React Native.....	103
Jest Unit Tests	103
React Native Testing Library	103
Mocking AsyncStorage.....	103
Snapshot Testing.....	103
E2E Testing (Detox).....	104
7.5.12 Complete App Example: Todo App	104
App Structure.....	104
7.5.13 Interview Questions (60+)	105
Chapter 7: Frontend Engineering.....	105
7.1 React Fundamentals	105
Components.....	105
Props	105
State (useState).....	105
useEffect	105
useContext.....	105
useReducer	105
useRef	105
useMemo	105
useCallback	105

Custom Hooks	105
7.2 Lists & Forms	105
Lists & Keys	105
Controlled Forms	105
Form Validation	105
7.3 React Router	105
Basic Routing.....	105
useParams & useNavigate	105
Protected Routes	105
7.4 State Management - Redux.....	105
Setup	105
Slice	105
Component	105
Async Actions.....	105
7.5 React Query	105
Setup	105
Query	105
Mutation	105
7.6 Styling	105
CSS Modules	105
Styled Components.....	105
Tailwind CSS.....	105
7.7 Performance Optimization	105
React.memo.....	105
Code Splitting.....	105
Virtualization.....	105
Debouncing & Throttling	105
7.8 Testing	105
Jest Unit Test.....	105
React Testing Library	105
Mock API	105
7.9 Next.js.....	105

Pages Router	105
Static Generation	105
API Routes	105
App Router (Next.js 13+)	105
Server Components	105
7.10 React Native	105
Basic Components	105
Lists	105
Navigation	105
7.11 Interview Questions (50+)	105
Chapter 8: System Design	105
8.1 System Design Fundamentals.....	105
Scalability	105
Load Balancing	105
Caching.....	105
Database	105
Replication & Sharding	105
Consistency Models	105
Message Queues.....	105
API Gateway.....	105
Microservices vs Monolith.....	105
8.2 Design: URL Shortener (bit.ly)	105
Requirements.....	105
Capacity Estimation	105
API Design	105
Database Schema.....	105
Algorithm	105
Architecture	105
Key Design Decisions	105
8.3 Design: Twitter	105
Requirements.....	105
Capacity Estimation	105

API Design	105
Database Schema.....	105
Timeline Generation	105
Architecture	105
8.4 Design: Instagram.....	105
Requirements.....	105
Photo Storage	105
Database Schema.....	105
Feed Generation	105
8.5 Design: WhatsApp	105
Requirements.....	105
Architecture	105
Database Schema.....	105
Real-time Communication	105
Message Delivery.....	105
8.6 Design: Netflix	105
Requirements.....	105
Video Processing Pipeline	105
Adaptive Bitrate Streaming	105
Recommendation System	105
Database Schema.....	105
8.7 Design: Uber	105
Requirements.....	105
Geo-spatial Indexing	105
Matching Algorithm	105
Real-time Tracking	105
Database Schema.....	105
8.8 Design: Google Drive	105
Requirements.....	105
File Storage	105
Chunking & Deduplication	105
Sync Protocol	105

Database Schema.....	105
8.9 Trade-offs & Best Practices	105
Common Trade-offs	105
Best Practices.....	105
8.10 Interview Questions (40+)	105
Chapter 9: Advanced Algorithms	105
9.1 Dynamic Programming (30 problems)	105
Fibonacci (Memoization)	105
Fibonacci (Tabulation)	105
Climbing Stairs	105
House Robber	105
Longest Increasing Subsequence	105
Longest Common Subsequence	105
Edit Distance	105
0/1 Knapsack.....	105
Coin Change	105
Partition Equal Subset Sum	105
Palindromic Substrings	105
Longest Palindromic Substring	105
Decode Ways	105
Unique Paths.....	105
Minimum Path Sum	105
Triangle Minimum Path	105
Maximum Product Subarray.....	105
Word Break	105
Regular Expression Matching	105
Burst Balloons	105
9.2 Graph Algorithms (15 problems)	105
DFS (Recursive)	105
BFS.....	105
Topological Sort (DFS).....	105
Topological Sort (Kahn's BFS)	105

Dijkstra's Shortest Path.....	105
Bellman-Ford (Negative Weights)	105
Floyd-Warshall (All Pairs).....	105
Prim's MST	105
Kruskal's MST (Union-Find).....	105
Detect Cycle (Undirected).....	105
Detect Cycle (Directed).....	105
Strongly Connected Components (Kosaraju)	105
9.3 Greedy Algorithms (10 problems)	105
Activity Selection	105
Jump Game	105
Jump Game II	105
Gas Station.....	105
Minimum Number of Arrows	105
9.4 Backtracking (10 problems).....	105
N-Queens	105
Sudoku Solver	105
9.5 Interview Questions (30+)	105

Chapter 1: Object-Oriented Programming (OOP)

1.1 The Four Pillars

Encapsulation

Bundling data and methods, hiding internals via access modifiers.

```
public class BankAccount {
    private double balance;
    private String accountId;
    public BankAccount(String id) { this.accountId = id; this.balance = 0; }
    public double getBalance() { return balance; }
    public void deposit(double amt) { if(amt > 0) balance += amt; }
    public boolean withdraw(double amt) {
        if(amt > 0 && amt <= balance) { balance -= amt; return true; }
        return false;
    }
}
```

Inheritance

Child class acquires properties from parent using **extends**.

```
class Vehicle {
    protected String brand;
    public Vehicle(String brand) { this.brand = brand; }
    public void start() { System.out.println(brand + " starting"); }
}
class Car extends Vehicle {
    private int doors;
    public Car(String brand, int doors) { super(brand); this.doors = doors; }
    public void honk() { System.out.println("Beep beep!"); }
}
class ElectricCar extends Car {
    private int battery;
    public ElectricCar(String brand, int doors, int battery) {
        super(brand, doors);
        this.battery = battery;
    }
    @Override
    public void start() { System.out.println(brand + " silent start"); }
}
```

Polymorphism

Compile-time (Overloading): Same name, different parameters

```
class Calculator {
    public int add(int a, int b) { return a + b; }
    public int add(int a, int b, int c) { return a + b + c; }
    public double add(double a, double b) { return a + b; }
    public String add(String a, String b) { return a + b; }
}
```

Runtime (Overriding): Redefining parent method in child

```
class Animal {
    public void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    public void sound() { System.out.println("Woof!"); }
}
class Cat extends Animal {
    @Override
    public void sound() { System.out.println("Meow!"); }
}
// Animal a = new Dog(); a.sound(); // Outputs: Woof!
```

Abstraction

Hiding implementation, showing only essentials via abstract classes/interfaces.

```
abstract class Shape {
    protected String color;
    public Shape(String color) { this.color = color; }
    public abstract double area();
    public abstract double perimeter();
    public void display() { System.out.println("Area: " + area()); }
}
class Circle extends Shape {
    private double radius;
    public Circle(String color, double radius) { super(color); this.radius = radius; }
    public double area() { return Math.PI * radius * radius; }
    public double perimeter() { return 2 * Math.PI * radius; }
}
class Rectangle extends Shape {
    private double length, width;
    public Rectangle(String color, double l, double w) { super(color); length = l; width = w; }
    public double area() { return length * width; }
```

```
public double perimeter() { return 2 * (length + width); }
}
```

1.2 Interfaces vs Abstract Classes

Feature	Interface	Abstract Class
Methods	All abstract (default/static in Java 8+)	Both abstract & concrete
Variables	public static final only	Any access modifier
Inheritance	Multiple (implements)	Single (extends)
Constructor	No	Yes
When to use	Contract for unrelated classes	Common base for related classes

```
interface Flyable {
    void fly();
    int MAX_ALTITUDE = 50000;
    default void land() { System.out.println("Landing"); }
}
interface Swimmable {
    void swim();
}
class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("Duck flying"); }
    public void swim() { System.out.println("Duck swimming"); }
}
```

1.3 Method Overloading vs Overriding

Feature	Overloading	Overriding
Definition	Same name, diff params, same class	Same signature, parent-child
Polymorphism	Compile-time	Runtime
Return type	Can differ	Same or covariant
Access	Can differ	Cannot be more restrictive
private/static/final	Can overload	Cannot override

1.4 Advanced Concepts

Composition vs Inheritance

Prefer composition for flexibility.

```
interface FlightBehavior { void fly(); }
class CanFly implements FlightBehavior { public void fly() { System.out.println("Flying"); } }
class CannotFly implements FlightBehavior { public void fly() { System.out.println("Can't fly"); } }
class Bird {
    private FlightBehavior flightBehavior;
    public Bird(FlightBehavior fb) { this.flightBehavior = fb; }
    public void performFly() { flightBehavior.fly(); }
    public void setFlightBehavior(FlightBehavior fb) { this.flightBehavior = fb; }
}
```

Association, Aggregation, Composition

- **Association:** Uses-a (Teacher teaches Student)
- **Aggregation:** Has-a, weak (Department has Teachers)
- **Composition:** Part-of, strong (Car has Engine)

1.5 SOLID Principles

S - Single Responsibility

One class, one responsibility.

```
// Bad
class Employee {
    void calculateSalary() {}
    void saveToDatabase() {}
    void generateReport() {}
}

// Good
class Employee { /* data */ }
class SalaryCalculator { void calculate(Employee e) {} }
class EmployeeRepository { void save(Employee e) {} }
class ReportGenerator { void generate(Employee e) {} }
```

O - Open/Closed

Open for extension, closed for modification.

```
interface Customer { double getDiscount(double amt); }
class RegularCustomer implements Customer { public double getDiscount(double amt) { return 0; } }
class GoldCustomer implements Customer { public double getDiscount(double amt) { return amt * 0.2; } }
```

L - Liskov Substitution

Child objects should replace parent objects without breaking code.

```
// Bad: Square violates LSP
class Rectangle { void setWidth(int w) {} void setHeight(int h) {} }
class Square extends Rectangle {
    void setWidth(int w) { super.setWidth(w); super.setHeight(w); } // Breaks LSP
}
// Good: Use interfaces
interface Shape { int area(); }
class Rectangle implements Shape { /* implementation */ }
class Square implements Shape { /* implementation */ }
```

I - Interface Segregation

No client should depend on unused methods.

```
// Bad
interface Worker { void work(); void eat(); void sleep(); }
// Good
interface Workable { void work(); }
interface Eatable { void eat(); }
interface Sleepable { void sleep(); }
class Human implements Workable, Eatable, Sleepable { /* all methods */ }
class Robot implements Workable { /* only work */ }
```

D - Dependency Inversion

Depend on abstractions, not concretions.

```
interface Database { void save(String data); }
class MySQL implements Database { public void save(String data) {} }
class MongoDB implements Database { public void save(String data) {} }
class UserService {
    private Database db;
    public UserService(Database db) { this.db = db; }
    public void saveUser(String user) { db.save(user); }
}
```

1.6 Interview Questions (50+)

Q1: What are the 4 pillars of OOP? **A:** Encapsulation, Inheritance, Polymorphism, Abstraction

Q2: Difference between abstract class and interface? **A:** Abstract: single inheritance, can have constructors, any access modifiers. Interface: multiple inheritance, no constructors, public methods only (pre-Java 8).

Q3: Can we override static methods? **A:** No. Static methods are hidden, not overridden (method hiding).

Q4: Can we override private methods? **A:** No. Private methods are not visible to subclasses.

Q5: Can we override final methods? **A:** No. Final methods cannot be overridden.

Q6: Difference between method overloading and overriding? **A:** Overloading: same name, diff params, compile-time. Overriding: same signature, runtime, parent-child.

Q7: What is the diamond problem? **A:** Multiple inheritance ambiguity. Java solves with interfaces (must explicitly override conflicting default methods).

Q8: What is covariant return type? **A:** Overriding method can return subtype of parent's return type.

```
class Animal { Animal reproduce() { return new Animal(); } }  
class Dog extends Animal { Dog reproduce() { return new Dog(); } }
```

Q9: Difference between this and super? **A:** **this**: current instance. **super**: parent instance.

Q10: What is a marker interface? **A:** Empty interface (no methods). Examples: Serializable, Cloneable, Remote.

Q11: Can interface have constructors? **A:** No. Interfaces cannot have constructors.

Q12: Can abstract class have final methods? **A:** Yes. Final methods can exist in abstract classes.

Q13: Can we declare interface methods as private? **A:** Yes, from Java 9+ (private methods in interfaces).

Q14: What is multiple inheritance of type? **A:** Class implementing multiple interfaces.

Q15: Difference between composition and aggregation? **A:** Composition: strong ownership, part dies with whole. Aggregation: weak, parts can exist independently.

Q16: Why favor composition over inheritance? **A:** More flexible, loose coupling, runtime behavior changes, no diamond problem.

Q17: Can we have multiple classes in single Java file? **A:** Yes, but only one public class with filename matching.

Q18: What is constructor chaining? **A:** Calling one constructor from another using **this()** or **super()**.

Q19: Can constructors be overridden? **A:** No. Constructors can be overloaded, not overridden.

Q20: What happens if we don't override hashCode() with equals()? **A:** Breaks collections (HashMap, HashSet). Equal objects may have different hash codes.

Q21: Difference between == and equals()? **A:** ==: reference comparison. equals(): content comparison (if overridden).

Q22: Can final class be extended? **A:** No. Final classes cannot be subclassed.

Q23: Can we have final abstract class? **A:** No. Contradictory (abstract means must extend, final means cannot extend).

Q24: What is the use of super keyword? **A:** Access parent constructor, methods, or fields.

Q25: Can static methods access instance variables? **A:** No. Static context cannot access instance members directly.

Q26: Can we override equals() without overriding hashCode()? **A:** Yes, but violates contract. Should override both.

Q27: What is method hiding? **A:** Static method in child with same signature as parent (not overriding).

Q28: Can we have abstract static method? **A:** No. Static methods cannot be abstract.

Q29: Difference between protected and default access? **A:** Protected: accessible in subclasses. Default: package-private only.

Q30: Can interface extend another interface? **A:** Yes, using extends. Can extend multiple interfaces.

Q31: What is the purpose of transient keyword? **A:** Exclude field from serialization.

Q32: Can we serialize static variables? **A:** No. Static variables belong to class, not instance.

Q33: What is object cloning? **A:** Creating exact copy of object. Use clone() method, implement Cloneable.

Q34: Difference between shallow and deep copy? **A:** Shallow: copies references. Deep: copies actual objects recursively.

Q35: Can we call non-static method from static method? **A:** No, unless you create an instance first.

Q36: What is the use of finalize() method? **A:** Called by GC before destroying object. Deprecated in Java 9+.

Q37: Difference between abstract class and concrete class? **A:** Abstract: cannot instantiate, may have abstract methods. Concrete: can instantiate, all methods implemented.

Q38: Can we have private abstract method? **A:** No. Abstract methods must be overridden, private methods are not visible to subclasses.

- Q39:** What is upcasting and downcasting? **A:** Upcasting: child to parent (implicit). Downcasting: parent to child (explicit, may throw `ClassCastException`).
- Q40:** What is instanceof operator? **A:** Checks if object is instance of specific class/interface.
- Q41:** Can we override main method? **A:** No. Main is static, static methods cannot be overridden.
- Q42:** What is anonymous inner class? **A:** Class without name, defined and instantiated in single expression.
- Q43:** Difference between nested and inner class? **A:** Inner: non-static nested. Nested: can be static or non-static.
- Q44:** Can we have static class in Java? **A:** Yes, but only as nested class.
- Q45:** What is the use of instanceof? **A:** Type checking before casting to avoid `ClassCastException`.
- Q46:** Can interface have static variables? **A:** Yes. All interface variables are **public static final** by default.
- Q47:** What is default method in interface? **A:** Method with implementation in interface (Java 8+). Provides backward compatibility.
- Q48:** Can we have synchronized abstract method? **A:** No. Abstract methods have no body to synchronize.
- Q49:** What is the purpose of protected constructor? **A:** Allow subclass instantiation, prevent direct instantiation from outside package.
- Q50:** Difference between is-a and has-a? **A:** is-a: inheritance. has-a: composition/aggregation.

1.7 Practice Problems

P1: Design Library System - Books, Members, borrow/return **P2: Vehicle Hierarchy** - Vehicle > Car/Bike, polymorphic array **P3: Shape Calculator** - Abstract Shape, Circle/Rectangle/Triangle **P4: Banking System** - Account types, transactions, inheritance **P5: Employee Management** - Employee hierarchy, salary calculation **P6: Singleton Pattern** - Thread-safe implementation **P7: Factory Pattern** - Create objects based on type **P8: Observer Pattern** - Event notification system **P9: Strategy Pattern** - Payment methods selection **P10: Decorator Pattern** - Add features dynamically

Chapter 2: Java Core Fundamentals

2.1 JVM Architecture

Components: ClassLoader → Runtime Data Areas → Execution Engine → Native Interface

Memory Areas:

- **Heap:** Objects, instance variables (shared, GC managed)
- **Stack:** Local variables, method frames (thread-specific, LIFO)
- **Method Area (Metaspace):** Class metadata, static variables, constants
- **PC Register:** Current instruction address
- **Native Method Stack:** Native method calls

GC Algorithms: Serial (single-thread), Parallel (multi-thread), CMS (low pause), G1GC (default Java 9+, predictable pause)

```
class MemoryDemo {
    static int counter = 0; // Method Area
    int id; // Heap (with object)
    void method() {
        int local = 10; // Stack
        String str = "text"; // "text" in String Pool (heap), str reference in stack
    }
}
```

2.2 String, StringBuilder, StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes	No	Yes (synchronized)
Performance	Slowest	Fastest	Slower than SB
Use case	Fixed strings	Single-threaded concat	Multi-threaded concat

```
String s1 = "Hello"; // String pool
String s2 = "Hello"; // Same reference
String s3 = new String("Hello"); // Heap
System.out.println(s1 == s2); // true (same reference)
System.out.println(s1 == s3); // false (different references)
System.out.println(s1.equals(s3)); // true (same content)
```

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World").insert(5, ",").delete(5, 6);
```

String Methods:

```
String s = " Hello World ";
s.length(); s.charAt(0); s.trim(); s.toLowerCase(); s.toUpperCase();
```

```
s.indexOf("World"); s.contains("World"); s.startsWith("He"); s.endsWith("ld");
s.substring(2, 7); s.replace("World", "Java"); s.split(" ");
s.equals("Hello"); s.equalsIgnoreCase("HELLO"); s.compareTo("Hello");
s.isEmpty(); s.isBlank(); // Java 11+
```

2.3 final, static, this, super

final:

- Variable: constant
- Method: cannot override
- Class: cannot extend

```
final int MAX = 100; // Cannot change
final void display() {} // Cannot override
final class Immutable {} // Cannot extend
final StringBuilder sb = new StringBuilder();
sb.append("x"); // OK, object mutable
// sb = new StringBuilder(); // ERROR, reference cannot change
```

static:

```
class Counter {
    static int count = 0; // Shared by all instances
    static { System.out.println("Static block"); } // Runs once
    static void increment() { count++; }
}
```

this vs super:

```
class Parent {
    int x = 10;
    Parent() { System.out.println("Parent"); }
    void show() { System.out.println("Parent show"); }
}
class Child extends Parent {
    int x = 20;
    Child() {
        super(); // Call parent constructor
        System.out.println("Child");
    }
    void display() {
        System.out.println(this.x); // 20 (current class)
        System.out.println(super.x); // 10 (parent class)
    }
}
```

```

    super.show(); // Call parent method
}
}

```

2.4 Wrapper Classes & Autoboxing

```

Integer i1 = new Integer(10); // Deprecated
Integer i2 = Integer.valueOf(10); // Recommended
Integer i3 = 10; // Autoboxing
int i4 = i3; // Unboxing

```

```

// Caching: -128 to 127
Integer a = 127, b = 127;
System.out.println(a == b); // true (cached)
Integer c = 128, d = 128;
System.out.println(c == d); // false (not cached)

```

```

// Methods
Integer.parseInt("123"); Integer.compare(10, 20);
Integer.MAX_VALUE; Integer.MIN_VALUE; Integer.SIZE; Integer.BYTES;
Integer.toBinaryString(10); Integer.toHexString(10);

```

2.5 Exception Handling

Hierarchy: Throwable → Error / Exception → RuntimeException

Checked vs Unchecked:

Type	Checked	Unchecked
Check time	Compile-time	Runtime
Must handle	Yes	No
Examples	IOException, SQLException	NullPointerException, ArrayIndexOutOfBoundsException

```

// Try-catch-finally
try {
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {

```

```

    System.out.println("Always executes");
}

// Multi-catch
try { /* code */ }
catch (IOException | SQLException e) { /* handle */ }

// Try-with-resources
try (FileReader fr = new FileReader("file.txt");
     BufferedReader br = new BufferedReader(fr)) {
    String line = br.readLine();
} catch (IOException e) { e.printStackTrace(); }

// Custom exception
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String msg) { super(msg); }
}

```

2.6 Collections Framework

List

```

List<String> list1 = new ArrayList<>(); // Dynamic array, fast access O(1)
List<String> list2 = new LinkedList<>(); // Doubly linked, fast insert/delete
List<String> list3 = new Vector<>(); // Synchronized ArrayList
List<String> list4 = new Stack<>(); // LIFO

list1.add("A"); list1.get(0); list1.remove(0); list1.size();
list1.contains("A"); list1.indexOf("A"); list1.clear();
Collections.sort(list1); Collections.reverse(list1);

```

Set

```

Set<String> set1 = new HashSet<>(); // No order, O(1) operations
Set<String> set2 = new LinkedHashSet<>(); // Insertion order
Set<String> set3 = new TreeSet<>(); // Sorted order, O(log n)

```

Map

```
Map<String, Integer> map1 = new HashMap<>(); // No order, allows null
Map<String, Integer> map2 = new LinkedHashMap<>(); // Insertion order
Map<String, Integer> map3 = new TreeMap<>(); // Sorted by keys
Map<String, Integer> map4 = new Hashtable<>(); // Synchronized, no null
Map<String, Integer> map5 = new ConcurrentHashMap<>(); // Thread-safe

map1.put("A", 1); map1.get("A"); map1.remove("A"); map1.containsKey("A");
map1.keySet(); map1.values(); map1.entrySet();
for (Map.Entry<String, Integer> entry : map1.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

Queue

```
Queue<Integer> q1 = new LinkedList<>(); // FIFO
Queue<Integer> q2 = new PriorityQueue<>(); // Min heap
Deque<Integer> dq = new ArrayDeque<>(); // Double-ended queue

q1.offer(1); q1.poll(); q1.peek();
dq.offerFirst(1); dq.offerLast(2); dq.pollFirst(); dq.pollLast();
```

HashMap Internal: Array of buckets, hash collision handled by LinkedList (Java 7) or Balanced Tree (Java 8+ when size > 8).

```
map.put(key, value);
// 1. hash = key.hashCode()
// 2. index = hash % arrayLength
// 3. Store at index, handle collision with linked list/tree
```

2.7 Multithreading

Creating Threads:

```
// Method 1: Extend Thread
class MyThread extends Thread {
    public void run() { System.out.println("Thread running"); }
}
new MyThread().start();

// Method 2: Implement Runnable
```

```
class MyRunnable implements Runnable {
    public void run() { System.out.println("Thread running"); }
}
new Thread(new MyRunnable()).start();
```

// Method 3: Lambda

```
new Thread(() -> System.out.println("Thread running")).start();
```

Thread Lifecycle: New → Runnable → Running → Blocked/Waiting → Terminated

Synchronization:

```
class Counter {
    private int count = 0;
    public synchronized void increment() { count++; }
    public int getCount() { return count; }
}
```

```
synchronized(object) { /* critical section */
```

// Locks

```
ReentrantLock lock = new ReentrantLock();
lock.lock();
try { /* critical section */ }
finally { lock.unlock(); }
```

Thread Methods:

```
Thread.sleep(1000); // Pause current thread
thread.join(); // Wait for thread to complete
thread.interrupt(); // Interrupt thread
Thread.yield(); // Give up CPU
Thread.currentThread(); // Get current thread
```

Executor Framework:

```
ExecutorService executor = Executors.newFixedThreadPool(5);
executor.submit(() -> System.out.println("Task"));
executor.shutdown();
```

// Types

```
Executors.newSingleThreadExecutor(); // 1 thread
Executors.newFixedThreadPool(n); // n threads
Executors.newCachedThreadPool(); // Dynamic pool
```

```
Executors.newScheduledThreadPool(n); // Scheduled tasks
```

CompletableFuture:

```
CompletableFuture.supplyAsync(() -> "Result")
    .thenApply(s -> s.toUpperCase())
    .thenAccept(System.out::println);
```

2.8 Java 8+ Features

Lambda Expressions

// Old way

```
Comparator<String> comp1 = new Comparator<String>() {
    public int compare(String s1, String s2) { return s1.compareTo(s2); }
};
```

// Lambda

```
Comparator<String> comp2 = (s1, s2) -> s1.compareTo(s2);
```

Functional Interfaces

`@FunctionalInterface`

```
interface Calculator { int calculate(int a, int b); }
```

```
Calculator add = (a, b) -> a + b;
```

```
Calculator multiply = (a, b) -> a * b;
```

// Built-in: Predicate, Function, Consumer, Supplier

```
Predicate<Integer> isEven = x -> x % 2 == 0;
```

```
Function<String, Integer> length = s -> s.length();
```

```
Consumer<String> print = System.out::println;
```

```
Supplier<Double> random = Math::random;
```

Stream API

```
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
```

// Intermediate operations (lazy)

```
list.stream()
    .filter(x -> x % 2 == 0)
```



```
.map(x -> x * 2)
.sorted()
.distinct()
.limit(3)
.skip(1);
```

// Terminal operations (eager)

```
list.stream().forEach(System.out::println);
list.stream().count();
list.stream().collect(Collectors.toList());
list.stream().reduce(0, (a, b) -> a + b);
list.stream().anyMatch(x -> x > 3);
list.stream().allMatch(x -> x > 0);
list.stream().noneMatch(x -> x < 0);
list.stream().findFirst();
list.stream().findAny();
```

// Collectors

```
Collectors.toList();
Collectors.toSet();
Collectors.toMap(k -> k, v -> v);
Collectors.joining(", ");
Collectors.groupingBy(x -> x % 2);
Collectors.partitioningBy(x -> x % 2 == 0);
Collectors.counting();
Collectors.summingInt(x -> x);
Collectors.averagingInt(x -> x);
```

Optional

```
Optional<String> opt = Optional.of("value");
Optional<String> empty = Optional.empty();
Optional<String> nullable = Optional.ofNullable(null);
```

```
opt.isPresent(); // true if value exists
opt.ifPresent(s -> System.out.println(s));
opt.orElse("default");
opt.orElseGet(() -> "default");
opt.orElseThrow(() -> new Exception());
opt.map(s -> s.toUpperCase());
opt.filter(s -> s.length() > 5);
```

Method References

// Static method

```
Function<String, Integer> parse1 = s -> Integer.parseInt(s);
Function<String, Integer> parse2 = Integer::parseInt;
```

// Instance method

```
Consumer<String> print1 = s -> System.out.println(s);
Consumer<String> print2 = System.out::println;
```

// Constructor

```
Supplier<ArrayList> list1 = () -> new ArrayList();
Supplier<ArrayList> list2 = ArrayList::new;
```

Date/Time API

```
LocalDate date = LocalDate.now();
LocalTime time = LocalTime.now();
LocalDateTime dateTime = LocalDateTime.now();
ZonedDateTime zonedDT = ZonedDateTime.now();
```

```
date.plusDays(1); date.minusMonths(1); date.isAfter(otherDate);
date.format(DateTimeFormatter.ISO_DATE);
```

2.9 Interview Questions (100+)

Q1: Why is main() public static void? **A:** Public (JVM access), static (no instance needed), void (no return to JVM).

Q2: Difference between == and equals()? **A:** == compares references, equals() compares content.

Q3: Why String is immutable? **A:** Security, thread-safety, String pool efficiency, hashCode caching.

Q4: String vs StringBuilder vs StringBuffer? **A:** String immutable; StringBuilder fast, not thread-safe; StringBuffer thread-safe, slower.

Q5: Explain String pool. **A:** Special memory area in heap where literal strings are stored to avoid duplicates.

Q6: Difference between ArrayList and LinkedList? **A:** ArrayList: array-based, fast access O(1), slow insert/delete. LinkedList: node-based, slow access O(n), fast insert/delete.

Q7: How HashMap works internally? **A:** Array of buckets, uses hashCode() to find bucket, equals() to find key. Collision handled by LinkedList/Tree.

Q8: Difference between HashMap and Hashtable? **A:** HashMap: not synchronized, allows null, faster. Hashtable: synchronized, no null, slower.

Q9: Difference between HashSet and TreeSet? **A:** HashSet: no order, O(1). TreeSet: sorted, O(log n).

Q10: What is fail-fast iterator? **A:** Throws ConcurrentModificationException if collection modified during iteration.

Q11: Difference between Iterator and ListIterator? **A:** Iterator: forward only, all collections. ListIterator: bidirectional, lists only.

Q12: Explain checked vs unchecked exceptions. **A:** Checked: compile-time, must handle. Unchecked: runtime, optional handling.

Q13: What is try-with-resources? **A:** Auto-closes resources implementing AutoCloseable. No explicit finally needed.

Q14: Can we have multiple catch blocks? **A:** Yes. Order: specific to general. Multi-catch with | for same handling.

Q15: Difference between throw and throws? **A:** throw: explicitly throw exception. throws: declare method may throw.

Q16: What is finally block? **A:** Executes always (except System.exit()). Used for cleanup.

Q17: Can finally block be skipped? **A:** Only by System.exit() or JVM crash.

Q18: Difference between final, finally, finalize? **A:** final: keyword. finally: block. finalize: method (deprecated).

Q19: What is Thread lifecycle? **A:** New → Runnable → Running → Blocked/Waiting → Terminated.

Q20: Difference between process and thread? **A:** Process: independent execution. Thread: lightweight process, shares memory.

Q21: How to create thread? **A:** Extend Thread or implement Runnable.

Q22: Difference between start() and run()? **A:** start(): creates new thread, calls run(). run(): executes in current thread.

Q23: What is synchronization? **A:** Controlling access to shared resources by multiple threads.

Q24: Difference between synchronized method and block? **A:** Method: locks entire method. Block: locks specific code section.

Q25: What is deadlock? **A:** Two threads waiting for resources held by each other.

Q26: How to avoid deadlock? **A:** Lock ordering, timeouts, deadlock detection.

Q27: Difference between wait() and sleep()? **A:** wait(): releases lock, needs notify(). sleep(): keeps lock.

Q28: What is thread pool? **A:** Reusable worker threads for executing tasks.

Q29: Explain Executor framework. **A:** High-level concurrency API for managing thread pools.

Q30: What is lambda expression? **A:** Anonymous function implementing functional interface.

Q31: What is functional interface? **A:** Interface with single abstract method. Can have default/static methods.

Q32: Built-in functional interfaces? **A:** Predicate, Function, Consumer, Supplier, BiFunction, UnaryOperator, etc.

Q33: What is method reference? **A:** Shorthand for lambda calling existing method.

Q34: Types of method references? **A:** Static, instance, constructor, arbitrary object.

Q35: What is Stream API? **A:** Functional-style operations on collections (filter, map, reduce).

Q36: Intermediate vs terminal operations? **A:** Intermediate: lazy, return stream. Terminal: eager, return result.

Q37: Difference between map() and flatMap()? **A:** map: one-to-one. flatMap: one-to-many, flattens nested streams.

Q38: What is Optional? **A:** Container that may or may not contain value. Avoids null checks.

Q39: Difference between findFirst() and findAny()? **A:** findFirst: deterministic. findAny: non-deterministic, faster in parallel.

Q40: What is parallel stream? **A:** Stream that processes elements in parallel using ForkJoinPool.

Q41: When to use parallel stream? **A:** Large datasets, independent operations, multi-core CPU.

Q42: What is default method in interface? **A:** Method with implementation in interface (Java 8+).

Q43: Can interface have private methods? **A:** Yes, from Java 9+.

Q44: What is var keyword? **A:** Local variable type inference (Java 10+).

Q45: Difference between Comparable and Comparator? **A:** Comparable: natural ordering, compareTo(). Comparator: custom ordering, compare().

Q46: What is serialization? **A:** Converting object to byte stream for storage/transmission.

Q47: What is transient keyword? **A:** Excludes field from serialization.

Q48: Difference between Serializable and Externalizable? **A:** Serializable: automatic. Externalizable: custom, manual.

Q49: What is reflection? **A:** Inspecting/modifying runtime behavior of classes.

Q50: What is garbage collection? **A:** Automatic memory management, reclaims unused objects.

Q51: Types of GC? **A:** Serial, Parallel, CMS, G1GC, ZGC, Shenandoah.

Q52: How to suggest GC? **A:** System.gc() (not guaranteed to run).

Q53: What is memory leak in Java? **A:** Objects not GC'd due to unintended references.

Q54: Difference between heap and stack? **A:** Heap: objects, GC managed. Stack: primitives, method frames, LIFO.

Q55: What is PermGen vs Metaspace? **A:** PermGen (Java 7): fixed size, heap. Metaspace (Java 8+): native memory, auto-grows.

Q56: Explain ClassLoader hierarchy. **A:** Bootstrap → Extension → Application → Custom.

Q57: What is JIT compiler? **A:** Compiles bytecode to native code at runtime for performance.

Q58: Difference between JDK, JRE, JVM? **A:** JDK: dev tools. JRE: runtime. JVM: executes bytecode.

Q59: What is bytecode? **A:** Platform-independent intermediate code (.class files).

Q60: What is classpath? **A:** Path where JVM searches for classes.

Q61: Difference between abstract class and interface (Java 8+)? **A:** Abstract: single inheritance, constructors. Interface: multiple inheritance, no constructors, default methods.

Q62: Can we have static method in interface? **A:** Yes (Java 8+). Called via interface name.

Q63: Difference between Enumeration and Iterator? **A:** Enumeration: legacy, no remove(). Iterator: modern, has remove().

Q64: What is ConcurrentHashMap? **A:** Thread-safe HashMap using segment locking (Java 7) or CAS (Java 8+).

Q65: Difference between synchronized and concurrent collections? **A:** Synchronized: lock entire collection. Concurrent: fine-grained locking.

Q66: What is CopyOnWriteArrayList? **A:** Thread-safe list, creates copy on modification. Good for read-heavy scenarios.

Q67: What is BlockingQueue? **A:** Queue that blocks when full (put) or empty (take).

Q68: What is volatile keyword? **A:** Ensures visibility of changes across threads. No caching in thread-local.

Q69: Difference between volatile and atomic? **A:** Volatile: visibility only. Atomic: visibility + atomicity.

Q70: What is happens-before relationship? **A:** Memory consistency guarantee between operations.

Q71: What is immutable class? **A:** Class whose state cannot change after creation.

Q72: How to create immutable class? **A:** Final class, private final fields, no setters, defensive copies.

Q73: Why String is immutable? **A:** Security, thread-safety, caching, hashCode consistency.

Q74: Difference between StringBuilder and StringBuffer? **A:** StringBuilder: not thread-safe, faster. StringBuffer: thread-safe, slower.

Q75: What is String intern()? **A:** Moves string to pool, returns pooled reference.

Q76: Difference between StringBuilder append() and String concat? **A:** append(): modifies existing. concat(): creates new string.

Q77: What is enum? **A:** Special class representing fixed constants.

Q78: Can enum have constructor? **A:** Yes, but private only.

Q79: Can enum implement interface? **A:** Yes.

Q80: What is singleton pattern? **A:** Ensures class has only one instance.

Q81: Thread-safe singleton implementation? **A:** Double-checked locking, enum, static holder.

Q82: What is factory pattern? **A:** Creates objects without exposing creation logic.

Q83: Difference between == and equals() for enum? **A:** Both work same (enum constants are singleton).

Q84: What is generic? **A:** Type parameterization for compile-time type safety.

Q85: What is type erasure? **A:** Generics removed at runtime, replaced with Object or bounds.

Q86: Difference between List and ArrayList? **A:** List: unknown type. ArrayList: specifically Object.

Q87: What is bounded wildcard? **A:** Restricts generic type. <? extends T>: upper bound. <? super T>: lower bound.

Q88: What is PECS principle? **A:** Producer Extends, Consumer Super.

Q89: Can we have generic array? **A:** No. Type erasure prevents it.

Q90: What is annotation? **A:** Metadata for classes, methods, fields.

Q91: Built-in annotations? **A:** @Override, @Deprecated, @SuppressWarnings, @FunctionalInterface, @SafeVarargs.

Q92: How to create custom annotation? **A:** @interface MyAnnotation { }

Q93: What is retention policy? **A:** SOURCE (compile), CLASS (bytecode), RUNTIME (reflection).

Q94: What is marker annotation? **A:** Annotation with no elements. Example: @Override.

Q95: Difference between == and .equals() for wrapper classes? **A:** ==: reference. equals(): value. Watch out for caching (-128 to 127).

Q96: What is autoboxing? **A:** Automatic conversion primitive ↔ wrapper.

Q97: Performance impact of autoboxing? **A:** Overhead in loops. Use primitives for performance-critical code.

Q98: What is varargs? **A:** Variable number of arguments. Type... name.

Q99: Can varargs be overloaded? **A:** Yes, but ambiguous calls cause compile error.

Q100: What is assert keyword? **A:** Testing assumptions. Disabled by default. Enable with -ea flag.

Chapter 3: Design Patterns

3.1 Creational Patterns

Singleton

Ensures only one instance exists.

```
// Thread-safe enum singleton (best)
enum Singleton { INSTANCE;
    public void doSomething() {}
}
```

// Lazy initialization with double-checked locking

```
class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) instance = new Singleton();
            }
        }
        return instance;
    }
}
```

// Bill Pugh Singleton (lazy + thread-safe)

```
class Singleton {
    private Singleton() {}
    private static class Holder { static final Singleton INSTANCE = new Singleton(); }
    public static Singleton getInstance() { return Holder.INSTANCE; }
}
```

Factory Method

Creates objects without specifying exact class.

```
interface Shape { void draw(); }
class Circle implements Shape { public void draw() { System.out.println("Circle"); } }
class Rectangle implements Shape { public void draw() { System.out.println("Rectangle"); } }
class ShapeFactory {
    public Shape getShape(String type) {
        if ("CIRCLE".equals(type)) return new Circle();
        if ("RECTANGLE".equals(type)) return new Rectangle();
        return null;
    }
}
```

Abstract Factory

Creates families of related objects.

```
interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}
class WindowsFactory implements GUIFactory {
    public Button createButton() { return new WindowsButton(); }
}
```



```

    public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}
class MacFactory implements GUIFactory {
    public Button createButton() { return new MacButton(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}

```

Builder

Constructs complex objects step by step.

```

class Pizza {
    private String dough;
    private String sauce;
    private String topping;
    private Pizza(Builder builder) {
        this.dough = builder.dough;
        this.sauce = builder.sauce;
        this.topping = builder.topping;
    }
    static class Builder {
        private String dough;
        private String sauce;
        private String topping;
        public Builder dough(String dough) { this.dough = dough; return this; }
        public Builder sauce(String sauce) { this.sauce = sauce; return this; }
        public Builder topping(String topping) { this.topping = topping; return this; }
        public Pizza build() { return new Pizza(this); }
    }
}
// Usage: Pizza pizza = new Pizza.Builder().dough("thin").sauce("tomato").topping("cheese").build();

```

Prototype

Clones existing objects.

```

class Shape implements Cloneable {
    private String type;
    public Shape(String type) { this.type = type; }
    public Object clone() {
        try { return super.clone(); }
        catch (CloneNotSupportedException e) { return null; }
    }
}

```

3.2 Structural Patterns

Adapter

Converts interface to another interface.

```
interface MediaPlayer { void play(String filename); }
interface AdvancedMediaPlayer { void playVlc(String filename); void playMp4(String filename); }
class VlcPlayer implements AdvancedMediaPlayer {
    public void playVlc(String filename) { System.out.println("Playing vlc: " + filename); }
    public void playMp4(String filename) {}
}
class MediaAdapter implements MediaPlayer {
    AdvancedMediaPlayer player;
    public MediaAdapter(String type) {
        if ("vlc".equals(type)) player = new VlcPlayer();
    }
    public void play(String filename) { player.playVlc(filename); }
}
```

Decorator

Adds new functionality dynamically.

```
interface Coffee { double cost(); String description(); }
class SimpleCoffee implements Coffee {
    public double cost() { return 1.0; }
    public String description() { return "Simple coffee"; }
}
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;
    public CoffeeDecorator(Coffee coffee) { this.coffee = coffee; }
}
class Milk extends CoffeeDecorator {
    public Milk(Coffee coffee) { super(coffee); }
    public double cost() { return coffee.cost() + 0.5; }
    public String description() { return coffee.description() + ", milk"; }
}
class Sugar extends CoffeeDecorator {
    public Sugar(Coffee coffee) { super(coffee); }
    public double cost() { return coffee.cost() + 0.2; }
    public String description() { return coffee.description() + ", sugar"; }
}
// Coffee coffee = new Milk(new Sugar(new SimpleCoffee()));
```

Facade

Simplified interface to complex system.

```
class CPU { void freeze() {} void jump(long position) {} void execute() {} }
class Memory { void load(long position, byte[] data) {} }
class HardDrive { byte[] read(long lba, int size) { return new byte[size]; } }
class ComputerFacade {
    private CPU cpu = new CPU();
    private Memory memory = new Memory();
    private HardDrive hardDrive = new HardDrive();
    public void start() {
        cpu.freeze();
        memory.load(0, hardDrive.read(0, 1024));
        cpu.jump(0);
        cpu.execute();
    }
}
```

Proxy

Controls access to object.

```
interface Image { void display(); }
class RealImage implements Image {
    private String filename;
    public RealImage(String filename) {
        this.filename = filename;
        loadFromDisk();
    }
    private void loadFromDisk() { System.out.println("Loading " + filename); }
    public void display() { System.out.println("Displaying " + filename); }
}
class ProxyImage implements Image {
    private RealImage realImage;
    private String filename;
    public ProxyImage(String filename) { this.filename = filename; }
    public void display() {
        if (realImage == null) realImage = new RealImage(filename);
        realImage.display();
    }
}
```

Composite

Tree structure of objects.

```

interface Component { void show(); }
class Leaf implements Component {
    private String name;
    public Leaf(String name) { this.name = name; }
    public void show() { System.out.println(name); }
}
class Composite implements Component {
    private List<Component> children = new ArrayList<>();
    public void add(Component c) { children.add(c); }
    public void show() { for (Component c : children) c.show(); }
}

```

3.3 Behavioral Patterns

Observer

One-to-many dependency notification.

```

interface Observer { void update(String message); }
class Subject {
    private List<Observer> observers = new ArrayList<>();
    public void attach(Observer o) { observers.add(o); }
    public void notifyObservers(String msg) {
        for (Observer o : observers) o.update(msg);
    }
}
class ConcreteObserver implements Observer {
    private String name;
    public ConcreteObserver(String name) { this.name = name; }
    public void update(String msg) { System.out.println(name + " received: " + msg); }
}

```

Strategy

Encapsulates algorithms, makes them interchangeable.

```

interface PaymentStrategy { void pay(int amount); }
class CreditCard implements PaymentStrategy {
    public void pay(int amount) { System.out.println("Paid " + amount + " using credit card"); }
}
class PayPal implements PaymentStrategy {
    public void pay(int amount) { System.out.println("Paid " + amount + " using PayPal"); }
}
class ShoppingCart {
    private PaymentStrategy paymentStrategy;
    public void setPaymentStrategy(PaymentStrategy ps) { this.paymentStrategy = ps; }
}

```

```
public void checkout(int amount) { paymentStrategy.pay(amount); }
}
```

Command

Encapsulates request as object.

```
interface Command { void execute(); }
class Light {
    public void on() { System.out.println("Light ON"); }
    public void off() { System.out.println("Light OFF"); }
}
class LightOnCommand implements Command {
    private Light light;
    public LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.on(); }
}
class RemoteControl {
    private Command command;
    public void setCommand(Command command) { this.command = command; }
    public void pressButton() { command.execute(); }
}
```

Template Method

Defines algorithm skeleton, subclasses override steps.

```
abstract class Game {
    abstract void initialize();
    abstract void startPlay();
    abstract void endPlay();
    public final void play() {
        initialize();
        startPlay();
        endPlay();
    }
}
class Cricket extends Game {
    void initialize() { System.out.println("Cricket initialized"); }
    void startPlay() { System.out.println("Cricket started"); }
    void endPlay() { System.out.println("Cricket ended"); }
}
```

State

Alters behavior when internal state changes.

```
interface State { void doAction(Context context); }
class StartState implements State {
    public void doAction(Context context) {
        System.out.println("In start state");
        context.setState(this);
    }
}
class StopState implements State {
    public void doAction(Context context) {
        System.out.println("In stop state");
        context.setState(this);
    }
}
class Context {
    private State state;
    public void setState(State state) { this.state = state; }
    public State getState() { return state; }
}
```

Chain of Responsibility

Passes request along chain of handlers.

```
abstract class Logger {
    public static int INFO = 1, DEBUG = 2, ERROR = 3;
    protected int level;
    protected Logger nextLogger;
    public void setNextLogger(Logger next) { this.nextLogger = next; }
    public void logMessage(int level, String message) {
        if (this.level <= level) write(message);
        if (nextLogger != null) nextLogger.logMessage(level, message);
    }
    abstract protected void write(String message);
}
class ConsoleLogger extends Logger {
    public ConsoleLogger(int level) { this.level = level; }
    protected void write(String message) { System.out.println("Console: " + message); }
}
```

Iterator

Sequential access to collection elements.

```

interface Iterator { boolean hasNext(); Object next(); }
interface Container { Iterator getIterator(); }
class NameRepository implements Container {
    public String[] names = {"A", "B", "C"};
    public Iterator getIterator() {
        return new Iterator() {
            int index;
            public boolean hasNext() { return index < names.length; }
            public Object next() { return hasNext() ? names[index++] : null; }
        };
    }
}

```

Mediator

Reduces coupling between objects by centralizing communication.

```

interface Mediator { void send(String msg, Colleague colleague); }
abstract class Colleague {
    protected Mediator mediator;
    public Colleague(Mediator mediator) { this.mediator = mediator; }
}
class ConcreteColleague1 extends Colleague {
    public ConcreteColleague1(Mediator mediator) { super(mediator); }
    public void send(String msg) { mediator.send(msg, this); }
    public void receive(String msg) { System.out.println("Colleague1 received: " + msg); }
}

```

Memento

Captures and restores object state.

```

class Memento {
    private String state;
    public Memento(String state) { this.state = state; }
    public String getState() { return state; }
}
class Originator {
    private String state;
    public void setState(String state) { this.state = state; }
    public Memento save() { return new Memento(state); }
    public void restore(Memento memento) { state = memento.getState(); }
}
class Caretaker {
    private List<Memento> mementos = new ArrayList<>();
    public void add(Memento m) { mementos.add(m); }
    public Memento get(int index) { return mementos.get(index); }
}

```

```
}
```

Visitor

Separates algorithm from object structure.

```
interface Visitor { void visit(Book book); void visit(Fruit fruit); }
interface Item { void accept(Visitor visitor); }
class Book implements Item {
    private int price;
    public Book(int price) { this.price = price; }
    public int getPrice() { return price; }
    public void accept(Visitor visitor) { visitor.visit(this); }
}
class ShoppingCartVisitor implements Visitor {
    public void visit(Book book) { System.out.println("Book: " + book.getPrice()); }
    public void visit(Fruit fruit) { System.out.println("Fruit: " + fruit.getPrice()); }
}
```

3.4 Interview Questions (50+)

Q1: What are design patterns? **A:** Reusable solutions to common software problems.

Q2: Categories of design patterns? **A:** Creational, Structural, Behavioral.

Q3: Singleton pattern use cases? **A:** Logger, Configuration, Database connection, Cache.

Q4: How to make Singleton thread-safe? **A:** Enum, double-checked locking, static holder, synchronized method.

Q5: Why enum singleton is best? **A:** Thread-safe by default, serialization safe, reflection safe.

Q6: Factory vs Abstract Factory? **A:** Factory: creates one type. Abstract Factory: creates families of related types.

Q7: Builder pattern benefits? **A:** Readable code, handles optional parameters, immutable objects.

Q8: Prototype pattern when to use? **A:** When object creation is costly, copy existing object faster.

Q9: Adapter pattern real example? **A:** Legacy code integration, third-party library integration.

Q10: Decorator vs Inheritance? **A:** Decorator: dynamic, flexible. Inheritance: static, compile-time.

Q11: Facade pattern benefits? **A:** Simplifies complex system, reduces dependencies.

Q12: Proxy pattern types? **A:** Virtual (lazy loading), Protection (access control), Remote (RMI).

Q13: Composite pattern use case? **A:** File system, GUI components, organization hierarchy.

- Q14:** Observer pattern real examples? **A:** Event listeners, MVC, Publish-Subscribe systems.
- Q15:** Strategy pattern vs State pattern? **A:** Strategy: algorithm selection. State: behavior changes with state.
- Q16:** Command pattern benefits? **A:** Undo/redo, macro recording, request queuing.
- Q17:** Template Method vs Strategy? **A:** Template: inheritance-based. Strategy: composition-based.
- Q18:** Chain of Responsibility use case? **A:** Logging frameworks, event handling, approval workflows.
- Q19:** Iterator pattern in Java? **A:** Collections framework uses it extensively.
- Q20:** Mediator pattern benefits? **A:** Reduces coupling, centralizes control, easier maintenance.
- Q21:** Memento pattern use case? **A:** Undo/redo, transaction rollback, checkpoint save.
- Q22:** Visitor pattern when to use? **A:** Operations on diverse object structure, open-closed principle.
- Q23:** Difference between Adapter and Bridge? **A:** Adapter: make incompatible interfaces work. Bridge: separate abstraction from implementation.
- Q24:** Flyweight pattern purpose? **A:** Memory optimization by sharing common data.
- Q25:** Interpreter pattern example? **A:** SQL parsing, expression evaluation, regex.
- Q26:** What is MVC pattern? **A:** Model-View-Controller. Separates data, UI, and logic.
- Q27:** DAO pattern purpose? **A:** Abstract data persistence layer.
- Q28:** Dependency Injection vs Service Locator? **A:** DI: injects dependencies. Service Locator: looks up dependencies.
- Q29:** Repository pattern vs DAO? **A:** Repository: domain-centric. DAO: data-centric.
- Q30:** What is Anti-pattern? **A:** Common but ineffective solution that causes problems.
- Q31:** God Object anti-pattern? **A:** Single class knowing/doing too much.
- Q32:** Spaghetti Code anti-pattern? **A:** Tangled, hard-to-follow control flow.
- Q33:** Golden Hammer anti-pattern? **A:** Using same solution for every problem.
- Q34:** Singleton anti-pattern concerns? **A:** Global state, testing difficulty, hidden dependencies.
- Q35:** When NOT to use Singleton? **A:** When state shouldn't be shared, need multiple instances.
- Q36:** Difference between Factory and Builder? **A:** Factory: simple creation. Builder: step-by-step construction.
- Q37:** Strategy pattern in Java Collections? **A:** Comparator interface.

- Q38:** Observer pattern in Java? **A:** Observable class, Observer interface, Event listeners.
- Q39:** Decorator pattern in Java IO? **A:** BufferedReader wrapping FileReader.
- Q40:** Proxy pattern in Spring? **A:** AOP proxies for transaction management.
- Q41:** What is Null Object pattern? **A:** Provides default behavior instead of null checks.
- Q42:** Object Pool pattern purpose? **A:** Reuse expensive-to-create objects (connections, threads).
- Q43:** Front Controller pattern? **A:** Single entry point for web requests.
- Q44:** Business Delegate pattern? **A:** Decouples presentation and business tiers.
- Q45:** Transfer Object (DTO) pattern? **A:** Carries data between processes/layers.
- Q46:** Intercepting Filter pattern? **A:** Pre/post-processing of requests (servlet filters).
- Q47:** What is SOLID principles? **A:** Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.
- Q48:** DRY principle? **A:** Don't Repeat Yourself.
- Q49:** KISS principle? **A:** Keep It Simple, Stupid.
- Q50:** YAGNI principle? **A:** You Aren't Gonna Need It (avoid over-engineering).

Chapter 4: Coding Challenges

4.1 String Problems (20)

Reverse String

```
String reverse(String s) {  
    return new StringBuilder(s).reverse().toString();  
}  
  
String reverseManual(String s) {  
    char[] arr = s.toCharArray();  
    int i = 0, j = arr.length - 1;  
    while (i < j) {  
        char temp = arr[i];  
        arr[i++] = arr[j];  
        arr[j--] = temp;  
    }  
    return new String(arr);  
}
```

```
}
```

Palindrome Check

```
boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i++) != s.charAt(j--)) return false;
    }
    return true;
}
```

Anagram Check

```
boolean isAnagram(String s1, String s2) {
    if (s1.length() != s2.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s1.length(); i++) {
        count[s1.charAt(i) - 'a']++;
        count[s2.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

First Non-Repeating Character

```
char firstNonRepeating(String s) {
    Map<Character, Integer> freq = new HashMap<>();
    for (char c : s.toCharArray()) freq.put(c, freq.getOrDefault(c, 0) + 1);
    for (char c : s.toCharArray()) if (freq.get(c) == 1) return c;
    return '_';
}
```

Longest Substring Without Repeating

```
int lengthOfLongestSubstring(String s) {
    Set<Character> set = new HashSet<>();
    int max = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        while (set.contains(s.charAt(right))) {
            set.remove(s.charAt(left++));
        }
        set.add(s.charAt(right));
    }
    return max;
}
```

```

        max = Math.max(max, right - left + 1);
    }
    return max;
}

```

Valid Parentheses

```

boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push('(');
        else if (c == '{') stack.push('{');
        else if (c == '[') stack.push('[');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}

```

String Compression

```

String compress(String s) {
    StringBuilder sb = new StringBuilder();
    int count = 1;
    for (int i = 1; i < s.length(); i++) {
        if (s.charAt(i) == s.charAt(i - 1)) count++;
        else {
            sb.append(s.charAt(i - 1)).append(count);
            count = 1;
        }
    }
    sb.append(s.charAt(s.length() - 1)).append(count);
    return sb.length() < s.length() ? sb.toString() : s;
}

```

Longest Common Prefix

```

String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    String prefix = strs[0];
    for (int i = 1; i < strs.length; i++) {
        while (strs[i].indexOf(prefix) != 0) {
            prefix = prefix.substring(0, prefix.length() - 1);
            if (prefix.isEmpty()) return "";
        }
    }
}

```

```

    return prefix;
}

```

Count and Say

```

String countAndSay(int n) {
    String s = "1";
    for (int i = 1; i < n; i++) {
        StringBuilder sb = new StringBuilder();
        int count = 1;
        for (int j = 1; j < s.length(); j++) {
            if (s.charAt(j) == s.charAt(j - 1)) count++;
            else {
                sb.append(count).append(s.charAt(j - 1));
                count = 1;
            }
        }
        sb.append(count).append(s.charAt(s.length() - 1));
        s = sb.toString();
    }
    return s;
}

```

Group Anagrams

```

List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] arr = s.toCharArray();
        Arrays.sort(arr);
        String key = new String(arr);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}

```

4.2 Array Problems (25)

Two Sum

```

int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) return new int[]{map.get(complement), i};
    }
}

```

```

        map.put(nums[i], i);
    }
    return new int[]{};
}

```

Maximum Subarray (Kadane's)

```

int maxSubArray(int[] nums) {
    int max = nums[0], current = nums[0];
    for (int i = 1; i < nums.length; i++) {
        current = Math.max(nums[i], current + nums[i]);
        max = Math.max(max, current);
    }
    return max;
}

```

Merge Sorted Arrays

```

void merge(int[] nums1, int m, int[] nums2, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
    while (j >= 0) {
        if (i >= 0 && nums1[i] > nums2[j]) nums1[k--] = nums1[i--];
        else nums1[k--] = nums2[j--];
    }
}

```

Remove Duplicates from Sorted Array

```

int removeDuplicates(int[] nums) {
    if (nums.length == 0) return 0;
    int i = 0;
    for (int j = 1; j < nums.length; j++) {
        if (nums[j] != nums[i]) nums[++i] = nums[j];
    }
    return i + 1;
}

```

Rotate Array

```

void rotate(int[] nums, int k) {
    k %= nums.length;
    reverse(nums, 0, nums.length - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, nums.length - 1);
}

```

```
}  
void reverse(int[] nums, int start, int end) {  
    while (start < end) {  
        int temp = nums[start];  
        nums[start++] = nums[end];  
        nums[end--] = temp;  
    }  
}
```

Product of Array Except Self

```
int[] productExceptSelf(int[] nums) {  
    int n = nums.length;  
    int[] result = new int[n];  
    result[0] = 1;  
    for (int i = 1; i < n; i++) result[i] = result[i - 1] * nums[i - 1];  
    int right = 1;  
    for (int i = n - 1; i >= 0; i--) {  
        result[i] *= right;  
        right *= nums[i];  
    }  
    return result;  
}
```

Find Missing Number

```
int missingNumber(int[] nums) {  
    int n = nums.length;  
    int expectedSum = n * (n + 1) / 2;  
    int actualSum = 0;  
    for (int num : nums) actualSum += num;  
    return expectedSum - actualSum;  
}
```

Move Zeroes

```
void moveZeroes(int[] nums) {  
    int index = 0;  
    for (int i = 0; i < nums.length; i++) {  
        if (nums[i] != 0) nums[index++] = nums[i];  
    }  
    while (index < nums.length) nums[index++] = 0;  
}
```

Container With Most Water

```
int maxArea(int[] height) {
    int max = 0, left = 0, right = height.length - 1;
    while (left < right) {
        int h = Math.min(height[left], height[right]);
        max = Math.max(max, h * (right - left));
        if (height[left] < height[right]) left++;
        else right--;
    }
    return max;
}
```

3Sum

```
List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> result = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue;
        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (sum == 0) {
                result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;
                left++;
                right--;
            } else if (sum < 0) left++;
            else right--;
        }
    }
    return result;
}
```

4.3 Math Problems (15)

Reverse Integer

```
int reverse(int x) {
    long result = 0;
    while (x != 0) {
        result = result * 10 + x % 10;
        x /= 10;
    }
}
```



```

return (result < Integer.MIN_VALUE || result > Integer.MAX_VALUE) ? 0 : (int) result;
}

```

Palindrome Number

```

boolean isPalindrome(int x) {
    if (x < 0) return false;
    int original = x, reversed = 0;
    while (x != 0) {
        reversed = reversed * 10 + x % 10;
        x /= 10;
    }
    return original == reversed;
}

```

Power of Two

```

boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}

```

Fizz Buzz

```

List<String> fizzBuzz(int n) {
    List<String> result = new ArrayList<>();
    for (int i = 1; i <= n; i++) {
        if (i % 15 == 0) result.add("FizzBuzz");
        else if (i % 3 == 0) result.add("Fizz");
        else if (i % 5 == 0) result.add("Buzz");
        else result.add(String.valueOf(i));
    }
    return result;
}

```

Count Primes (Sieve of Eratosthenes)

```

int countPrimes(int n) {
    if (n <= 2) return 0;
    boolean[] notPrime = new boolean[n];
    int count = 0;
    for (int i = 2; i < n; i++) {
        if (!notPrime[i]) {
            count++;
            for (int j = 2; i * j < n; j++) notPrime[i * j] = true;
        }
    }
    return count;
}

```

```

    }
}
return count;
}

```

Happy Number

```

boolean isHappy(int n) {
    Set<Integer> seen = new HashSet<>();
    while (n != 1 && !seen.contains(n)) {
        seen.add(n);
        n = sumOfSquares(n);
    }
    return n == 1;
}

int sumOfSquares(int n) {
    int sum = 0;
    while (n > 0) {
        int digit = n % 10;
        sum += digit * digit;
        n /= 10;
    }
    return sum;
}

```

Factorial

```

long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

long factorialIterative(int n) {
    long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}

```

GCD

```

int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}

```

LCM

```
int lcm(int a, int b) {  
    return (a * b) / gcd(a, b);  
}
```

Excel Sheet Column Number

```
int titleToNumber(String s) {  
    int result = 0;  
    for (char c : s.toCharArray()) {  
        result = result * 26 + (c - 'A' + 1);  
    }  
    return result;  
}
```

4.4 Recursion Problems (20)

Fibonacci

```
int fib(int n) {  
    if (n <= 1) return n;  
    return fib(n - 1) + fib(n - 2);  
}  
  
int fibDP(int n) {  
    if (n <= 1) return n;  
    int[] dp = new int[n + 1];  
    dp[1] = 1;  
    for (int i = 2; i <= n; i++) dp[i] = dp[i - 1] + dp[i - 2];  
    return dp[n];  
}
```

Climbing Stairs

```
int climbStairs(int n) {  
    if (n <= 2) return n;  
    int a = 1, b = 2;  
    for (int i = 3; i <= n; i++) {  
        int temp = a + b;  
        a = b;  
        b = temp;  
    }  
    return b;  
}
```

```
}
```

Generate Parentheses

```
List<String> generateParenthesis(int n) {
    List<String> result = new ArrayList<>();
    backtrack(result, "", 0, 0, n);
    return result;
}

void backtrack(List<String> result, String current, int open, int close, int max) {
    if (current.length() == max * 2) {
        result.add(current);
        return;
    }
    if (open < max) backtrack(result, current + "(", open + 1, close, max);
    if (close < open) backtrack(result, current + ")", open, close + 1, max);
}
```

Letter Combinations of Phone Number

```
List<String> letterCombinations(String digits) {
    if (digits.isEmpty()) return new ArrayList<>();
    String[] mapping = {"", "", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv", "wxyz"};
    List<String> result = new ArrayList<>();
    backtrack(result, mapping, digits, "", 0);
    return result;
}

void backtrack(List<String> result, String[] mapping, String digits, String current, int index) {
    if (index == digits.length()) {
        result.add(current);
        return;
    }
    String letters = mapping[digits.charAt(index) - '0'];
    for (char c : letters.toCharArray()) {
        backtrack(result, mapping, digits, current + c, index + 1);
    }
}
```

Permutations

```
List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(result, new ArrayList<>(), nums);
    return result;
}
```

```

void backtrack(List<List<Integer>> result, List<Integer> temp, int[] nums) {
    if (temp.size() == nums.length) {
        result.add(new ArrayList<>(temp));
        return;
    }
    for (int num : nums) {
        if (temp.contains(num)) continue;
        temp.add(num);
        backtrack(result, temp, nums);
        temp.remove(temp.size() - 1);
    }
}

```

Subsets

```

List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(result, new ArrayList<>(), nums, 0);
    return result;
}

void backtrack(List<List<Integer>> result, List<Integer> temp, int[] nums, int start) {
    result.add(new ArrayList<>(temp));
    for (int i = start; i < nums.length; i++) {
        temp.add(nums[i]);
        backtrack(result, temp, nums, i + 1);
        temp.remove(temp.size() - 1);
    }
}

```

Combination Sum

```

List<List<Integer>> combinationSum(int[] candidates, int target) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(result, new ArrayList<>(), candidates, target, 0);
    return result;
}

void backtrack(List<List<Integer>> result, List<Integer> temp, int[] candidates, int remain, int start) {
    if (remain < 0) return;
    if (remain == 0) {
        result.add(new ArrayList<>(temp));
        return;
    }
    for (int i = start; i < candidates.length; i++) {
        temp.add(candidates[i]);
        backtrack(result, temp, candidates, remain - candidates[i], i);
        temp.remove(temp.size() - 1);
    }
}

```

```
}
```

Word Search

```
boolean exist(char[][] board, String word) {
    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            if (dfs(board, word, i, j, 0)) return true;
        }
    }
    return false;
}

boolean dfs(char[][] board, String word, int i, int j, int index) {
    if (index == word.length()) return true;
    if (i < 0 || i >= board.length || j < 0 || j >= board[0].length || board[i][j] != word.charAt(index)) return false;
    char temp = board[i][j];
    board[i][j] = '#';
    boolean found = dfs(board, word, i + 1, j, index + 1) ||
                    dfs(board, word, i - 1, j, index + 1) ||
                    dfs(board, word, i, j + 1, index + 1) ||
                    dfs(board, word, i, j - 1, index + 1);
    board[i][j] = temp;
    return found;
}
```

N-Queens

```
List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>();
    char[][] board = new char[n][n];
    for (char[] row : board) Arrays.fill(row, '.');
    backtrack(result, board, 0);
    return result;
}

void backtrack(List<List<String>> result, char[][] board, int row) {
    if (row == board.length) {
        result.add(construct(board));
        return;
    }
    for (int col = 0; col < board.length; col++) {
        if (isValid(board, row, col)) {
            board[row][col] = 'Q';
            backtrack(result, board, row + 1);
            board[row][col] = '.';
        }
    }
}
```

```

boolean isValid(char[][] board, int row, int col) {
    for (int i = 0; i < row; i++) if (board[i][col] == 'Q') return false;
    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) if (board[i][j] == 'Q') return false;
    for (int i = row - 1, j = col + 1; i >= 0 && j < board.length; i--, j++) if (board[i][j] == 'Q') return false;
    return true;
}

List<String> construct(char[][] board) {
    List<String> result = new ArrayList<>();
    for (char[] row : board) result.add(new String(row));
    return result;
}

```

Sudoku Solver

```

void solveSudoku(char[][] board) {
    solve(board);
}

boolean solve(char[][] board) {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (board[i][j] == '.') {
                for (char c = '1'; c <= '9'; c++) {
                    if (isValidSudoku(board, i, j, c)) {
                        board[i][j] = c;
                        if (solve(board)) return true;
                        board[i][j] = '.';
                    }
                }
                return false;
            }
        }
    }
    return true;
}

boolean isValidSudoku(char[][] board, int row, int col, char c) {
    for (int i = 0; i < 9; i++) {
        if (board[row][i] == c || board[i][col] == c ||
            board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] == c) return false;
    }
    return true;
}

```

4.5 Bit Manipulation (10)

Single Number

```
int singleNumber(int[] nums) {  
    int result = 0;  
    for (int num : nums) result ^= num;  
    return result;  
}
```

Number of 1 Bits

```
int hammingWeight(int n) {  
    int count = 0;  
    while (n != 0) {  
        count++;  
        n &= (n - 1);  
    }  
    return count;  
}
```

Reverse Bits

```
int reverseBits(int n) {  
    int result = 0;  
    for (int i = 0; i < 32; i++) {  
        result = (result << 1) | (n & 1);  
        n >>= 1;  
    }  
    return result;  
}
```

Missing Number (XOR)

```
int missingNumber(int[] nums) {  
    int xor = nums.length;  
    for (int i = 0; i < nums.length; i++) xor ^= i ^ nums[i];  
    return xor;  
}
```


Power of Four

```
boolean isPowerOfFour(int n) {
    return n > 0 && (n & (n - 1)) == 0 && (n & 0xAAAAAAAA) == 0;
}
```

Chapter 5: Data Structures & Algorithms

5.1 Arrays & Hashing (15 problems)

Contains Duplicate

```
boolean containsDuplicate(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums) {
        if (!set.add(num)) return true;
    }
    return false;
}
```

Valid Anagram

```
boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < s.length(); i++) {
        count[s.charAt(i) - 'a']++;
        count[t.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}
```

Top K Frequent Elements

```
int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> count = new HashMap<>();
    for (int num : nums) count.put(num, count.getOrDefault(num, 0) + 1);
    PriorityQueue<Integer> heap = new PriorityQueue<>((a, b) -> count.get(a) - count.get(b));
    for (int num : count.keySet()) {
        heap.offer(num);
        if (heap.size() > k) heap.poll();
    }
    int[] result = new int[k];
```

```

for (int i = k - 1; i >= 0; i--) result[i] = heap.poll();
return result;
}

```

Encode and Decode Strings

```

String encode(List<String> strs) {
    StringBuilder sb = new StringBuilder();
    for (String s : strs) sb.append(s.length()).append("#").append(s);
    return sb.toString();
}

List<String> decode(String s) {
    List<String> result = new ArrayList<>();
    int i = 0;
    while (i < s.length()) {
        int j = i;
        while (s.charAt(j) != '#') j++;
        int len = Integer.parseInt(s.substring(i, j));
        result.add(s.substring(j + 1, j + 1 + len));
        i = j + 1 + len;
    }
    return result;
}

```

Longest Consecutive Sequence

```

int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums) set.add(num);
    int longest = 0;
    for (int num : nums) {
        if (!set.contains(num - 1)) {
            int length = 1;
            while (set.contains(num + length)) length++;
            longest = Math.max(longest, length);
        }
    }
    return longest;
}

```

5.2 Two Pointers (10 problems)

Valid Palindrome

```
boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
        if (Character.toLowerCase(s.charAt(left++)) != Character.toLowerCase(s.charAt(right--))) return false;
    }
    return true;
}
```

Two Sum II (Sorted)

```
int[] twoSum(int[] numbers, int target) {
    int left = 0, right = numbers.length - 1;
    while (left < right) {
        int sum = numbers[left] + numbers[right];
        if (sum == target) return new int[] {left + 1, right + 1};
        else if (sum < target) left++;
        else right--;
    }
    return new int[] {};
}
```

3Sum Closest

```
int threeSumClosest(int[] nums, int target) {
    Arrays.sort(nums);
    int closest = nums[0] + nums[1] + nums[2];
    for (int i = 0; i < nums.length - 2; i++) {
        int left = i + 1, right = nums.length - 1;
        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];
            if (Math.abs(target - sum) < Math.abs(target - closest)) closest = sum;
            if (sum < target) left++;
            else right--;
        }
    }
    return closest;
}
```

Trapping Rain Water

```
int trap(int[] height) {
    int left = 0, right = height.length - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left < right) {
        if (height[left] < height[right]) {
            if (height[left] >= leftMax) leftMax = height[left];
            else water += leftMax - height[left];
            left++;
        } else {
            if (height[right] >= rightMax) rightMax = height[right];
            else water += rightMax - height[right];
            right--;
        }
    }
    return water;
}
```

5.3 Sliding Window (8 problems)

Best Time to Buy and Sell Stock

```
int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE, maxProfit = 0;
    for (int price : prices) {
        if (price < minPrice) minPrice = price;
        else maxProfit = Math.max(maxProfit, price - minPrice);
    }
    return maxProfit;
}
```

Longest Repeating Character Replacement

```
int characterReplacement(String s, int k) {
    int[] count = new int[26];
    int maxCount = 0, maxLength = 0, start = 0;
    for (int end = 0; end < s.length(); end++) {
        maxCount = Math.max(maxCount, ++count[s.charAt(end) - 'A']);
        while (end - start + 1 - maxCount > k) {
            count[s.charAt(start++) - 'A']--;
        }
        maxLength = Math.max(maxLength, end - start + 1);
    }
    return maxLength;
}
```

```
}
```

Minimum Window Substring

```
String minWindow(String s, String t) {
    if (s.length() == 0 || t.length() == 0) return "";
    Map<Character, Integer> dictT = new HashMap<>();
    for (char c : t.toCharArray()) dictT.put(c, dictT.getOrDefault(c, 0) + 1);
    int required = dictT.size(), formed = 0;
    Map<Character, Integer> windowCounts = new HashMap<>();
    int l = 0, r = 0, minLen = Integer.MAX_VALUE, minLeft = 0;
    while (r < s.length()) {
        char c = s.charAt(r);
        windowCounts.put(c, windowCounts.getOrDefault(c, 0) + 1);
        if (dictT.containsKey(c) && windowCounts.get(c).intValue() == dictT.get(c).intValue()) formed++;
        while (l <= r && formed == required) {
            c = s.charAt(l);
            if (r - l + 1 < minLen) {
                minLen = r - l + 1;
                minLeft = l;
            }
            windowCounts.put(c, windowCounts.get(c) - 1);
            if (dictT.containsKey(c) && windowCounts.get(c) < dictT.get(c)) formed--;
            l++;
        }
        r++;
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(minLeft, minLeft + minLen);
}
```

5.4 Stack (10 problems)

Valid Parentheses

```
boolean isValid(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(') stack.push('(');
        else if (c == '{') stack.push('{');
        else if (c == '[') stack.push('[');
        else if (stack.isEmpty() || stack.pop() != c) return false;
    }
    return stack.isEmpty();
}
```

Min Stack

```
class MinStack {
    Stack<Integer> stack = new Stack<>();
    Stack<Integer> minStack = new Stack<>();
    public void push(int val) {
        stack.push(val);
        val = Math.min(val, minStack.isEmpty() ? val : minStack.peek());
        minStack.push(val);
    }
    public void pop() {
        stack.pop();
        minStack.pop();
    }
    public int top() { return stack.peek(); }
    public int getMin() { return minStack.peek(); }
}
```

Evaluate Reverse Polish Notation

```
int evalRPN(String[] tokens) {
    Stack<Integer> stack = new Stack<>();
    for (String token : tokens) {
        if ("+".equals(token)) stack.push(stack.pop() + stack.pop());
        else if ("-".equals(token)) {
            int b = stack.pop(), a = stack.pop();
            stack.push(a - b);
        } else if ("*".equals(token)) stack.push(stack.pop() * stack.pop());
        else if ("/".equals(token)) {
            int b = stack.pop(), a = stack.pop();
            stack.push(a / b);
        } else stack.push(Integer.parseInt(token));
    }
    return stack.pop();
}
```

Daily Temperatures

```
int[] dailyTemperatures(int[] temperatures) {
    int n = temperatures.length;
    int[] result = new int[n];
    Stack<Integer> stack = new Stack<>();
    for (int i = 0; i < n; i++) {
        while (!stack.isEmpty() && temperatures[i] > temperatures[stack.peek()]) {
            int idx = stack.pop();
            result[idx] = i - idx;
        }
    }
}
```

```

    stack.push(i);
}
return result;
}

```

Largest Rectangle in Histogram

```

int largestRectangleArea(int[] heights) {
    Stack<Integer> stack = new Stack<>();
    int maxArea = 0;
    for (int i = 0; i <= heights.length; i++) {
        int h = (i == heights.length) ? 0 : heights[i];
        while (!stack.isEmpty() && h < heights[stack.peek()]) {
            int height = heights[stack.pop()];
            int width = stack.isEmpty() ? i : i - stack.peek() - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}

```

5.5 Binary Search (10 problems)

Binary Search

```

int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

```

Search in Rotated Sorted Array

```

int search(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        if (nums[left] <= nums[mid]) {

```

```

        if (target >= nums[left] && target < nums[mid]) right = mid - 1;
        else left = mid + 1;
    } else {
        if (target > nums[mid] && target <= nums[right]) left = mid + 1;
        else right = mid - 1;
    }
}
return -1;
}

```

Find Minimum in Rotated Sorted Array

```

int findMin(int[] nums) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[right]) left = mid + 1;
        else right = mid;
    }
    return nums[left];
}

```

Search a 2D Matrix

```

boolean searchMatrix(int[][] matrix, int target) {
    int m = matrix.length, n = matrix[0].length;
    int left = 0, right = m * n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        int midVal = matrix[mid / n][mid % n];
        if (midVal == target) return true;
        if (midVal < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}

```

Koko Eating Bananas

```

int minEatingSpeed(int[] piles, int h) {
    int left = 1, right = Arrays.stream(piles).max().getAsInt();
    while (left < right) {
        int mid = left + (right - left) / 2;
        int hours = 0;
        for (int pile : piles) hours += (pile + mid - 1) / mid;
        if (hours <= h) right = mid;
    }
}

```



```

    else left = mid + 1;
  }
  return left;
}

```

5.6 Linked List (12 problems)

Reverse Linked List

```

ListNode reverseList(ListNode head) {
  ListNode prev = null;
  while (head != null) {
    ListNode next = head.next;
    head.next = prev;
    prev = head;
    head = next;
  }
  return prev;
}

```

Merge Two Sorted Lists

```

ListNode mergeTwoLists(ListNode l1, ListNode l2) {
  ListNode dummy = new ListNode(0), curr = dummy;
  while (l1 != null && l2 != null) {
    if (l1.val < l2.val) {
      curr.next = l1;
      l1 = l1.next;
    } else {
      curr.next = l2;
      l2 = l2.next;
    }
    curr = curr.next;
  }
  curr.next = (l1 != null) ? l1 : l2;
  return dummy.next;
}

```

Linked List Cycle

```

boolean hasCycle(ListNode head) {
  ListNode slow = head, fast = head;
  while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
  }
}

```

```

    if (slow == fast) return true;
}
return false;
}

```

Remove Nth Node From End

```

ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode first = dummy, second = dummy;
    for (int i = 0; i <= n; i++) first = first.next;
    while (first != null) {
        first = first.next;
        second = second.next;
    }
    second.next = second.next.next;
    return dummy.next;
}

```

Reorder List

```

void reorderList(ListNode head) {
    if (head == null || head.next == null) return;
    ListNode slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    ListNode second = slow.next;
    slow.next = null;
    second = reverse(second);
    ListNode first = head;
    while (second != null) {
        ListNode temp1 = first.next, temp2 = second.next;
        first.next = second;
        second.next = temp1;
        first = temp1;
        second = temp2;
    }
}

ListNode reverse(ListNode head) {
    ListNode prev = null;
    while (head != null) {
        ListNode next = head.next;
        head.next = prev;
        prev = head;
    }
}

```

```

    head = next;
}
return prev;
}

```

Add Two Numbers

```

ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0), curr = dummy;
    int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int sum = carry;
        if (l1 != null) { sum += l1.val; l1 = l1.next; }
        if (l2 != null) { sum += l2.val; l2 = l2.next; }
        carry = sum / 10;
        curr.next = new ListNode(sum % 10);
        curr = curr.next;
    }
    return dummy.next;
}

```

Copy List with Random Pointer

```

Node copyRandomList(Node head) {
    if (head == null) return null;
    Map<Node, Node> map = new HashMap<>();
    Node curr = head;
    while (curr != null) {
        map.put(curr, new Node(curr.val));
        curr = curr.next;
    }
    curr = head;
    while (curr != null) {
        map.get(curr).next = map.get(curr.next);
        map.get(curr).random = map.get(curr.random);
        curr = curr.next;
    }
    return map.get(head);
}

```

LRU Cache

```

class LRUCache {
    class Node {
        int key, val;
        Node prev, next;
    }
}

```

```

    Node(int k, int v) { key = k; val = v; }
}
Map<Integer, Node> map = new HashMap<>();
Node head = new Node(0, 0), tail = new Node(0, 0);
int capacity;
public LRUCache(int capacity) {
    this.capacity = capacity;
    head.next = tail;
    tail.prev = head;
}
public int get(int key) {
    if (!map.containsKey(key)) return -1;
    Node node = map.get(key);
    remove(node);
    insert(node);
    return node.val;
}
public void put(int key, int value) {
    if (map.containsKey(key)) remove(map.get(key));
    if (map.size() == capacity) remove(tail.prev);
    insert(new Node(key, value));
}
void remove(Node node) {
    map.remove(node.key);
    node.prev.next = node.next;
    node.next.prev = node.prev;
}
void insert(Node node) {
    map.put(node.key, node);
    node.next = head.next;
    node.next.prev = node;
    head.next = node;
    node.prev = head;
}
}

```

5.7 Trees (15 problems)

Maximum Depth of Binary Tree

```

int maxDepth(TreeNode root) {
    return root == null ? 0 : 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}

```

Same Tree

```
boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null) return false;
    return p.val == q.val && isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
}
```

Invert Binary Tree

```
TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = invertTree(root.right);
    root.right = invertTree(temp);
    return root;
}
```

Binary Tree Level Order Traversal

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
    return result;
}
```

Validate Binary Search Tree

```
boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}
boolean validate(TreeNode node, Integer low, Integer high) {
    if (node == null) return true;
```

```

if ((low != null && node.val <= low) || (high != null && node.val >= high)) return false;
return validate(node.left, low, node.val) && validate(node.right, node.val, high);
}

```

Lowest Common Ancestor of BST

```

TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (p.val < root.val && q.val < root.val) return lowestCommonAncestor(root.left, p, q);
    if (p.val > root.val && q.val > root.val) return lowestCommonAncestor(root.right, p, q);
    return root;
}

```

Binary Tree Right Side View

```

List<Integer> rightSideView(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;
    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            if (i == size - 1) result.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
    }
    return result;
}

```

Kth Smallest Element in BST

```

int kthSmallest(TreeNode root, int k) {
    Stack<TreeNode> stack = new Stack<>();
    while (true) {
        while (root != null) {
            stack.push(root);
            root = root.left;
        }
        root = stack.pop();
        if (--k == 0) return root.val;
        root = root.right;
    }
}

```

Serialize and Deserialize Binary Tree

```
String serialize(TreeNode root) {
    if (root == null) return "null,";
    return root.val + "," + serialize(root.left) + serialize(root.right);
}

TreeNode deserialize(String data) {
    Queue<String> queue = new LinkedList<>(Arrays.asList(data.split(",")));
    return buildTree(queue);
}

TreeNode buildTree(Queue<String> queue) {
    String val = queue.poll();
    if ("null".equals(val)) return null;
    TreeNode node = new TreeNode(Integer.parseInt(val));
    node.left = buildTree(queue);
    node.right = buildTree(queue);
    return node;
}
```

5.8 Tries (3 problems)

Implement Trie

```
class Trie {
    class TrieNode {
        TrieNode[] children = new TrieNode[26];
        boolean isEnd;
    }
    TrieNode root = new TrieNode();
    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children[c - 'a'] == null) node.children[c - 'a'] = new TrieNode();
            node = node.children[c - 'a'];
        }
        node.isEnd = true;
    }
    public boolean search(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            node = node.children[c - 'a'];
            if (node == null) return false;
        }
        return node.isEnd;
    }
    public boolean startsWith(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
```

```

        node = node.children[c - 'a'];
        if (node == null) return false;
    }
    return true;
}
}

```

Word Search II

```

List<String> findWords(char[][] board, String[] words) {
    TrieNode root = buildTrie(words);
    List<String> result = new ArrayList<>();
    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            dfs(board, i, j, root, result);
        }
    }
    return result;
}

void dfs(char[][] board, int i, int j, TrieNode node, List<String> result) {
    char c = board[i][j];
    if (c == '#' || node.children[c - 'a'] == null) return;
    node = node.children[c - 'a'];
    if (node.word != null) {
        result.add(node.word);
        node.word = null;
    }
    board[i][j] = '#';
    if (i > 0) dfs(board, i - 1, j, node, result);
    if (j > 0) dfs(board, i, j - 1, node, result);
    if (i < board.length - 1) dfs(board, i + 1, j, node, result);
    if (j < board[0].length - 1) dfs(board, i, j + 1, node, result);
    board[i][j] = c;
}

TrieNode buildTrie(String[] words) {
    TrieNode root = new TrieNode();
    for (String word : words) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (node.children[c - 'a'] == null) node.children[c - 'a'] = new TrieNode();
            node = node.children[c - 'a'];
        }
        node.word = word;
    }
    return root;
}

class TrieNode {
    TrieNode[] children = new TrieNode[26];
}

```



```
String word;
}
```

5.9 Heaps / Priority Queue (5 problems)

Kth Largest Element

```
int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();
    for (int num : nums) {
        heap.offer(num);
        if (heap.size() > k) heap.poll();
    }
    return heap.peek();
}
```

Merge K Sorted Lists

```
ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> heap = new PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode list : lists) if (list != null) heap.offer(list);
    ListNode dummy = new ListNode(0), curr = dummy;
    while (!heap.isEmpty()) {
        ListNode node = heap.poll();
        curr.next = node;
        curr = curr.next;
        if (node.next != null) heap.offer(node.next);
    }
    return dummy.next;
}
```

Find Median from Data Stream

```
class MedianFinder {
    PriorityQueue<Integer> small = new PriorityQueue<>(Collections.reverseOrder());
    PriorityQueue<Integer> large = new PriorityQueue<>();
    public void addNum(int num) {
        small.offer(num);
        large.offer(small.poll());
        if (small.size() < large.size()) small.offer(large.poll());
    }
    public double findMedian() {
        return small.size() > large.size() ? small.peek() : (small.peek() + large.peek()) / 2.0;
    }
}
```

```
}
```

5.10 Graphs (12 problems)

Number of Islands

```
int numIslands(char[][] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                count++;
            }
        }
    }
    return count;
}

void dfs(char[][] grid, int i, int j) {
    if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length || grid[i][j] == '0') return;
    grid[i][j] = '0';
    dfs(grid, i + 1, j);
    dfs(grid, i - 1, j);
    dfs(grid, i, j + 1);
    dfs(grid, i, j - 1);
}
```

Clone Graph

```
Node cloneGraph(Node node) {
    if (node == null) return null;
    Map<Node, Node> map = new HashMap<>();
    return dfs(node, map);
}

Node dfs(Node node, Map<Node, Node> map) {
    if (map.containsKey(node)) return map.get(node);
    Node clone = new Node(node.val);
    map.put(node, clone);
    for (Node neighbor : node.neighbors) clone.neighbors.add(dfs(neighbor, map));
    return clone;
}
```

Pacific Atlantic Water Flow

```
List<List<Integer>> pacificAtlantic(int[][] heights) {
    int m = heights.length, n = heights[0].length;
    boolean[][] pacific = new boolean[m][n], atlantic = new boolean[m][n];
    for (int i = 0; i < m; i++) {
        dfs(heights, pacific, i, 0);
        dfs(heights, atlantic, i, n - 1);
    }
    for (int j = 0; j < n; j++) {
        dfs(heights, pacific, 0, j);
        dfs(heights, atlantic, m - 1, j);
    }
    List<List<Integer>> result = new ArrayList<>();
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (pacific[i][j] && atlantic[i][j]) result.add(Arrays.asList(i, j));
        }
    }
    return result;
}

void dfs(int[][] heights, boolean[][] visited, int i, int j) {
    if (visited[i][j]) return;
    visited[i][j] = true;
    int[][] dirs = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
    for (int[] dir : dirs) {
        int x = i + dir[0], y = j + dir[1];
        if (x >= 0 && x < heights.length && y >= 0 && y < heights[0].length && heights[x][y] >= heights[i][j]) {
            dfs(heights, visited, x, y);
        }
    }
}
}
```

Course Schedule

```
boolean canFinish(int numCourses, int[][] prerequisites) {
    List<Integer>[] graph = new ArrayList[numCourses];
    for (int i = 0; i < numCourses; i++) graph[i] = new ArrayList<>();
    for (int[] pre : prerequisites) graph[pre[1]].add(pre[0]);
    int[] visited = new int[numCourses];
    for (int i = 0; i < numCourses; i++) {
        if (dfs(graph, visited, i)) return false;
    }
    return true;
}

boolean dfs(List<Integer>[] graph, int[] visited, int i) {
    if (visited[i] == 1) return true;
    if (visited[i] == 2) return false;
```

```

visited[i] = 1;
for (int next : graph[i]) {
    if (dfs(graph, visited, next)) return true;
}
visited[i] = 2;
return false;
}

```

Course Schedule II

```

int[] findOrder(int numCourses, int[][] prerequisites) {
    List<Integer>[] graph = new ArrayList[numCourses];
    for (int i = 0; i < numCourses; i++) graph[i] = new ArrayList<>();
    for (int[] pre : prerequisites) graph[pre[1]].add(pre[0]);
    int[] visited = new int[numCourses];
    Stack<Integer> stack = new Stack<>();
    for (int i = 0; i < numCourses; i++) {
        if (dfs(graph, visited, i, stack)) return new int[0];
    }
    int[] result = new int[numCourses];
    for (int i = 0; i < numCourses; i++) result[i] = stack.pop();
    return result;
}

boolean dfs(List<Integer>[] graph, int[] visited, int i, Stack<Integer> stack) {
    if (visited[i] == 1) return true;
    if (visited[i] == 2) return false;
    visited[i] = 1;
    for (int next : graph[i]) {
        if (dfs(graph, visited, next, stack)) return true;
    }
    visited[i] = 2;
    stack.push(i);
    return false;
}

```

Chapter 6: Backend Engineering

6.1 Node.js Fundamentals

Event Loop

```

// Event loop phases: Timers → Pending → Idle → Poll → Check → Close
console.log('1');
setTimeout(() => console.log('2'), 0);
setImmediate(() => console.log('3'));
process.nextTick(() => console.log('4'));
Promise.resolve().then(() => console.log('5'));
console.log('6');

```

```
// Output: 1 → 6 → 4 → 5 → 2 → 3
```

Event Emitter

```
const EventEmitter = require('events');
class MyEmitter extends EventEmitter {}
const emitter = new MyEmitter();
emitter.on('event', (data) => console.log(data));
emitter.emit('event', 'Hello World');
```

Streams

```
const fs = require('fs');
// Readable
const readable = fs.createReadStream('input.txt');
readable.on('data', chunk => console.log(chunk.toString()));
// Writable
const writable = fs.createWriteStream('output.txt');
writable.write('Hello\n');
writable.end();
// Pipe
readable.pipe(writable);
```

Buffer

```
const buf = Buffer.from('Hello');
console.log(buf); // <Buffer 48 65 6c 6c 6f>
console.log(buf.toString()); // Hello
const buf2 = Buffer.alloc(10);
buf2.write('World');
```

Cluster Module

```
const cluster = require('cluster');
const os = require('os');
if (cluster.isMaster) {
  for (let i = 0; i < os.cpus().length; i++) cluster.fork();
  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.id} died`);
    cluster.fork();
  });
} else {
  require('./server.js');
```

```
}
```

6.2 Express.js

Basic Server

```
const express = require('express');
const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.get('/', (req, res) => res.send('Hello'));
app.listen(3000, () => console.log('Server running'));
```

Middleware

```
// Logger middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});
// Error handling
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});
// Authentication middleware
const auth = (req, res, next) => {
  const token = req.headers.authorization;
  if (!token) return res.status(401).send('Unauthorized');
  try {
    const decoded = jwt.verify(token, SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    res.status(401).send('Invalid token');
  }
};
```

Routing

```
const router = express.Router();
router.get('/users', (req, res) => {});
router.get('/users/:id', (req, res) => {
  const { id } = req.params;
  res.json({ id });
});
```

```

});
router.post('/users', (req, res) => {
  const { name, email } = req.body;
  res.status(201).json({ name, email });
});
router.put('/users/:id', (req, res) => {});
router.delete('/users/:id', (req, res) => {});
app.use('/api', router);

```

File Upload

```

const multer = require('multer');
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename: (req, file, cb) => cb(null, Date.now() + '-' + file.originalname)
});
const upload = multer({ storage });
app.post('/upload', upload.single('file'), (req, res) => {
  res.json({ filename: req.file.filename });
});

```

6.3 REST API Design

CRUD Operations

```

// GET all
app.get('/api/users', async (req, res) => {
  const users = await User.findAll();
  res.json(users);
});
// GET one
app.get('/api/users/:id', async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json(user);
});
// POST create
app.post('/api/users', async (req, res) => {
  const user = await User.create(req.body);
  res.status(201).json(user);
});
// PUT update
app.put('/api/users/:id', async (req, res) => {
  const user = await User.update(req.params.id, req.body);
  res.json(user);
});

```

```
// DELETE
app.delete('/api/users/:id', async (req, res) => {
  await User.delete(req.params.id);
  res.status(204).send();
});
```

Pagination & Filtering

```
app.get('/api/users', async (req, res) => {
  const { page = 1, limit = 10, sort = 'createdAt', order = 'desc' } = req.query;
  const offset = (page - 1) * limit;
  const users = await User.findAndCountAll({
    limit: parseInt(limit),
    offset: parseInt(offset),
    order: [[sort, order]]
  });
  res.json({
    data: users.rows,
    total: users.count,
    page: parseInt(page),
    pages: Math.ceil(users.count / limit)
  });
});
```

Error Handling

```
class ApiError extends Error {
  constructor(statusCode, message) {
    super(message);
    this.statusCode = statusCode;
  }
}

app.use((err, req, res, next) => {
  const { statusCode = 500, message } = err;
  res.status(statusCode).json({
    status: 'error',
    statusCode,
    message
  });
});
```


6.4 Authentication & Authorization

JWT Authentication

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
// Register
app.post('/register', async (req, res) => {
  const { email, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await User.create({ email, password: hashedPassword });
  res.status(201).json({ id: user.id, email: user.email });
});
// Login
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ where: { email } });
  if (!user) return res.status(401).json({ error: 'Invalid credentials' });
  const valid = await bcrypt.compare(password, user.password);
  if (!valid) return res.status(401).json({ error: 'Invalid credentials' });
  const token = jwt.sign({ id: user.id, email: user.email }, SECRET, { expiresIn: '1d' });
  res.json({ token });
});
// Protected route
app.get('/profile', auth, (req, res) => {
  res.json(req.user);
});
```

Session-based Auth

```
const session = require('express-session');
app.use(session({
  secret: 'secret',
  resave: false,
  saveUninitialized: false,
  cookie: { secure: false, maxAge: 3600000 }
}));
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ where: { email } });
  if (user && await bcrypt.compare(password, user.password)) {
    req.session.userId = user.id;
    res.json({ message: 'Logged in' });
  } else {
    res.status(401).json({ error: 'Invalid credentials' });
  }
});
app.post('/logout', (req, res) => {
```

```
req.session.destroy();
res.json({ message: 'Logged out' });
});
```

OAuth 2.0

```
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;
passport.use(new GoogleStrategy({
  clientID: GOOGLE_CLIENT_ID,
  clientSecret: GOOGLE_CLIENT_SECRET,
  callbackURL: '/auth/google/callback'
}, (accessToken, refreshToken, profile, done) => {
  User.findOrCreate({ googleId: profile.id }, (err, user) => done(err, user));
}));
app.get('/auth/google', passport.authenticate('google', { scope: ['profile', 'email'] }));
app.get('/auth/google/callback', passport.authenticate('google', { failureRedirect: '/login' }), (req, res) => {
  res.redirect('/dashboard');
});
```

6.5 Database - SQL (PostgreSQL)

Basic Queries

```
-- CREATE
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- INSERT
INSERT INTO users (email, name) VALUES ('john@example.com', 'John Doe');
-- SELECT
SELECT * FROM users WHERE email = 'john@example.com';
-- UPDATE
UPDATE users SET name = 'Jane Doe' WHERE id = 1;
-- DELETE
DELETE FROM users WHERE id = 1;
```

Joins

```
-- INNER JOIN
SELECT users.name, orders.total
FROM users INNER JOIN orders ON users.id = orders.user_id;
```

```
-- LEFT JOIN
SELECT users.name, orders.total
FROM users LEFT JOIN orders ON users.id = orders.user_id;
-- RIGHT JOIN
SELECT users.name, orders.total
FROM users RIGHT JOIN orders ON users.id = orders.user_id;
```

Aggregations

```
SELECT COUNT(*) FROM users;
SELECT AVG(price) FROM products;
SELECT SUM(total) FROM orders;
SELECT MAX(price), MIN(price) FROM products;
SELECT category, COUNT(*) FROM products GROUP BY category;
SELECT category, AVG(price) FROM products GROUP BY category HAVING AVG(price) > 100;
```

Indexes

```
CREATE INDEX idx_users_email ON users(email);
CREATE UNIQUE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_name_email ON users(name, email);
DROP INDEX idx_users_email;
```

Transactions

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
-- Rollback on error
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
-- Error occurs
ROLLBACK;
```

6.6 Prisma ORM

Schema Definition

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
generator client {
```

```

provider = "prisma-client-js"
}
model User {
  id    Int    @id @default(autoincrement())
  email String @unique
  name  String?
  posts Post[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
model Post {
  id    Int    @id @default(autoincrement())
  title String
  content String?
  published Boolean @default(false)
  author User   @relation(fields: [authorId], references: [id])
  authorId Int
}

```

CRUD Operations

```

const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
// Create
const user = await prisma.user.create({
  data: { email: 'john@example.com', name: 'John' }
});
// Read
const users = await prisma.user.findMany();
const user = await prisma.user.findUnique({ where: { email: 'john@example.com' } });
// Update
const updated = await prisma.user.update({
  where: { id: 1 },
  data: { name: 'Jane' }
});
// Delete
await prisma.user.delete({ where: { id: 1 } });

```

Relations

```

// Create with relation
const user = await prisma.user.create({
  data: {
    email: 'john@example.com',
    posts: {
      create: [
        { title: 'Post 1', content: 'Content 1' },

```

```

    { title: 'Post 2', content: 'Content 2' }
  ]
}
});
// Query with relations
const userWithPosts = await prisma.user.findUnique({
  where: { id: 1 },
  include: { posts: true }
});

```

Filtering & Sorting

```

const users = await prisma.user.findMany({
  where: {
    email: { contains: '@example.com' },
    name: { startsWith: 'J' }
  },
  orderBy: { createdAt: 'desc' },
  skip: 10,
  take: 10
});

```

6.7 MongoDB & Mongoose

Schema Definition

```

const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  name: String,
  age: { type: Number, min: 0, max: 120 },
  createdAt: { type: Date, default: Date.now }
});
const User = mongoose.model('User', userSchema);

```

CRUD Operations

```

// Create
const user = new User({ email: 'john@example.com', name: 'John' });
await user.save();
// Or
await User.create({ email: 'jane@example.com', name: 'Jane' });
// Read
const users = await User.find();

```

```

const user = await User.findOne({ email: 'john@example.com' });
const userById = await User.findById(id);
// Update
await User.updateOne({ _id: id }, { name: 'Updated' });
await User.findByIdAndUpdate(id, { name: 'Updated' }, { new: true });
// Delete
await User.deleteOne({ _id: id });
await User.findByIdAndDelete(id);

```

Queries

```

// Find with conditions
const users = await User.find({ age: { $gte: 18 } });
const users = await User.find({ name: /john/i });
// Sorting & Pagination
const users = await User.find().sort({ createdAt: -1 }).skip(10).limit(10);
// Select fields
const users = await User.find().select('name email -_id');

```

Population (Joins)

```

const postSchema = new mongoose.Schema({
  title: String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
});
const Post = mongoose.model('Post', postSchema);
// Populate
const posts = await Post.find().populate('author');
const posts = await Post.find().populate('author', 'name email');

```

6.8 Caching (Redis)

Basic Operations

```

const redis = require('redis');
const client = redis.createClient();
await client.connect();
// SET/GET
await client.set('key', 'value');
const value = await client.get('key');
// SET with expiration
await client.setEx('key', 3600, 'value');
// DELETE
await client.del('key');
// EXISTS

```

```
const exists = await client.exists('key');
```

Caching Middleware

```
const cache = (duration) => async (req, res, next) => {
  const key = `cache:${req.originalUrl}`;
  const cached = await redis.get(key);
  if (cached) return res.json(JSON.parse(cached));
  res.originalJson = res.json;
  res.json = (data) => {
    redis.setEx(key, duration, JSON.stringify(data));
    res.originalJson(data);
  };
  next();
};
app.get('/api/users', cache(3600), async (req, res) => {
  const users = await User.findAll();
  res.json(users);
});
```

Redis Data Structures

```
// Lists
await client.lPush('queue', 'item1');
const item = await client.rPop('queue');
// Sets
await client.sAdd('tags', 'tag1', 'tag2');
const members = await client.sMembers('tags');
// Hashes
await client.hSet('user:1', 'name', 'John');
const name = await client.hGet('user:1', 'name');
// Sorted Sets
await client.zAdd('leaderboard', { score: 100, value: 'player1' });
const top = await client.zRange('leaderboard', 0, 9, { REV: true });
```

6.9 WebSockets

Socket.IO Server

```
const io = require('socket.io')(server);
io.on('connection', (socket) => {
  console.log('User connected');
  socket.on('message', (data) => {
    io.emit('message', data); // Broadcast to all
  });
});
```

```
socket.on('disconnect', () => {  
  console.log('User disconnected');  
});  
});
```

Socket.IO Client

```
const socket = io('http://localhost:3000');  
socket.on('connect', () => console.log('Connected'));  
socket.emit('message', 'Hello');  
socket.on('message', (data) => console.log(data));
```

Rooms

```
// Join room  
socket.join('room1');  
// Emit to room  
io.to('room1').emit('message', 'Hello room');  
// Leave room  
socket.leave('room1');
```

6.10 Interview Questions (50+)

Q1: What is Node.js? **A:** JavaScript runtime built on Chrome's V8 engine, enables server-side JS.

Q2: Event loop phases? **A:** Timers → Pending → Idle/Prepare → Poll → Check → Close.

Q3: process.nextTick() vs setImmediate()? **A:** nextTick: executes before event loop continues. setImmediate: executes in check phase.

Q4: What are streams? **A:** Efficient way to handle reading/writing files, network communication, or any end-to-end information exchange.

Q5: Types of streams? **A:** Readable, Writable, Duplex, Transform.

Q6: What is Buffer? **A:** Temporary storage for binary data.

Q7: Middleware in Express? **A:** Functions that execute during request-response cycle, have access to req, res, next.

Q8: Error handling in Express? **A:** Use error-handling middleware with (err, req, res, next) signature.

Q9: JWT vs Session? **A:** JWT: stateless, scalable. Session: stateful, server stores data.

Q10: How to secure API? **A:** JWT, rate limiting, CORS, helmet, input validation, HTTPS.

- Q11:** REST principles? **A:** Stateless, client-server, cacheable, uniform interface, layered system.
- Q12:** HTTP status codes? **A:** 2xx success, 3xx redirect, 4xx client error, 5xx server error.
- Q13:** PUT vs PATCH? **A:** PUT: replaces entire resource. PATCH: partial update.
- Q14:** SQL vs NoSQL? **A:** SQL: structured, ACID, relations. NoSQL: flexible, scalable, document/key-value.
- Q15:** ACID properties? **A:** Atomicity, Consistency, Isolation, Durability.
- Q16:** Database normalization? **A:** Organizing data to reduce redundancy.
- Q17:** Normal forms? **A:** 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency).
- Q18:** Index in database? **A:** Data structure that improves query speed.
- Q19:** Clustered vs Non-clustered index? **A:** Clustered: physically orders data. Non-clustered: separate structure with pointers.
- Q20:** Transaction isolation levels? **A:** Read Uncommitted, Read Committed, Repeatable Read, Serializable.
- Q21:** What is ORM? **A:** Object-Relational Mapping, converts between DB and objects.
- Q22:** N+1 query problem? **A:** Multiple queries when one would suffice. Solution: eager loading.
- Q23:** Connection pooling? **A:** Maintains multiple DB connections for reuse.
- Q24:** Database replication? **A:** Copying data across multiple servers for availability.
- Q25:** Master-slave replication? **A:** Master handles writes, slaves handle reads.
- Q26:** Sharding? **A:** Horizontal partitioning of data across multiple databases.
- Q27:** CAP theorem? **A:** Consistency, Availability, Partition tolerance. Can only have 2 of 3.
- Q28:** What is caching? **A:** Storing frequently accessed data in fast storage.
- Q29:** Cache invalidation strategies? **A:** Time-based (TTL), Write-through, Write-behind, Cache-aside.
- Q30:** Redis data types? **A:** String, List, Set, Sorted Set, Hash, Bitmap, HyperLogLog.
- Q31:** Redis persistence? **A:** RDB (snapshots), AOF (append-only file).
- Q32:** WebSocket vs HTTP? **A:** WebSocket: bidirectional, persistent. HTTP: request-response, stateless.
- Q33:** Long polling vs WebSocket? **A:** Long polling: HTTP-based, less efficient. WebSocket: persistent connection, real-time.
- Q34:** Server-Sent Events (SSE)? **A:** Server pushes updates to client over HTTP, unidirectional.

- Q35:** Rate limiting algorithms? **A:** Token bucket, Leaky bucket, Fixed window, Sliding window.
- Q36:** Load balancing? **A:** Distributing traffic across multiple servers.
- Q37:** Load balancing algorithms? **A:** Round robin, Least connections, IP hash, Weighted round robin.
- Q38:** Horizontal vs Vertical scaling? **A:** Horizontal: add more servers. Vertical: upgrade existing server.
- Q39:** Microservices vs Monolith? **A:** Microservices: independent services. Monolith: single codebase.
- Q40:** API Gateway? **A:** Single entry point for microservices, handles routing, authentication, rate limiting.
- Q41:** Message queue benefits? **A:** Asynchronous processing, decoupling, load leveling, reliability.
- Q42:** RabbitMQ vs Kafka? **A:** RabbitMQ: traditional message broker. Kafka: distributed streaming platform, high throughput.
- Q43:** What is Docker? **A:** Containerization platform for packaging applications.
- Q44:** Container vs VM? **A:** Container: shares OS kernel, lightweight. VM: full OS, heavier.
- Q45:** CI/CD? **A:** Continuous Integration/Continuous Deployment, automates build, test, deploy.
- Q46:** Blue-green deployment? **A:** Two identical environments, switch traffic after testing.
- Q47:** Canary deployment? **A:** Gradual rollout to subset of users.
- Q48:** Health checks? **A:** Endpoints to verify service availability.
- Q49:** Logging best practices? **A:** Structured logging, log levels, centralized logging, avoid PII.
- Q50:** Monitoring metrics? **A:** Response time, error rate, throughput, CPU/memory usage, database queries.

Chapter 7.5: React Native

7.5.1 React Native Core Concepts

Platform-Specific Code

```
import { Platform, StyleSheet } from 'react-native';
// Method 1: Platform.OS
const styles = StyleSheet.create({
  container: {
    marginTop: Platform.OS === 'ios' ? 20 : 0,
    ...Platform.select({
      ios: { shadowColor: '#000', shadowOffset: { width: 0, height: 2 } },
      android: { elevation: 4 }
    })
  }
});
```

```

    })
  }
});
// Method 2: Platform-specific files
// Button.ios.js and Button.android.js
import Button from './Button'; // Auto-selects correct file

```

Dimensions & Responsive Design

```

import { Dimensions, useWindowDimensions } from 'react-native';
const { width, height } = Dimensions.get('window');
const scale = width / 375; // iPhone 11 base
function normalize(size) { return Math.round(scale * size); }
// Hook-based (updates on rotation)
function MyComponent() {
  const { width, height } = useWindowDimensions();
  return <View style={{ width: width * 0.9 }} />;
}

```

Flexbox Layout

```

const styles = StyleSheet.create({
  container: { flex: 1, flexDirection: 'column', justifyContent: 'center', alignItems: 'center' },
  row: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center' },
  item: { flex: 1, margin: 10 }
});

```

SafeAreaView

```

import { SafeAreaView } from 'react-native-safe-area-context';
function App() {
  return (
    <SafeAreaView style={{ flex: 1 }}>
      <Text>Content safe from notch</Text>
    </SafeAreaView>
  );
}

```

StatusBar

```

import { StatusBar } from 'react-native';
<StatusBar barStyle="dark-content" backgroundColor="#fff" />

```

KeyboardAvoidingView

```
import { KeyboardAvoidingView, Platform } from 'react-native';
<KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : 'height'} style={{ flex: 1 }}>
  <TextInput placeholder="Email" />
</KeyboardAvoidingView>
```

ScrollView vs FlatList

```
// ScrollView: Small lists, renders all at once
<ScrollView>
  {items.map(item => <Item key={item.id} {...item} />)}
</ScrollView>
// FlatList: Large lists, virtual scrolling
<FlatList
  data={items}
  keyExtractor={item => item.id}
  renderItem={({ item }) => <Item {...item} />}
  onEndReached={loadMore}
  onEndReachedThreshold={0.5}
  ListEmptyComponent={<Text>No items</Text>}
  ListHeaderComponent={<Header />}
  ListFooterComponent={<Footer />}
  refreshing={refreshing}
  onRefresh={onRefresh}
/>
```

SectionList

```
import { SectionList } from 'react-native';
const DATA = [
  { title: 'A', data: ['Apple', 'Avocado'] },
  { title: 'B', data: ['Banana', 'Blueberry'] }
];
<SectionList
  sections={DATA}
  keyExtractor={({item, index}) => item + index}
  renderItem={({ item }) => <Text>{item}</Text>}
  renderSectionHeader={({ section: { title } }) => <Text style={{ fontWeight: 'bold' }}>{title}</Text>}
/>
```

Image

```
import { Image } from 'react-native';
// Local
<Image source={require('./logo.png')} style={{ width: 100, height: 100 }} />
```

```
// Remote
<Image source={{ uri: 'https://example.com/image.jpg' }} style={{ width: 100, height: 100 }} />
// With loading
const [loading, setLoading] = useState(true);
<Image
  source={{ uri: url }}
  onLoadStart={() => setLoading(true)}
  onLoadEnd={() => setLoading(false)}
/>
```

ImageBackground

```
import { ImageBackground } from 'react-native';
<ImageBackground source={require('./bg.jpg')} style={{ flex: 1 }}>
  <Text>Content over image</Text>
</ImageBackground>
```

TouchableOpacity vs Pressable

```
// TouchableOpacity (legacy)
<TouchableOpacity onPress={handlePress} activeOpacity={0.7}>
  <Text>Press me</Text>
</TouchableOpacity>
// Pressable (modern)
<Pressable
  onPress={handlePress}
  onLongPress={handleLongPress}
  style={({ pressed }) => [{ backgroundColor: pressed ? '#ddd' : '#fff' }]}>
  {{{ pressed }} => <Text>{pressed ? 'Pressed' : 'Press me'}</Text>
</Pressable>
```

Modal

```
import { Modal } from 'react-native';
const [visible, setVisible] = useState(false);
<Modal
  visible={visible}
  animationType="slide"
  transparent={true}
  onRequestClose={() => setVisible(false)}>
  <View style={{ flex: 1, justifyContent: 'center', backgroundColor: 'rgba(0,0,0,0.5)' }}>
    <View style={{ backgroundColor: 'white', margin: 20, padding: 20 }}>
      <Text>Modal Content</Text>
      <Button title="Close" onPress={() => setVisible(false)} />
    </View>
  </View>
```

```
</Modal>
```

ActivityIndicator

```
import { ActivityIndicator } from 'react-native';
<ActivityIndicator size="large" color="#0000ff" />
```

RefreshControl

```
import { ScrollView, RefreshControl } from 'react-native';
const [refreshing, setRefreshing] = useState(false);
const onRefresh = () => {
  setRefreshing(true);
  fetchData().finally(() => setRefreshing(false));
};
<ScrollView refreshControl={<RefreshControl refreshing={refreshing} onRefresh={onRefresh} />}>
  {/* Content */}
</ScrollView>
```

7.5.2 React Navigation

Installation & Setup

```
npm install @react-navigation/native
npm install react-native-screens react-native-safe-area-context
npm install @react-navigation/native-stack @react-navigation/bottom-tabs @react-navigation/drawer
```

Stack Navigator

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
const Stack = createNativeStackNavigator();
function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          initialRouteName="Home"
          screenOptions={{
            headerStyle: { backgroundColor: '#f4511e' },
            headerTintColor: '#fff',
            headerTitleStyle: { fontWeight: 'bold' }
          }} />
        <Stack.Screen name="Home" component={HomeScreen} options={{ title: 'My Home' }} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

```

    </Stack.Navigator>
  </NavigationContainer>
);
}

```

Navigation Props

```

function HomeScreen({ navigation, route }) {
  return (
    <View>
      <Button title="Go to Details" onPress={() => navigation.navigate('Details', { itemId: 42 })} />
      <Button title="Go Back" onPress={() => navigation.goBack()} />
      <Button title="Push Details" onPress={() => navigation.push('Details')} />
      <Button title="Pop to Top" onPress={() => navigation.popToTop()} />
    </View>
  );
}

function DetailsScreen({ route }) {
  const { itemId } = route.params;
  return <Text>Item ID: {itemId}</Text>;
}

```

Tab Navigator

```

import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';
const Tab = createBottomTabNavigator();

function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator
        screenOptions={({ route }) => ({
          tabBarIcon: ({ focused, color, size }) => {
            let iconName;
            if (route.name === 'Home') iconName = focused ? 'home' : 'home-outline';
            else if (route.name === 'Settings') iconName = focused ? 'settings' : 'settings-outline';
            return <Ionicons name={iconName} size={size} color={color} />;
          },
          tabBarActiveTintColor: 'tomato',
          tabBarInactiveTintColor: 'gray'
        })>
        <Tab.Screen name="Home" component={HomeScreen} />
        <Tab.Screen name="Settings" component={SettingsScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}

```

```
}
```

Drawer Navigator

```
import { createDrawerNavigator } from '@react-navigation/drawer';
const Drawer = createDrawerNavigator();
function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator>
        <Drawer.Screen name="Home" component={HomeScreen} />
        <Drawer.Screen name="Profile" component={ProfileScreen} />
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

Nested Navigation

```
function HomeTabs() {
  return (
    <Tab.Navigator>
      <Tab.Screen name="Feed" component={FeedScreen} />
      <Tab.Screen name="Messages" component={MessagesScreen} />
    </Tab.Navigator>
  );
}
function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeTabs} options={{ headerShown: false }} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

useNavigation Hook

```
import { useNavigation } from '@react-navigation/native';
function MyButton() {
  const navigation = useNavigation();
  return <Button title="Go" onPress={() => navigation.navigate('Details')} />;
}
```


useFocusEffect

```
import { useFocusEffect } from '@react-navigation/native';
function ProfileScreen() {
  useFocusEffect(
    React.useCallback(() => {
      console.log('Screen focused');
      fetchData();
      return () => console.log('Screen unfocused'); // Cleanup
    }, [])
  );
}
```

7.5.3 Zustand State Management

Installation

```
npm install zustand
```

Basic Store

```
import create from 'zustand';
const useStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
  decrement: () => set((state) => ({ count: state.count - 1 })),
  reset: () => set({ count: 0 })
}));
// Usage
function Counter() {
  const { count, increment, decrement } = useStore();
  return (
    <View>
      <Text>{count}</Text>
      <Button title="+" onPress={increment} />
      <Button title="-" onPress={decrement} />
    </View>
  );
}
```

Async Actions

```
const useStore = create((set) => ({
  users: [],
  loading: false,
```

```

error: null,
fetchUsers: async () => {
  set({ loading: true, error: null });
  try {
    const response = await fetch('https://api.example.com/users');
    const users = await response.json();
    set({ users, loading: false });
  } catch (error) {
    set({ error: error.message, loading: false });
  }
}
});

```

Selectors

```

// Only re-renders when count changes
const count = useStore((state) => state.count);
// Shallow comparison for objects
const user = useStore((state) => state.user, shallow);

```

Persist Middleware

```

import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';
const useStore = create(
  persist(
    (set) => ({
      token: null,
      setToken: (token) => set({ token })
    }),
    {
      name: 'auth-storage',
      storage: createJSONStorage(() => AsyncStorage)
    }
  )
);

```

Immer Middleware

```

import { immer } from 'zustand/middleware/immer';
const useStore = create(
  immer((set) => ({
    todos: [],
    addTodo: (text) => set((state) => {
      state.todos.push({ id: Date.now(), text, completed: false });
    })
  })
);

```

```

    }),
    toggleTodo: (id) => set((state) => {
      const todo = state.todos.find(t => t.id === id);
      if (todo) todo.completed = !todo.completed;
    })
  })
});

```

Multiple Stores

```

const useAuthStore = create((set) => ({
  user: null,
  login: (user) => set({ user }),
  logout: () => set({ user: null })
}));

const useCartStore = create((set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] }))
}));

```

7.5.4 Expo & EAS

Expo CLI Setup

```

npm install -g expo-cli
expo init MyApp
cd MyApp
expo start

```

Expo Config (app.json)

```

{
  "expo": {
    "name": "MyApp",
    "slug": "my-app",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "splash": { "image": "./assets/splash.png", "resizeMode": "contain", "backgroundColor": "#ffffff" },
    "assetBundlePatterns": ["**/*"],
    "ios": { "supportsTablet": true, "bundleIdentifier": "com.company.myapp" },
    "android": { "adaptiveIcon": { "foregroundImage": "./assets/adaptive-icon.png", "backgroundColor": "#ffffff" },
    "package": "com.company.myapp",
    "web": { "favicon": "./assets/favicon.png" }
  }
}

```

```
}
```

Expo Location

```
import * as Location from 'expo-location';
async function getLocation() {
  const { status } = await Location.requestForegroundPermissionsAsync();
  if (status !== 'granted') return;
  const location = await Location.getCurrentPositionAsync({});
  console.log(location.coords.latitude, location.coords.longitude);
}
```

Expo Camera

```
import { Camera } from 'expo-camera';
function CameraScreen() {
  const [hasPermission, setHasPermission] = useState(null);
  const [type, setType] = useState(Camera.Constants.Type.back);
  useEffect(() => {
    (async () => {
      const { status } = await Camera.requestCameraPermissionsAsync();
      setHasPermission(status === 'granted');
    })();
  }, []);
  if (hasPermission === null) return <View />;
  if (hasPermission === false) return <Text>No camera access</Text>;
  return (
    <Camera style={{ flex: 1 }} type={type}>
      <Button title="Flip" onPress={() => setType(type === Camera.Constants.Type.back ?
Camera.Constants.Type.front : Camera.Constants.Type.back)} />
    </Camera>
  );
}
```

Expo ImagePicker

```
import * as ImagePicker from 'expo-image-picker';
async function pickImage() {
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsEditing: true,
    aspect: [4, 3],
    quality: 1
  });
  if (!result.canceled) {
    console.log(result.assets[0].uri);
  }
}
```

```

    }
  }
}

```

Expo Notifications

```

import * as Notifications from 'expo-notifications';
Notifications.setNotificationHandler({
  handleNotification: async () => ({ shouldShowAlert: true, shouldPlaySound: true, shouldSetBadge: false })
});
async function registerForPushNotifications() {
  const { status } = await Notifications.requestPermissionsAsync();
  if (status !== 'granted') return;
  const token = (await Notifications.getExpoPushTokenAsync()).data;
  console.log(token);
}
async function scheduleNotification() {
  await Notifications.scheduleNotificationAsync({
    content: { title: "You've got mail!", body: 'Here is the notification body' },
    trigger: { seconds: 2 }
  });
}

```

Expo SecureStore

```

import * as SecureStore from 'expo-secure-store';
async function save(key, value) {
  await SecureStore.setItemAsync(key, value);
}
async function get(key) {
  return await SecureStore.getItemAsync(key);
}

```

Expo FileSystem

```

import * as FileSystem from 'expo-file-system';
async function downloadFile() {
  const fileUri = FileSystem.documentDirectory + 'file.jpg';
  const { uri } = await FileSystem.downloadAsync('https://example.com/file.jpg', fileUri);
  console.log('Downloaded to:', uri);
}

```

EAS Build

```
npm install -g eas-cli
eas login
eas build:configure
# Build for Android
eas build --platform android
# Build for iOS
eas build --platform ios
# Submit to stores
eas submit --platform android
eas submit --platform ios
```

EAS Update

```
eas update:configure
eas update --branch production --message "Fix bug"
```

eas.json

```
{
  "build": {
    "development": { "developmentClient": true, "distribution": "internal" },
    "preview": { "distribution": "internal" },
    "production": {}
  },
  "submit": {
    "production": {}
  }
}
```

7.5.5 AsyncStorage

Installation

```
npm install @react-native-async-storage/async-storage
```

Basic Operations

```
import AsyncStorage from '@react-native-async-storage/async-storage';
// Save
await AsyncStorage.setItem('token', 'abc123');
// Get
const token = await AsyncStorage.getItem('token');
```

```

// Save object
await AsyncStorage.setItem('user', JSON.stringify({ name: 'John', age: 30 }));
// Get object
const user = JSON.parse(await AsyncStorage.getItem('user'));
// Remove
await AsyncStorage.removeItem('token');
// Clear all
await AsyncStorage.clear();
// Get all keys
const keys = await AsyncStorage.getAllKeys();

```

Custom Hook

```

import { useState, useEffect } from 'react';
import AsyncStorage from '@react-native-async-storage/async-storage';
function useAsyncStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(initialValue);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    (async () => {
      try {
        const item = await AsyncStorage.getItem(key);
        setStoredValue(item ? JSON.parse(item) : initialValue);
      } catch (error) {
        console.error(error);
      } finally {
        setLoading(false);
      }
    })();
  }, [key]);
  const setValue = async (value) => {
    try {
      setStoredValue(value);
      await AsyncStorage.setItem(key, JSON.stringify(value));
    } catch (error) {
      console.error(error);
    }
  };
  return [storedValue, setValue, loading];
}
// Usage
function App() {
  const [user, setUser, loading] = useAsyncStorage('user', null);
  if (loading) return <ActivityIndicator />;
  return <Text>{user?.name}</Text>;
}

```

7.5.6 Animations

Animated API

```
import { Animated } from 'react-native';
function FadeInView({ children }) {
  const fadeAnim = useRef(new Animated.Value(0)).current;
  useEffect(() => {
    Animated.timing(fadeAnim, { toValue: 1, duration: 1000, useNativeDriver: true }).start();
  }, [fadeAnim]);
  return <Animated.View style={{ opacity: fadeAnim }}>{children}</Animated.View>;
}
```

Spring Animation

```
const scale = useRef(new Animated.Value(1)).current;
const scaleUp = () => {
  Animated.spring(scale, { toValue: 1.5, friction: 3, useNativeDriver: true }).start();
};
<Animated.View style={{ transform: [{ scale }] }}>
  <Pressable onPress={scaleUp}><Text>Press me</Text></Pressable>
</Animated.View>
```

Parallel Animations

```
const fadeAnim = useRef(new Animated.Value(0)).current;
const slideAnim = useRef(new Animated.Value(-100)).current;
Animated.parallel([
  Animated.timing(fadeAnim, { toValue: 1, duration: 500, useNativeDriver: true }),
  Animated.timing(slideAnim, { toValue: 0, duration: 500, useNativeDriver: true })
]).start();
```

Sequence Animations

```
Animated.sequence([
  Animated.timing(fadeAnim, { toValue: 1, duration: 500, useNativeDriver: true }),
  Animated.timing(fadeAnim, { toValue: 0, duration: 500, useNativeDriver: true })
]).start();
```

Loop Animation

```
const rotation = useRef(new Animated.Value(0)).current;
useEffect(() => {
  Animated.loop(
```



```

    Animated.timing(rotation, { toValue: 1, duration: 2000, useNativeDriver: true })
  ).start();
}, []);
const spin = rotation.interpolate({ inputRange: [0, 1], outputRange: ['0deg', '360deg'] });
<Animated.View style={{ transform: [{ rotate: spin }] }} />

```

LayoutAnimation

```

import { LayoutAnimation, UIManager, Platform } from 'react-native';
if (Platform.OS === 'android' && UIManager.setLayoutAnimationEnabledExperimental) {
  UIManager.setLayoutAnimationEnabledExperimental(true);
}
function App() {
  const [expanded, setExpanded] = useState(false);
  const toggle = () => {
    LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
    setExpanded(!expanded);
  };
  return (
    <View>
      <Pressable onPress={toggle}><Text>Toggle</Text></Pressable>
      {expanded && <View style={{ height: 100, backgroundColor: 'blue' }} />}
    </View>
  );
}

```

7.5.7 Gestures (React Native Gesture Handler)

Installation

npm install react-native-gesture-handler

TapGestureHandler

```

import { GestureHandlerRootView, TapGestureHandler } from 'react-native-gesture-handler';
function App() {
  const onSingleTap = () => console.log('Single tap');
  const onDoubleTap = () => console.log('Double tap');
  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <TapGestureHandler onActivated={onSingleTap} numberOfTaps={1}>
        <TapGestureHandler onActivated={onDoubleTap} numberOfTaps={2}>
          <View style={{ width: 100, height: 100, backgroundColor: 'blue' }} />
        </TapGestureHandler>
      </TapGestureHandler>
    </GestureHandlerRootView>
  );
}

```

```

    </GestureHandlerRootView>
  );
}

```

PanGestureHandler

```

import { PanGestureHandler } from 'react-native-gesture-handler';
import Animated, { useAnimatedGestureHandler, useAnimatedStyle, useSharedValue, withSpring } from 'react-native-reanimated';
function Draggable() {
  const translateX = useSharedValue(0);
  const translateY = useSharedValue(0);
  const gestureHandler = useAnimatedGestureHandler({
    onStart: (_, ctx) => {
      ctx.startX = translateX.value;
      ctx.startY = translateY.value;
    },
    onActive: (event, ctx) => {
      translateX.value = ctx.startX + event.translationX;
      translateY.value = ctx.startY + event.translationY;
    },
    onEnd: () => {
      translateX.value = withSpring(0);
      translateY.value = withSpring(0);
    }
  });
  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ translateX: translateX.value }, { translateY: translateY.value }]
  }));
  return (
    <PanGestureHandler onGestureEvent={gestureHandler}>
      <Animated.View style={[{ width: 100, height: 100, backgroundColor: 'blue' }, animatedStyle]} />
    </PanGestureHandler>
  );
}

```

Swipeable

```

import Swipeable from 'react-native-gesture-handler/Swipeable';
function SwipeableItem() {
  const renderRightActions = () => (
    <View style={{ backgroundColor: 'red', justifyContent: 'center', padding: 20 }}>
      <Text style={{ color: 'white' }}>Delete</Text>
    </View>
  );
  return (
    <Swipeable renderRightActions={renderRightActions}>

```

```

    <View style={{ padding: 20, backgroundColor: 'white' }}>
      <Text>Swipe left</Text>
    </View>
  </Swipeable>
);
}

```

7.5.8 API Integration

Fetch API

```

async function fetchUsers() {
  try {
    const response = await fetch('https://api.example.com/users');
    if (!response.ok) throw new Error('Network error');
    const data = await response.json();
    return data;
  } catch (error) {
    console.error(error);
  }
}

// POST request
async function createUser(user) {
  const response = await fetch('https://api.example.com/users', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(user)
  });
  return response.json();
}

```

Axios

```

npm install axios
import axios from 'axios';
const api = axios.create({
  baseURL: 'https://api.example.com',
  timeout: 10000,
  headers: { 'Content-Type': 'application/json' }
});

// Interceptors
api.interceptors.request.use((config) => {
  const token = await AsyncStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

```

```

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      // Logout
    }
    return Promise.reject(error);
  }
);
// Usage
const { data } = await api.get('/users');
await api.post('/users', { name: 'John' });

```

React Query with RN

npm install @tanstack/react-query

```

import { QueryClient, QueryClientProvider, useQuery, useMutation } from '@tanstack/react-query';
const queryClient = new QueryClient();
function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <UserList />
    </QueryClientProvider>
  );
}
function UserList() {
  const { data, isLoading, error, refetch } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('https://api.example.com/users').then(r => r.json())
  });
  if (isLoading) return <ActivityIndicator />;
  if (error) return <Text>Error: {error.message}</Text>;
  return (
    <FlatList
      data={data}
      keyExtractor={item => item.id}
      renderItem={({ item }) => <Text>{item.name}</Text>}
      refreshing={isLoading}
      onRefresh={refetch}
    />
  );
}

```

7.5.9 Performance Optimization

FlatList Optimization

```
<FlatList
  data={items}
  keyExtractor={item => item.id}
  renderItem={({ item }) => <Item {...item} />}
  getItemLayout={(data, index) => ({ length: 80, offset: 80 * index, index })}
  initialNumToRender={10}
  maxToRenderPerBatch={10}
  updateCellsBatchingPeriod={50}
  windowSize={21}
  removeClippedSubviews={true}
/>
```

React.memo for List Items

```
const Item = React.memo(({ id, title }) => {
  console.log('Render item', id);
  return <Text>{title}</Text>;
}, (prevProps, nextProps) => prevProps.id === nextProps.id);
```

useCallback for Handlers

```
const handlePress = useCallback((id) => {
  console.log('Pressed', id);
}, []);
```

Image Optimization

```
import FastImage from 'react-native-fast-image';
<FastImage
  source={{ uri: imageUrl, priority: FastImage.priority.normal }}
  style={{ width: 100, height: 100 }}
  resizeMode={FastImage.resizeMode.cover}
/>
```

Hermes Engine

```
// android/app/build.gradle
project.ext.react = [
  enableHermes: true
```

```
]
```

Native Driver

```
// Always use useNativeDriver: true for animations
Animated.timing(fadeAnim, {
  toValue: 1,
  duration: 1000,
  useNativeDriver: true // IMPORTANT
}).start();
```

7.5.10 Debugging & Development Tools

React Native Debugger

```
# Install
brew install --cask react-native-debugger
# Enable
# Shake device → "Debug" → Opens debugger
```

Flipper

```
# Comes with React Native 0.62+
# iOS: Pods installed automatically
# Android: Gradle configuration automatic
```

Console Logs

```
console.log('Info');
console.warn('Warning'); // Shows yellow box
console.error('Error'); // Shows red screen
console.tron.log('Reactotron'); // If using Reactotron
```

Remote Debugging

```
// Enable in app
import { NativeModules } from 'react-native';
NativeModules.DevSettings.setIsDebuggingRemotely(true);
```

Performance Monitor

```
// Shake device → "Show Perf Monitor"
// Or programmatically
import { NativeModules } from 'react-native';
NativeModules.DevSettings.setIsPerfMonitorShown(true);
```

React DevTools

```
npm install -g react-devtools
react-devtools
```

7.5.11 Testing React Native

Jest Unit Tests

```
// sum.test.js
import { sum } from './sum';
test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3);
});
```

React Native Testing Library

```
npm install --save-dev @testing-library/react-native
import { render, fireEvent } from '@testing-library/react-native';
import Counter from './Counter';
test('increments counter', () => {
  const { getByText } = render(<Counter />);
  const button = getByText('+');
  fireEvent.press(button);
  expect(getByText('1')).toBeTruthy();
});
```

Mocking AsyncStorage

```
import AsyncStorage from '@react-native-async-storage/async-storage/jest/async-storage-mock';
jest.mock('@react-native-async-storage/async-storage', () => AsyncStorage);
```

Snapshot Testing

```
import renderer from 'react-test-renderer';
test('renders correctly', () => {
```

```
const tree = renderer.create(<MyComponent />).toJSON();
expect(tree).toMatchSnapshot();
});
```

E2E Testing (Detox)

```
npm install --save-dev detox
detox init
// e2e/firstTest.e2e.js
describe('Example', () => {
  beforeAll(async () => {
    await device.launchApp();
  });
  it('should show hello screen', async () => {
    await expect(element(by.text('Hello'))).toBeVisible();
  });
  it('should tap button', async () => {
    await element(by.id('button')).tap();
    await expect(element(by.text('Tapped'))).toBeVisible();
  });
});
```

7.5.12 Complete App Example: Todo App

App Structure

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { create } from 'zustand';
import { persist, createJSONStorage } from 'zustand/middleware';
import AsyncStorage from '@react-native-async-storage/async-storage';
// Zustand Store
const useTodoStore = create(
  persist(
    (set) => ({
      todos: [],
      addTodo: (text) => set((state) => ({
        todos: [...state.todos, { id: Date.now(), text, completed: false }]
      })),
      toggleTodo: (id) => set((state) => ({
        todos: state.todos.map(t => t.id === id ? { ...t, completed: !t.completed } : t)
      })),
      deleteTodo: (id) => set((state) => ({
        todos: state.todos.filter(t => t.id !== id)
      }))
    })
  )
);
```



```

    { name: 'todo-storage', storage: createJSONStorage(() => AsyncStorage) }
  )
};
// Home Screen
function HomeScreen({ navigation }) {
  const { todos, toggleTodo, deleteTodo } = useTodoStore();
  const [showCompleted, setShowCompleted] = useState(true);
  const filteredTodos = showCompleted ? todos : todos.filter(t => !t.completed);
  return (
    <View style={{ flex: 1 }}>
      <View style={{ flexDirection: 'row', padding: 10 }}>
        <Button title="Add Todo" onPress={() => navigation.navigate('Add')} />
        <Button title={showCompleted ? 'Hide Completed' : 'Show All'} onPress={() =>
setShowCompleted(!showCompleted)} />
      </View>
      <FlatList
        data={filteredTodos}
        keyExtractor={item => item.id.toString()}
        renderItem={({ item }) => (
          <Swipeable renderRightActions={() => (
            <Pressable onPress={() => deleteTodo(item.id)} style={{ backgroundColor: 'red', justifyContent:
'center', padding: 20 }}>
              <Text style={{ color: 'white' }}>Delete</Text>
            </Pressable>
          )}>
            <Pressable onPress={() => toggleTodo(item.id)} style={{ padding: 15, backgroundColor: 'white',
borderBottomWidth: 1 }}>
              <Text style={{ textDecorationLine: item.completed ? 'line-through' : 'none' }}>{item.text}</Text>
            </Pressable>
          </Swipeable>
        )}
      />
    </View>
  );
}
// Add Screen
function AddScreen({ navigation }) {
  const [text, setText] = useState("");
  const addTodo = useTodoStore(state => state.addTodo);
  const handleAdd = () => {
    if (text.trim()) {
      addTodo(text);
      navigation.goBack();
    }
  };
  return (
    <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : 'height'} style={{ flex: 1, padding: 20 }}>
      <TextInput
        value={text}

```

```

        onChangeText={setText}
        placeholder="Enter todo"
        style={{ borderWidth: 1, padding: 10, marginBottom: 10 }}
      />
      <Button title="Add" onPress={handleAdd} />
    </KeyboardAvoidingView>
  );
}
// Main App
const Stack = createNativeStackNavigator();
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} options={{ title: 'My Todos' }} />
        <Stack.Screen name="Add" component={AddScreen} options={{ title: 'Add Todo' }} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

```

7.5.13 Interview Questions (60+)

- Q1:** React Native vs Flutter? **A:** RN: JavaScript, native components, larger community. Flutter: Dart, custom rendering, better performance.
- Q2:** React Native architecture? **A:** JavaScript thread, Native thread, Bridge (JSI in new architecture).
- Q3:** What is the Bridge? **A:** Communication layer between JS and Native code (being replaced by JSI).
- Q4:** New Architecture (Fabric)? **A:** Direct JS-Native communication via JSI, no bridge, synchronous rendering.
- Q5:** Turbo Modules? **A:** Lazy-loaded native modules, part of new architecture.
- Q6:** FlatList vs ScrollView? **A:** FlatList: virtual scrolling, large lists. ScrollView: all rendered, small lists.
- Q7:** How to optimize FlatList? **A:** getItemLayout, initialNumToRender, windowSize, removeClippedSubviews, React.memo.
- Q8:** useNativeDriver purpose? **A:** Runs animations on native thread, 60fps, doesn't block JS thread.
- Q9:** When can't use useNativeDriver? **A:** Layout properties (width, height, margin, padding). Only transform and opacity work.
- Q10:** Platform-specific code methods? **A:** Platform.OS, Platform.select(), .ios.js/.android.js files.
- Q11:** SafeAreaView vs SafeAreaContext? **A:** SafeAreaView: built-in, iOS only. SafeAreaContext: library, both platforms, more flexible.

- Q12:** KeyboardAvoidingView behavior? **A:** 'padding' (iOS), 'height', 'position'. Adjusts view when keyboard appears.
- Q13:** Dimensions.get vs useWindowDimensions? **A:** Dimensions.get: static. useWindowDimensions: hook, updates on rotation.
- Q14:** Flexbox differences from web? **A:** flexDirection: 'column' default, alignContent not supported, no auto margins.
- Q15:** Image caching? **A:** Use react-native-fast-image, built-in caching, priority levels.
- Q16:** AsyncStorage limitations? **A:** 6MB limit, unencrypted, key-value only. Use for non-sensitive data.
- Q17:** Secure storage options? **A:** expo-secure-store, react-native-keychain (iOS Keychain, Android Keystore).
- Q18:** Navigation library comparison? **A:** React Navigation: JS-based, flexible. React Native Navigation: native, better performance.
- Q19:** Stack vs Tab vs Drawer? **A:** Stack: linear flow. Tab: parallel sections. Drawer: side menu.
- Q20:** Deep linking? **A:** Configure linking in NavigationContainer, handle URLs in app.
- Q21:** Push notifications setup? **A:** expo-notifications or react-native-push-notification, FCM (Android), APNs (iOS).
- Q22:** Notification permissions? **A:** Request at runtime, explain why needed, handle denial gracefully.
- Q23:** Local vs Remote notifications? **A:** Local: scheduled by app. Remote: sent from server via FCM/APNs.
- Q24:** Zustand vs Redux? **A:** Zustand: simpler, less boilerplate, hooks-based. Redux: established, more tooling.
- Q25:** Zustand vs Context? **A:** Zustand: no re-render issues, better performance, middleware support.
- Q26:** Persist state in Zustand? **A:** Use persist middleware with AsyncStorage.
- Q27:** Expo vs Bare React Native? **A:** Expo: managed, easier, limited native modules. Bare: full control, custom native code.
- Q28:** Expo Go limitations? **A:** Can't use custom native modules, limited to Expo SDK.
- Q29:** EAS Build vs Classic Build? **A:** EAS: cloud-based, faster, consistent. Classic: deprecated.
- Q30:** OTA updates? **A:** eas update, CodePush. Updates JS bundle without app store approval.
- Q31:** Expo SDK? **A:** Pre-built native modules (Camera, Location, etc.) with unified API.
- Q32:** expo-camera vs react-native-camera? **A:** expo-camera: Expo SDK, easier. RNC: more features, bare RN only.

- Q33:** How to add custom fonts? **A:** expo-font, useFonts hook, Font.loadAsync().
- Q34:** Splash screen? **A:** Configure in app.json, expo-splash-screen for programmatic control.
- Q35:** App icon & adaptive icon? **A:** icon.png (iOS), adaptive-icon (Android), configure in app.json.
- Q36:** Environment variables? **A:** expo-constants, .env files, Constants.expoConfig.extra.
- Q37:** Error boundaries in RN? **A:** Same as React, use componentDidCatch or react-error-boundary.
- Q38:** Crash reporting? **A:** Sentry, Bugsnag, Firebase Crashlytics.
- Q39:** Analytics? **A:** Firebase Analytics, Amplitude, Mixpanel.
- Q40:** How to debug? **A:** React Native Debugger, Flipper, console.log, React DevTools.
- Q41:** Network inspector? **A:** Flipper Network plugin, React Native Debugger network tab.
- Q42:** Handling app state (background/foreground)? **A:** AppState API, listen to 'change' event.
- Q43:** Biometric authentication? **A:** expo-local-authentication, react-native-biometrics.
- Q44:** File system access? **A:** expo-file-system, react-native-fs.
- Q45:** Share content? **A:** Share API (built-in), expo-sharing.
- Q46:** In-app purchases? **A:** expo-in-app-purchases, react-native-iap.
- Q47:** Maps integration? **A:** react-native-maps, Mapbox, Google Maps.
- Q48:** WebView? **A:** react-native-webview, postMessage for communication.
- Q49:** SVG support? **A:** react-native-svg, svgr for importing SVGs.
- Q50:** Lottie animations? **A:** lottie-react-native, play JSON animations.
- Q51:** Gesture handling? **A:** react-native-gesture-handler, better than PanResponder.
- Q52:** Reanimated vs Animated? **A:** Reanimated: runs on UI thread, better performance. Animated: legacy, JS thread.
- Q53:** Layout animations? **A:** LayoutAnimation API, configure before setState.
- Q54:** Skia in RN? **A:** @shopify/react-native-skia, 2D graphics, high performance.
- Q55:** Code push? **A:** Microsoft CodePush, OTA updates, deprecated (use EAS Update).
- Q56:** Fastlane? **A:** Automate builds, screenshots, beta deployment.
- Q57:** CI/CD for RN? **A:** GitHub Actions, Bitrise, EAS Build, Jenkins.

Q58: Bundle size optimization? **A:** Hermes, ProGuard (Android), strip debug symbols.

Q59: Memory leaks prevention? **A:** Cleanup useEffect, remove listeners, clear timers, unsubscribe.

Q60: Testing libraries? **A:** Jest, React Native Testing Library, Detox (E2E).

Chapter 7: Frontend Engineering

7.1 React Fundamentals

Components

```
// Functional Component
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}
// Arrow Function Component
const Welcome = ({ name }) => <h1>Hello, {name}</h1>;
// Component with JSX
function App() {
  return (
    <div className="app">
      <h1>My App</h1>
      <Welcome name="John" />
    </div>
  );
}
```

Props

```
function Card({ title, description, children }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <p>{description}</p>
      {children}
    </div>
  );
}
// Usage
<Card title="Title" description="Desc">
  <button>Click me</button>
</Card>
```

State (useState)

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(prev => prev - 1)}>Decrement</button>
    </div>
  );
}
```

useEffect

```
useEffect(() => {
  // Runs after every render
  console.log('Effect ran');
});
useEffect(() => {
  // Runs once on mount
  console.log('Mount');
}, []);
useEffect(() => {
  // Runs when count changes
  console.log('Count changed');
}, [count]);
useEffect(() => {
  // Cleanup function
  return () => console.log('Cleanup');
}, []);
```

useContext

```
const ThemeContext = React.createContext();
function App() {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Child />
    </ThemeContext.Provider>
  );
}
function Child() {
  const { theme, setTheme } = useContext(ThemeContext);
  return <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>{theme}</button>;
}
```

```
}
```

useReducer

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
};

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <p>{state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    </div>
  );
}
```

useRef

```
function TextInput() {
  const inputRef = useRef(null);
  const focusInput = () => inputRef.current.focus();
  return (
    <div>
      <input ref={inputRef} />
      <button onClick={focusInput}>Focus</button>
    </div>
  );
}

// Storing mutable value
function Timer() {
  const intervalRef = useRef();
  useEffect(() => {
    intervalRef.current = setInterval(() => console.log('Tick'), 1000);
    return () => clearInterval(intervalRef.current);
  }, []);
}
```

useMemo

```
function ExpensiveComponent({ items }) {
  const expensiveValue = useMemo(() => {
    return items.reduce((acc, item) => acc + item.value, 0);
  }, [items]);
  return <div>{expensiveValue}</div>;
}
```

useCallback

```
function Parent() {
  const [count, setCount] = useState(0);
  const increment = useCallback(() => setCount(c => c + 1), []);
  return <Child onIncrement={increment} />;
}
```

Custom Hooks

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    fetch(url)
      .then(res => res.json())
      .then(setData)
      .catch(setError)
      .finally(() => setLoading(false));
  }, [url]);
  return { data, loading, error };
}

// Usage
function App() {
  const { data, loading, error } = useFetch('/api/users');
  if (loading) return <div>Loading...</div>;
  if (error) return <div>Error: {error.message}</div>;
  return <div>{JSON.stringify(data)}</div>;
}
```


7.2 Lists & Forms

Lists & Keys

```
function TodoList({ todos }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

Controlled Forms

```
function LoginForm() {
  const [formData, setFormData] = useState({ email: "", password: "" });
  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };
  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(formData);
  };
  return (
    <form onSubmit={handleSubmit}>
      <input name="email" value={formData.email} onChange={handleChange} />
      <input name="password" type="password" value={formData.password} onChange={handleChange} />
      <button type="submit">Login</button>
    </form>
  );
}
```

Form Validation

```
function Form() {
  const [email, setEmail] = useState("");
  const [error, setError] = useState("");
  const validate = () => {
    if (!email.includes('@')) {
      setError('Invalid email');
      return false;
    }
    setError("");
    return true;
  };
}
```

```

};
const handleSubmit = (e) => {
  e.preventDefault();
  if (validate()) console.log('Submit', email);
};
return (
  <form onSubmit={handleSubmit}>
    <input value={email} onChange={(e) => setEmail(e.target.value)} />
    {error && <span>{error}</span>}
    <button type="submit">Submit</button>
  </form>
);
}

```

7.3 React Router

Basic Routing

```

import { BrowserRouter, Routes, Route, Link, useNavigate } from 'react-router-dom';
function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/users/123">User</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users/:id" element={<User />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

```

useParams & useNavigate

```

import { useParams, useNavigate } from 'react-router-dom';
function User() {
  const { id } = useParams();
  const navigate = useNavigate();
  return (
    <div>
      <h1>User {id}</h1>

```

```

    <button onClick={() => navigate('/')}>Go Home</button>
    <button onClick={() => navigate(-1)}>Go Back</button>
  </div>
);
}

```

Protected Routes

```

function ProtectedRoute({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" />;
}
// Usage
<Route path="/dashboard" element={<ProtectedRoute><Dashboard /></ProtectedRoute>} />

```

7.4 State Management - Redux

Setup

```

// store.js
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';
export const store = configureStore({
  reducer: { counter: counterReducer }
});

```

Slice

```

// counterSlice.js
import { createSlice } from '@reduxjs/toolkit';
const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: {
    increment: (state) => { state.value += 1; },
    decrement: (state) => { state.value -= 1; },
    incrementByAmount: (state, action) => { state.value += action.payload; }
  }
});
export const { increment, decrement, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;

```

Component

```
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, incrementByAmount } from './counterSlice';
function Counter() {
  const count = useSelector(state => state.counter.value);
  const dispatch = useDispatch();
  return (
    <div>
      <p>{count}</p>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(incrementByAmount(5))}>+5</button>
    </div>
  );
}
```

Async Actions

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';
export const fetchUsers = createAsyncThunk('users/fetch', async () => {
  const response = await fetch('/api/users');
  return response.json();
});
const usersSlice = createSlice({
  name: 'users',
  initialState: { data: [], loading: false, error: null },
  extraReducers: (builder) => {
    builder
      .addCase(fetchUsers.pending, (state) => { state.loading = true; })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.loading = false;
        state.data = action.payload;
      })
      .addCase(fetchUsers.rejected, (state, action) => {
        state.loading = false;
        state.error = action.error.message;
      });
  }
});
```

7.5 React Query

Setup

```
import { QueryClient, QueryClientProvider, useQuery, useMutation } from '@tanstack/react-query';
const queryClient = new QueryClient();
function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <Users />
    </QueryClientProvider>
  );
}
```

Query

```
function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['users'],
    queryFn: () => fetch('/api/users').then(res => res.json())
  });
  if (isLoading) return <div>Loading...</div>;
  if (error) return <div>Error: {error.message}</div>;
  return <ul>{data.map(user => <li key={user.id}>{user.name}</li>)}</ul>;
}
```

Mutation

```
function AddUser() {
  const queryClient = useQueryClient();
  const mutation = useMutation({
    mutationFn: (newUser) => fetch('/api/users', {
      method: 'POST',
      body: JSON.stringify(newUser),
      headers: { 'Content-Type': 'application/json' }
    }).then(res => res.json()),
    onSuccess: () => {
      queryClient.invalidateQueries(['users']);
    }
  });
  const handleSubmit = (e) => {
    e.preventDefault();
    mutation.mutate({ name: 'John' });
  };
  return <button onClick={handleSubmit}>Add User</button>;
}
```

```
}
```

7.6 Styling

CSS Modules

```
import styles from './Button.module.css';
function Button() {
  return <button className={styles.button}>Click</button>;
}
```

Styled Components

```
import styled from 'styled-components';
const Button = styled.button`
  background: blue;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  &:hover { background: darkblue; }
`;
const PrimaryButton = styled(Button)`
  background: green;
`;
```

Tailwind CSS

```
function Button() {
  return (
    <button className="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded">
      Click me
    </button>
  );
}
```

7.7 Performance Optimization

React.memo

```
const ExpensiveComponent = React.memo(({ data }) => {
  console.log('Render');
  return <div>{data}</div>;
});
```

```
});
```

Code Splitting

```
import { lazy, Suspense } from 'react';
const LazyComponent = lazy(() => import('./LazyComponent'));
function App() {
  return (
    <Suspense fallback=<div>Loading...</div>>
      <LazyComponent />
    </Suspense>
  );
}
```

Virtualization

```
import { FixedSizeList } from 'react-window';
function LargeList({ items }) {
  return (
    <FixedSizeList height={600} itemCount={items.length} itemSize={50} width="100%">
      {({ index, style }) => <div style={style}>{items[index]}</div>}
    </FixedSizeList>
  );
}
```

Debouncing & Throttling

```
import { useState, useEffect } from 'react';
function useDebounce(value, delay) {
  const [debouncedValue, setDebouncedValue] = useState(value);
  useEffect(() => {
    const handler = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(handler);
  }, [value, delay]);
  return debouncedValue;
}
// Usage
function SearchInput() {
  const [search, setSearch] = useState('');
  const debouncedSearch = useDebounce(search, 500);
  useEffect(() => {
    if (debouncedSearch) {
      // API call
      console.log('Search:', debouncedSearch);
    }
  }, [debouncedSearch]);
}
```

```

    return <input value={search} onChange={(e) => setSearch(e.target.value)} />;
  }

```

7.8 Testing

Jest Unit Test

```

// sum.js
export const sum = (a, b) => a + b;
// sum.test.js
import { sum } from './sum';
test('adds 1 + 2 to equal 3', () => {
  expect(sum(1, 2)).toBe(3);
});

```

React Testing Library

```

import { render, screen, fireEvent } from '@testing-library/react';
import Counter from './Counter';
test('increments counter', () => {
  render(<Counter />);
  const button = screen.getByText('+');
  fireEvent.click(button);
  expect(screen.getByText('1')).toBeInTheDocument();
});

```

Mock API

```

import { rest } from 'msw';
import { setupServer } from 'msw/node';
const server = setupServer(
  rest.get('/api/users', (req, res, ctx) => {
    return res(ctx.json([
      { id: 1, name: 'John' }
    ]));
  })
);
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());
afterAll(() => server.close());

```


7.9 Next.js

Pages Router

```
// pages/index.js
export default function Home() {
  return <h1>Home</h1>;
}

// pages/about.js
export default function About() {
  return <h1>About</h1>;
}

// pages/posts/[id].js
export default function Post({ post }) {
  return <h1>{post.title}</h1>;
}

export async function getServerSideProps({ params }) {
  const res = await fetch(`https://api.example.com/posts/${params.id}`);
  const post = await res.json();
  return { props: { post } };
}
```

Static Generation

```
export async function getStaticProps() {
  const res = await fetch('https://api.example.com/posts');
  const posts = await res.json();
  return { props: { posts }, revalidate: 60 };
}

export async function getStaticPaths() {
  const res = await fetch('https://api.example.com/posts');
  const posts = await res.json();
  const paths = posts.map(post => ({ params: { id: post.id.toString() } }));
  return { paths, fallback: 'blocking' };
}
```

API Routes

```
// pages/api/users.js
export default function handler(req, res) {
  if (req.method === 'GET') {
    res.status(200).json([{ id: 1, name: 'John' }]);
  } else if (req.method === 'POST') {
    res.status(201).json({ id: 2, name: req.body.name });
  }
}
```

```
}
```

App Router (Next.js 13+)

```
// app/layout.js
export default function RootLayout({ children }) {
  return (
    <html>
      <body>{children}</body>
    </html>
  );
}

// app/page.js
export default function Home() {
  return <h1>Home</h1>;
}

// app/posts/[id]/page.js
export default async function Post({ params }) {
  const post = await fetch(`https://api.example.com/posts/${params.id}`).then(r => r.json());
  return <h1>{post.title}</h1>;
}
```

Server Components

```
// app/users/page.js
async function Users() {
  const users = await fetch('https://api.example.com/users').then(r => r.json());
  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

7.10 React Native

Basic Components

```
import { View, Text, Button, StyleSheet } from 'react-native';
function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Hello</Text>
      <Button title="Press me" onPress={() => console.log('Pressed')} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center' },
```

```

    title: { fontSize: 24, fontWeight: 'bold' }
  });

```

Lists

```

import { FlatList } from 'react-native';
function UserList({ users }) {
  return (
    <FlatList
      data={users}
      keyExtractor={item => item.id}
      renderItem={({ item }) => <Text>{item.name}</Text>}
    />
  );
}

```

Navigation

```

import { NavigationContainer } from '@react-navigation/native';
import { createStackNavigator } from '@react-navigation/stack';
const Stack = createStackNavigator();
function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={Home} />
        <Stack.Screen name="Details" component={Details} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
function Home({ navigation }) {
  return <Button title="Go to Details" onPress={() => navigation.navigate('Details')} />;
}

```

7.11 Interview Questions (50+)

Q1: Virtual DOM? **A:** Lightweight copy of actual DOM, React compares and updates only changed parts.

Q2: JSX? **A:** JavaScript XML, syntactic sugar for React.createElement().

Q3: Components vs Elements? **A:** Components: functions/classes returning elements. Elements: plain objects describing DOM.

Q4: Functional vs Class components? **A:** Functional: simpler, use hooks. Class: lifecycle methods, this binding.

Q5: Props vs State? **A:** Props: passed from parent, immutable. State: internal, mutable.

Q6: Controlled vs Uncontrolled components? **A:** Controlled: React controls state. Uncontrolled: DOM handles state.

Q7: Keys in lists? **A:** Unique identifiers help React track which items changed.

Q8: useState vs useReducer? **A:** useState: simple state. useReducer: complex state logic.

Q9: useEffect cleanup? **A:** Return function from useEffect, runs before next effect or unmount.

Q10: useEffect dependencies? **A:** Array determines when effect runs. Empty = once, omitted = every render.

Q11: useCallback vs useMemo? **A:** useCallback: memoizes functions. useMemo: memoizes values.

Q12: When to use useRef? **A:** Accessing DOM elements, storing mutable values without re-render.

Q13: Context API use case? **A:** Share data across component tree without prop drilling.

Q14: Redux vs Context? **A:** Redux: large apps, complex state, devtools. Context: simpler state sharing.

Q15: Redux middleware? **A:** Functions that intercept actions before reaching reducer (e.g., redux-thunk, redux-saga).

Q16: Pure components? **A:** Components that render same output for same props/state.

Q17: React.memo? **A:** Higher-order component that memoizes functional components.

Q18: shouldComponentUpdate? **A:** Lifecycle method to prevent unnecessary re-renders (class components).

Q19: Error boundaries? **A:** Components that catch JavaScript errors in child component tree.

Q20: Portals? **A:** Render children outside parent DOM hierarchy.

Q21: Fragments? **A:** Group children without adding extra DOM nodes.

Q22: Higher-Order Components (HOC)? **A:** Function that takes component and returns new component.

Q23: Render props? **A:** Technique for sharing code using prop whose value is a function.

Q24: React Router useNavigate? **A:** Hook to programmatically navigate.

Q25: Lazy loading? **A:** Load components only when needed using React.lazy().

Q26: Code splitting? **A:** Split bundle into smaller chunks, load on demand.

Q27: Server-Side Rendering (SSR)? **A:** Render React on server, send HTML to client.

Q28: Static Site Generation (SSG)? **A:** Pre-render pages at build time.

- Q29:** Incremental Static Regeneration (ISR)? **A:** Update static content after build without full rebuild.
- Q30:** `getServerSideProps` vs `getStaticProps`? **A:** SSP: runs on every request. GSP: runs at build time.
- Q31:** Client vs Server Components (Next.js 13)? **A:** Server: render on server, no JS sent. Client: interactive, runs in browser.
- Q32:** React Query benefits? **A:** Caching, background refetching, automatic retries, pagination.
- Q33:** State colocation? **A:** Keep state as close as possible to where it's used.
- Q34:** Lifting state up? **A:** Move state to common ancestor when multiple components need it.
- Q35:** Prop drilling? **A:** Passing props through multiple levels. Solution: Context, Redux.
- Q36:** Reconciliation? **A:** Process of comparing Virtual DOM with actual DOM and updating differences.
- Q37:** Synthetic events? **A:** Cross-browser wrapper around native browser events.
- Q38:** Event pooling? **A:** React reuses event objects for performance (removed in React 17).
- Q39:** StrictMode? **A:** Tool for highlighting potential problems in development.
- Q40:** Suspense? **A:** Component for handling loading states of lazy components.
- Q41:** Concurrent rendering? **A:** React can interrupt rendering to handle high-priority updates.
- Q42:** `useTransition`? **A:** Mark updates as non-urgent, keep UI responsive.
- Q43:** `useDeferredValue`? **A:** Defer updating part of UI to keep it responsive.
- Q44:** CSS-in-JS benefits? **A:** Scoped styles, dynamic styling, no naming conflicts.
- Q45:** CSS Modules? **A:** CSS files with locally scoped class names.
- Q46:** Styled Components? **A:** Library for writing CSS in JavaScript using tagged templates.
- Q47:** Tailwind CSS? **A:** Utility-first CSS framework.
- Q48:** React Testing Library philosophy? **A:** Test components as users would interact with them.
- Q49:** Snapshot testing? **A:** Capture component output, detect unexpected changes.
- Q50:** React Native vs Flutter? **A:** RN: JavaScript, native components. Flutter: Dart, custom rendering engine.

Chapter 8: System Design

8.1 System Design Fundamentals

Scalability

- **Vertical Scaling:** Increase server resources (CPU, RAM)
 - Pros: Simple, no code changes
 - Cons: Hardware limits, single point of failure, downtime
- **Horizontal Scaling:** Add more servers
 - Pros: No limits, high availability, better fault tolerance
 - Cons: Complex, data consistency challenges

Load Balancing

- Distributes traffic across multiple servers
- **Algorithms:** Round Robin, Least Connections, IP Hash, Weighted Round Robin
- **Layer 4 (Transport):** Based on IP/Port
- **Layer 7 (Application):** Based on content (HTTP headers, cookies)

Caching

- **Client-side:** Browser cache
- **CDN:** Static content (images, CSS, JS)
- **Application:** Redis, Memcached
- **Database:** Query results
- **Strategies:** Write-through, Write-behind, Cache-aside, Refresh-ahead

Database

- **SQL:** ACID, structured, relations (PostgreSQL, MySQL)
- **NoSQL:** Flexible schema, horizontal scaling
 - Document: MongoDB, Couchbase
 - Key-Value: Redis, DynamoDB
 - Column-family: Cassandra, HBase
 - Graph: Neo4j

Replication & Sharding

- **Replication:** Copy data across servers
 - Master-Slave: Master writes, slaves read
 - Master-Master: Both read/write
- **Sharding:** Horizontal partitioning
 - By Range: User ID 1-1000 → Shard 1

- By Hash: $\text{Hash}(\text{user_id}) \% \text{num_shards}$
- By Geography: US users → US shard

Consistency Models

- **Strong Consistency:** All nodes see same data immediately
- **Eventual Consistency:** Nodes eventually sync
- **CAP Theorem:** Choose 2 of Consistency, Availability, Partition tolerance

Message Queues

- Asynchronous communication between services
- **Use cases:** Decoupling, load leveling, guaranteed delivery
- **Systems:** RabbitMQ, Apache Kafka, Amazon SQS

API Gateway

- Single entry point for microservices
- **Functions:** Routing, authentication, rate limiting, caching, logging

Microservices vs Monolith

Aspect	Monolith	Microservices
Structure	Single codebase	Multiple services
Deployment	Deploy all	Independent
Scaling	Scale entire app	Scale individual services
Technology	Single stack	Polyglot
Complexity	Simple initially	Complex management

8.2 Design: URL Shortener (bit.ly)

Requirements

- **Functional:** Shorten URL, redirect to original, custom aliases, expiration
- **Non-functional:** High availability, low latency, scalable

Capacity Estimation

- 500M new URLs/month = ~200 URLs/sec
- 100:1 read-to-write ratio = 20K reads/sec
- Storage: $500\text{M} * 12 \text{ months} * 5 \text{ years} * 500 \text{ bytes} = 15\text{TB}$

API Design

POST /api/shorten

Body: { "url": "https://example.com/very/long/url", "custom_alias": "abc", "expire_at": "2025-12-31" }

Response: { "short_url": "http://short.ly/abc123" }

GET /{short_code}

Response: 301 Redirect to original URL

Database Schema

```
CREATE TABLE urls (  
  id BIGSERIAL PRIMARY KEY,  
  short_code VARCHAR(10) UNIQUE NOT NULL,  
  original_url TEXT NOT NULL,  
  user_id BIGINT,  
  created_at TIMESTAMP DEFAULT NOW(),  
  expire_at TIMESTAMP,  
  click_count INT DEFAULT 0  
);  
CREATE INDEX idx_short_code ON urls(short_code);
```

Algorithm

```
// Base62 encoding  
const chars = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ';  
function encode(id) {  
  let code = "";  
  while (id > 0) {  
    code = chars[id % 62] + code;  
    id = Math.floor(id / 62);  
  }  
  return code.padStart(7, '0');  
}  
function decode(code) {  
  let id = 0;  
  for (let char of code) {  
    id = id * 62 + chars.indexOf(char);  
  }  
  return id;  
}
```


Architecture

Client → Load Balancer → API Servers → Cache (Redis) → Database (Sharded)
 ↓
 Analytics Service → Analytics DB

Key Design Decisions

- **Short code generation:** Auto-increment ID + Base62
- **Caching:** Redis for hot URLs (80/20 rule)
- **Database:** Sharded by hash(short_code)
- **Rate limiting:** Prevent abuse
- **Analytics:** Separate service tracks clicks asynchronously

8.3 Design: Twitter

Requirements

- **Functional:** Tweet (280 chars), timeline, follow/unfollow, like, retweet
- **Non-functional:** 200M DAU, 100M tweets/day, low latency

Capacity Estimation

- Tweets: 100M/day = ~1200/sec
- Reads: 1200 * 100 (fanout) = 120K/sec
- Storage: 100M * 365 * 300 bytes = 11TB/year

API Design

POST /api/tweet

Body: { "text": "Hello world", "media": ["img1.jpg"] }

GET /api/timeline?user_id=123&page=1&limit=20

Response: [{ "id": 1, "user": {...}, "text": "...", "created_at": "..." }]

POST /api/follow

Body: { "followee_id": 456 }

Database Schema

```
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  username VARCHAR(50) UNIQUE,
  email VARCHAR(255) UNIQUE,
```

```

    created_at TIMESTAMP
  );
CREATE TABLE tweets (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT REFERENCES users(id),
  text VARCHAR(280),
  created_at TIMESTAMP,
  like_count INT DEFAULT 0,
  retweet_count INT DEFAULT 0
);
CREATE TABLE follows (
  follower_id BIGINT REFERENCES users(id),
  followee_id BIGINT REFERENCES users(id),
  created_at TIMESTAMP,
  PRIMARY KEY (follower_id, followee_id)
);
CREATE INDEX idx_tweets_user_id ON tweets(user_id);
CREATE INDEX idx_follows_follower ON follows(follower_id);

```

Timeline Generation

Approach 1: Pull (On-demand)

```

SELECT tweets.* FROM tweets
JOIN follows ON tweets.user_id = follows.followee_id
WHERE follows.follower_id = ?
ORDER BY tweets.created_at DESC LIMIT 20;

```

- Pros: Consistent, no storage overhead
- Cons: Slow for users with many follows

Approach 2: Push (Fanout-on-write)

```

// When user tweets, push to all followers' timelines
async function createTweet(userId, text) {
  const tweet = await saveTweet(userId, text);
  const followers = await getFollowers(userId);
  for (let follower of followers) {
    await redis.lpush(`timeline:${follower}` , tweet.id);
  }
}

```

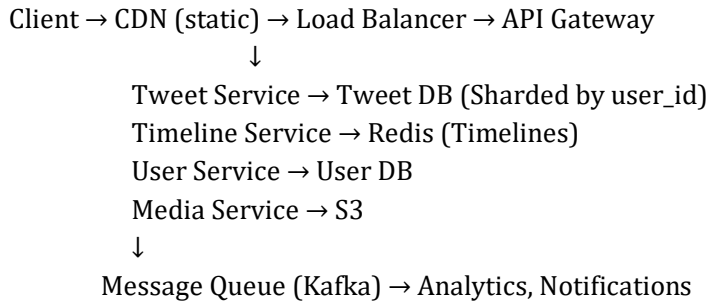
- Pros: Fast reads
- Cons: Slow writes for celebrities, storage overhead

Hybrid Approach

- Push for regular users (<10K followers)

- Pull for celebrities
- Cache hot timelines in Redis

Architecture



8.4 Design: Instagram

Requirements

- **Functional:** Upload photos, follow users, feed, like, comment
- **Non-functional:** 500M DAU, 100M photos/day, low latency

Photo Storage

- **Metadata DB:** PostgreSQL (user_id, photo_id, location, created_at)
- **Photo Storage:** S3, CloudFront CDN
- **Optimization:** Multiple sizes (thumbnail, small, medium, large)

Database Schema

```

CREATE TABLE photos (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT,
  s3_key VARCHAR(255),
  caption TEXT,
  latitude DECIMAL(9,6),
  longitude DECIMAL(9,6),
  created_at TIMESTAMP
);
CREATE TABLE photo_likes (
  photo_id BIGINT,
  user_id BIGINT,
  created_at TIMESTAMP,
  PRIMARY KEY (photo_id, user_id)
);
CREATE INDEX idx_photos_user_id ON photos(user_id);
  
```

Feed Generation

```
// Precompute feed for active users
async function generateFeed(userId) {
  const following = await getFollowing(userId);
  const photos = await getRecentPhotos(following, limit=100);
  const ranked = rankPhotos(photos); // ML-based ranking
  await redis.set(`feed:${userId}`, JSON.stringify(ranked), 'EX', 3600);
}
```

8.5 Design: WhatsApp

Requirements

- **Functional:** 1-on-1 chat, group chat, media, online status, read receipts
- **Non-functional:** 2B users, real-time, high availability

Architecture

```
Client (WebSocket) → Load Balancer → Chat Servers (Stateful)
      ↓
    Message Queue (Kafka)
      ↓
Message Storage (Cassandra, partitioned by user_id)
Media Storage (S3)
User Service (PostgreSQL)
```

Database Schema

```
CREATE TABLE messages (
  id UUID PRIMARY KEY,
  sender_id BIGINT,
  receiver_id BIGINT,
  content TEXT,
  media_url VARCHAR(255),
  sent_at TIMESTAMP,
  delivered_at TIMESTAMP,
  read_at TIMESTAMP
);
CREATE INDEX idx_messages_receiver ON messages(receiver_id, sent_at);
```

Real-time Communication

```
// WebSocket server
io.on('connection', (socket) => {
```

```

const userId = authenticate(socket);
socket.join(`user:${userId}`);

socket.on('send_message', async (data) => {
  const msg = await saveMessage(data);
  io.to(`user:${data.receiver_id}`).emit('new_message', msg);
  await kafka.produce('messages', msg); // Async processing
});

socket.on('disconnect', () => {
  updateUserStatus(userId, 'offline');
});
});

```

Message Delivery

1. Sender sends message → API Server
2. API Server saves to DB, publishes to Kafka
3. Chat Server delivers to receiver if online
4. If offline, store in queue, deliver when online
5. Send delivery/read receipts

8.6 Design: Netflix

Requirements

- **Functional:** Video streaming, search, recommendations, profiles
- **Non-functional:** 200M users, 4K video, low latency, high availability

Video Processing Pipeline

Upload → Transcoding (multiple resolutions) → CDN (CloudFront) → Client

↓

S3 Storage
Thumbnail generation
Metadata extraction

Adaptive Bitrate Streaming

// HLS (HTTP Live Streaming)

// Manifest file

#EXTM3U

#EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=640x360

360p.m3u8

#EXT-X-STREAM-INF:BANDWIDTH=1400000,RESOLUTION=1280x720

720p.m3u8
 #EXT-X-STREAM-INF:BANDWIDTH=2800000,RESOLUTION=1920x1080
 1080p.m3u8

Recommendation System

User behavior → Kafka → Spark (batch processing)
 ↓
 ML Models (collaborative filtering, content-based)
 ↓
 Redis (personalized recommendations)

Database Schema

```
CREATE TABLE videos (
  id BIGSERIAL PRIMARY KEY,
  title VARCHAR(255),
  description TEXT,
  duration INT,
  genre VARCHAR(50),
  created_at TIMESTAMP
);
CREATE TABLE user_watch_history (
  user_id BIGINT,
  video_id BIGINT,
  watched_at TIMESTAMP,
  progress INT,
  PRIMARY KEY (user_id, video_id)
);
```

8.7 Design: Uber

Requirements

- **Functional:** Request ride, match driver, track location, payment, ratings
- **Non-functional:** 15M trips/day, real-time location, low latency

Geo-spatial Indexing

```
// QuadTree or Geohash
function getQuadrant(lat, lon) {
  return geohash.encode(lat, lon, precision=6); // ~1.2km x 0.6km
}
// Find nearby drivers
async function getNearbyDrivers(lat, lon, radius) {
```

```

const hashes = geohash.getNeighbors(geohash.encode(lat, lon, 6));
return await redis.georadius('drivers', lon, lat, radius, 'km');
}

```

Matching Algorithm

```

async function matchRider(riderId, pickup) {
  const drivers = await getNearbyDrivers(pickup.lat, pickup.lon, 5);
  const available = drivers.filter(d => d.status === 'available');
  const closest = available.sort((a, b) =>
    distance(pickup, a.location) - distance(pickup, b.location)
  )[0];
  await sendRideRequest(closest.id, riderId);
}

```

Real-time Tracking

```

// Driver sends location every 5 seconds
socket.on('location_update', async (data) => {
  await redis.geoad('drivers', data.lon, data.lat, data.driver_id);
  const ride = await getRideByDriver(data.driver_id);
  if (ride) {
    io.to(`ride:${ride.id}`).emit('driver_location', data);
  }
});

```

Database Schema

```

CREATE TABLE rides (
  id BIGSERIAL PRIMARY KEY,
  rider_id BIGINT,
  driver_id BIGINT,
  pickup_lat DECIMAL(9,6),
  pickup_lon DECIMAL(9,6),
  dropoff_lat DECIMAL(9,6),
  dropoff_lon DECIMAL(9,6),
  status VARCHAR(20),
  fare DECIMAL(10,2),
  created_at TIMESTAMP
);
CREATE INDEX idx_rides_driver ON rides(driver_id, status);

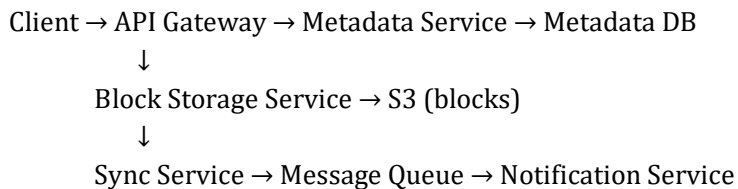
```

8.8 Design: Google Drive

Requirements

- **Functional:** Upload, download, sync, share files
- **Non-functional:** 1B users, 15GB free storage, sync across devices

File Storage



Chunking & Deduplication

```

// Split file into 4MB chunks
async function uploadFile(file) {
  const chunks = splitIntoChunks(file, 4 * 1024 * 1024);
  const chunkHashes = [];
  for (let chunk of chunks) {
    const hash = sha256(chunk);
    chunkHashes.push(hash);
    if (!await chunkExists(hash)) {
      await uploadToS3(hash, chunk);
    }
  }
  await saveMetadata(file.name, chunkHashes);
}
  
```

Sync Protocol

```

// Client polls for changes
async function sync() {
  const lastSyncTime = localStorage.getItem('last_sync');
  const changes = await api.getChanges(lastSyncTime);
  for (let change of changes) {
    if (change.type === 'upload') {
      await downloadFile(change.file_id);
    } else if (change.type === 'delete') {
      await deleteLocalFile(change.file_id);
    }
  }
  localStorage.setItem('last_sync', Date.now());
}
  
```



```
}
```

Database Schema

```
CREATE TABLE files (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT,  
  name VARCHAR(255),  
  size BIGINT,  
  parent_folder_id BIGINT,  
  is_deleted BOOLEAN DEFAULT FALSE,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
);  
CREATE TABLE file_chunks (  
  file_id BIGINT,  
  chunk_index INT,  
  chunk_hash VARCHAR(64),  
  s3_key VARCHAR(255),  
  PRIMARY KEY (file_id, chunk_index)  
);
```

8.9 Trade-offs & Best Practices

Common Trade-offs

Aspect	Option A	Option B
Consistency	Strong consistency	Eventual consistency
Latency	Lower latency	Higher consistency
Storage	Denormalized (faster reads)	Normalized (less redundancy)
Compute	Precompute (fast reads)	Compute on-demand (flexible)
Availability	Replicate data	Single source of truth

Best Practices

1. **Start simple:** Don't over-engineer
2. **Identify bottlenecks:** Profile before optimizing
3. **Cache aggressively:** 80/20 rule
4. **Decouple services:** Use message queues
5. **Monitor everything:** Metrics, logs, alerts
6. **Plan for failure:** Redundancy, backups, circuit breakers

7. **Test at scale:** Load testing, chaos engineering

8.10 Interview Questions (40+)

Q1: How to handle 1M concurrent users? **A:** Load balancing, horizontal scaling, caching, CDN, database sharding.

Q2: How to design a cache? **A:** LRU/LFU eviction, TTL, write-through/write-behind, distributed (Redis cluster).

Q3: How to prevent data loss? **A:** Replication, backups, transactions, message queues with acknowledgments.

Q4: How to handle hot partitions? **A:** Consistent hashing, add more shards, cache hot data.

Q5: SQL vs NoSQL for social network? **A:** NoSQL (Cassandra) for scalability, SQL (PostgreSQL) for users/transactions.

Q6: How to ensure data consistency? **A:** ACID transactions, 2PC (two-phase commit), saga pattern, eventual consistency.

Q7: Rate limiting strategies? **A:** Token bucket, leaky bucket, fixed/sliding window, distributed (Redis).

Q8: How to design search? **A:** Elasticsearch, inverted index, full-text search, autocomplete (trie).

Q9: How to handle celebrity problem (Twitter)? **A:** Hybrid approach: push for normal users, pull for celebrities.

Q10: CDN benefits? **A:** Reduced latency, bandwidth savings, DDoS protection, high availability.

Q11: Database indexing trade-offs? **A:** Faster reads, slower writes, storage overhead.

Q12: Replication lag solutions? **A:** Read from master for critical data, monitor lag, use quorum reads.

Q13: Circuit breaker pattern? **A:** Prevent cascading failures by stopping requests to failing service.

Q14: Idempotency importance? **A:** Prevents duplicate operations from retries, use unique request IDs.

Q15: Bloom filter use case? **A:** Check if element might exist (URL shortener collision check).

Q16: Count-Min Sketch? **A:** Probabilistic data structure for frequency counting.

Q17: How to handle file uploads? **A:** Direct to S3 (presigned URLs), chunking, resumable uploads.

Q18: How to prevent DDOS? **A:** Rate limiting, CDN (Cloudflare), WAF, IP blacklisting.

Q19: How to design notifications? **A:** Message queue (Kafka), delivery service (push/email/SMS), retry logic.

Q20: Database connection pooling? **A:** Reuse connections, prevent exhaustion, configure min/max size.

Q21: How to handle time zones? **A:** Store UTC, convert to user timezone in frontend.

Q22: Optimistic vs Pessimistic locking? **A:** Optimistic: assume no conflict, check before commit. Pessimistic: lock immediately.

Q23: Distributed transaction challenges? **A:** Network partitions, latency, complexity. Solutions: 2PC, saga, eventual consistency.

Q24: How to backup database? **A:** Regular snapshots, incremental backups, replicate to different region.

Q25: Monitoring metrics? **A:** Latency (p50, p99), error rate, throughput, CPU/memory, database query time.

Q26: Log aggregation? **A:** Centralized (ELK stack), structured logging, log levels, retention policy.

Q27: Blue-green deployment? **A:** Two identical environments, switch traffic after testing, easy rollback.

Q28: Canary deployment? **A:** Gradual rollout to small percentage, monitor metrics, expand if stable.

Q29: Feature flags? **A:** Toggle features without deployment, A/B testing, gradual rollout.

Q30: API versioning strategies? **A:** URL path (/v1/users), header (Accept: application/vnd.api+json;version=1).

Q31: GraphQL vs REST? **A:** GraphQL: flexible queries, single endpoint. REST: simple, cacheable, standard.

Q32: Webhook vs Polling? **A:** Webhook: push-based, real-time. Polling: pull-based, simpler.

Q33: OAuth 2.0 flow? **A:** Authorization code, client credentials, implicit, password grant.

Q34: JWT vs Session? **A:** JWT: stateless, scalable. Session: stateful, easier to revoke.

Q35: CORS? **A:** Cross-Origin Resource Sharing, browser security, configure allowed origins.

Q36: SQL injection prevention? **A:** Parameterized queries, ORMs, input validation, least privilege.

Q37: XSS prevention? **A:** Escape output, Content-Security-Policy header, HttpOnly cookies.

Q38: CSRF prevention? **A:** CSRF tokens, SameSite cookies, verify origin header.

Q39: Data encryption? **A:** At rest (AES-256), in transit (TLS), key management (KMS).

Q40: Disaster recovery plan? **A:** Backups, multi-region, RTO/RPO targets, failover testing.

Chapter 9: Advanced Algorithms

9.1 Dynamic Programming (30 problems)

Fibonacci (Memoization)

```
int fib(int n, Map<Integer, Integer> memo) {  
    if (n <= 1) return n;  
    if (memo.containsKey(n)) return memo.get(n);  
    int result = fib(n-1, memo) + fib(n-2, memo);  
    memo.put(n, result);  
    return result;  
}
```

Fibonacci (Tabulation)

```
int fib(int n) {  
    if (n <= 1) return n;  
    int[] dp = new int[n+1];  
    dp[1] = 1;  
    for (int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];  
    return dp[n];  
}
```

Climbing Stairs

```
int climbStairs(int n) {  
    if (n <= 2) return n;  
    int[] dp = new int[n+1];  
    dp[1] = 1; dp[2] = 2;  
    for (int i = 3; i <= n; i++) dp[i] = dp[i-1] + dp[i-2];  
    return dp[n];  
}
```

House Robber

```
int rob(int[] nums) {  
    if (nums.length == 0) return 0;  
    if (nums.length == 1) return nums[0];  
    int prev2 = 0, prev1 = 0;  
    for (int num : nums) {  
        int temp = Math.max(prev1, prev2 + num);  
        prev2 = prev1;  
        prev1 = temp;  
    }  
}
```

```

    return prev1;
}

```

Longest Increasing Subsequence

```

int lengthOfLIS(int[] nums) {
    int[] dp = new int[nums.length];
    Arrays.fill(dp, 1);
    for (int i = 1; i < nums.length; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
        }
    }
    return Arrays.stream(dp).max().getAsInt();
}

```

Longest Common Subsequence

```

int longestCommonSubsequence(String s1, String s2) {
    int[][] dp = new int[s1.length()+1][s2.length()+1];
    for (int i = 1; i <= s1.length(); i++) {
        for (int j = 1; j <= s2.length(); j++) {
            if (s1.charAt(i-1) == s2.charAt(j-1)) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[s1.length()][s2.length()];
}

```

Edit Distance

```

int minDistance(String word1, String word2) {
    int m = word1.length(), n = word2.length();
    int[][] dp = new int[m+1][n+1];
    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1.charAt(i-1) == word2.charAt(j-1)) dp[i][j] = dp[i-1][j-1];
            else dp[i][j] = 1 + Math.min(dp[i-1][j-1], Math.min(dp[i-1][j], dp[i][j-1]));
        }
    }
    return dp[m][n];
}

```

0/1 Knapsack

```
int knapsack(int[] weights, int[] values, int capacity) {
    int n = weights.length;
    int[][] dp = new int[n+1][capacity+1];
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= capacity; w++) {
            if (weights[i-1] <= w) {
                dp[i][w] = Math.max(dp[i-1][w], values[i-1] + dp[i-1][w-weights[i-1]]);
            } else dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][capacity];
}
```

Coin Change

```
int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount+1];
    Arrays.fill(dp, amount+1);
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (i >= coin) dp[i] = Math.min(dp[i], dp[i-coin] + 1);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Partition Equal Subset Sum

```
boolean canPartition(int[] nums) {
    int sum = Arrays.stream(nums).sum();
    if (sum % 2 != 0) return false;
    int target = sum / 2;
    boolean[] dp = new boolean[target+1];
    dp[0] = true;
    for (int num : nums) {
        for (int j = target; j >= num; j--) {
            dp[j] = dp[j] || dp[j-num];
        }
    }
    return dp[target];
}
```

Palindromic Substrings

```

int countSubstrings(String s) {
    int count = 0;
    for (int i = 0; i < s.length(); i++) {
        count += expandAroundCenter(s, i, i);
        count += expandAroundCenter(s, i, i+1);
    }
    return count;
}

int expandAroundCenter(String s, int left, int right) {
    int count = 0;
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        count++;
        left--;
        right++;
    }
    return count;
}

```

Longest Palindromic Substring

```

String longestPalindrome(String s) {
    if (s.length() < 2) return s;
    int start = 0, maxLen = 0;
    for (int i = 0; i < s.length(); i++) {
        int len1 = expandAroundCenter(s, i, i);
        int len2 = expandAroundCenter(s, i, i+1);
        int len = Math.max(len1, len2);
        if (len > maxLen) {
            start = i - (len - 1) / 2;
            maxLen = len;
        }
    }
    return s.substring(start, start + maxLen);
}

int expandAroundCenter(String s, int left, int right) {
    while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
        left--;
        right++;
    }
    return right - left - 1;
}

```

Decode Ways

```
int numDecodings(String s) {
    if (s.charAt(0) == '0') return 0;
    int[] dp = new int[s.length()+1];
    dp[0] = 1; dp[1] = 1;
    for (int i = 2; i <= s.length(); i++) {
        int oneDigit = Integer.parseInt(s.substring(i-1, i));
        int twoDigits = Integer.parseInt(s.substring(i-2, i));
        if (oneDigit >= 1) dp[i] += dp[i-1];
        if (twoDigits >= 10 && twoDigits <= 26) dp[i] += dp[i-2];
    }
    return dp[s.length()];
}
```

Unique Paths

```
int uniquePaths(int m, int n) {
    int[][] dp = new int[m][n];
    for (int i = 0; i < m; i++) dp[i][0] = 1;
    for (int j = 0; j < n; j++) dp[0][j] = 1;
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
        }
    }
    return dp[m-1][n-1];
}
```

Minimum Path Sum

```
int minPathSum(int[][] grid) {
    int m = grid.length, n = grid[0].length;
    for (int i = 1; i < m; i++) grid[i][0] += grid[i-1][0];
    for (int j = 1; j < n; j++) grid[0][j] += grid[0][j-1];
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            grid[i][j] += Math.min(grid[i-1][j], grid[i][j-1]);
        }
    }
    return grid[m-1][n-1];
}
```


Triangle Minimum Path

```
int minimumTotal(List<List<Integer>> triangle) {
    for (int i = triangle.size() - 2; i >= 0; i--) {
        for (int j = 0; j < triangle.get(i).size(); j++) {
            int min = Math.min(triangle.get(i+1).get(j), triangle.get(i+1).get(j+1));
            triangle.get(i).set(j, triangle.get(i).get(j) + min);
        }
    }
    return triangle.get(0).get(0);
}
```

Maximum Product Subarray

```
int maxProduct(int[] nums) {
    int max = nums[0], min = nums[0], result = nums[0];
    for (int i = 1; i < nums.length; i++) {
        if (nums[i] < 0) {
            int temp = max;
            max = min;
            min = temp;
        }
        max = Math.max(nums[i], max * nums[i]);
        min = Math.min(nums[i], min * nums[i]);
        result = Math.max(result, max);
    }
    return result;
}
```

Word Break

```
boolean wordBreak(String s, List<String> wordDict) {
    Set<String> dict = new HashSet<>(wordDict);
    boolean[] dp = new boolean[s.length()+1];
    dp[0] = true;
    for (int i = 1; i <= s.length(); i++) {
        for (int j = 0; j < i; j++) {
            if (dp[j] && dict.contains(s.substring(j, i))) {
                dp[i] = true;
                break;
            }
        }
    }
    return dp[s.length()];
}
```

Regular Expression Matching

```

boolean isMatch(String s, String p) {
    boolean[][] dp = new boolean[s.length()+1][p.length()+1];
    dp[0][0] = true;
    for (int j = 1; j <= p.length(); j++) {
        if (p.charAt(j-1) == '*') dp[0][j] = dp[0][j-2];
    }
    for (int i = 1; i <= s.length(); i++) {
        for (int j = 1; j <= p.length(); j++) {
            if (p.charAt(j-1) == s.charAt(i-1) || p.charAt(j-1) == '.') {
                dp[i][j] = dp[i-1][j-1];
            } else if (p.charAt(j-1) == '*') {
                dp[i][j] = dp[i][j-2];
                if (p.charAt(j-2) == s.charAt(i-1) || p.charAt(j-2) == '.') {
                    dp[i][j] = dp[i][j] || dp[i-1][j];
                }
            }
        }
    }
    return dp[s.length()][p.length()];
}

```

Burst Balloons

```

int maxCoins(int[] nums) {
    int n = nums.length + 2;
    int[] arr = new int[n];
    arr[0] = arr[n-1] = 1;
    for (int i = 1; i < n-1; i++) arr[i] = nums[i-1];
    int[][] dp = new int[n][n];
    for (int len = 2; len < n; len++) {
        for (int left = 0; left < n - len; left++) {
            int right = left + len;
            for (int i = left + 1; i < right; i++) {
                dp[left][right] = Math.max(dp[left][right],
                    arr[left] * arr[i] * arr[right] + dp[left][i] + dp[i][right]);
            }
        }
    }
    return dp[0][n-1];
}

```

9.2 Graph Algorithms (15 problems)

DFS (Recursive)

```
void dfs(int[][] graph, int node, boolean[] visited) {
    visited[node] = true;
    System.out.println(node);
    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) dfs(graph, neighbor, visited);
    }
}
```

BFS

```
void bfs(int[][] graph, int start) {
    boolean[] visited = new boolean[graph.length];
    Queue<Integer> queue = new LinkedList<>();
    queue.offer(start);
    visited[start] = true;
    while (!queue.isEmpty()) {
        int node = queue.poll();
        System.out.println(node);
        for (int neighbor : graph[node]) {
            if (!visited[neighbor]) {
                queue.offer(neighbor);
                visited[neighbor] = true;
            }
        }
    }
}
```

Topological Sort (DFS)

```
List<Integer> topologicalSort(int n, int[][] edges) {
    List<Integer>[] graph = new ArrayList[n];
    for (int i = 0; i < n; i++) graph[i] = new ArrayList<>();
    for (int[] edge : edges) graph[edge[0]].add(edge[1]);
    Stack<Integer> stack = new Stack<>();
    boolean[] visited = new boolean[n];
    for (int i = 0; i < n; i++) {
        if (!visited[i]) dfs(graph, i, visited, stack);
    }
    List<Integer> result = new ArrayList<>();
    while (!stack.isEmpty()) result.add(stack.pop());
    return result;
}
```

```

void dfs(List<Integer>[] graph, int node, boolean[] visited, Stack<Integer> stack) {
    visited[node] = true;
    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) dfs(graph, neighbor, visited, stack);
    }
    stack.push(node);
}

```

Topological Sort (Kahn's BFS)

```

List<Integer> topologicalSort(int n, int[][] edges) {
    List<Integer>[] graph = new ArrayList[n];
    int[] indegree = new int[n];
    for (int i = 0; i < n; i++) graph[i] = new ArrayList<>();
    for (int[] edge : edges) {
        graph[edge[0]].add(edge[1]);
        indegree[edge[1]]++;
    }
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) queue.offer(i);
    }
    List<Integer> result = new ArrayList<>();
    while (!queue.isEmpty()) {
        int node = queue.poll();
        result.add(node);
        for (int neighbor : graph[node]) {
            if (--indegree[neighbor] == 0) queue.offer(neighbor);
        }
    }
    return result.size() == n ? result : new ArrayList<>();
}

```

Dijkstra's Shortest Path

```

int[] dijkstra(int[][] graph, int src) {
    int n = graph.length;
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int node = curr[0], d = curr[1];
        if (d > dist[node]) continue;
        for (int i = 0; i < n; i++) {

```

```

    if (graph[node][i] != 0) {
        int newDist = dist[node] + graph[node][i];
        if (newDist < dist[i]) {
            dist[i] = newDist;
            pq.offer(new int[]{i, newDist});
        }
    }
}
return dist;
}

```

Bellman-Ford (Negative Weights)

```

int[] bellmanFord(int n, int[][] edges, int src) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) {
        for (int[] edge : edges) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    // Check for negative cycle
    for (int[] edge : edges) {
        int u = edge[0], v = edge[1], w = edge[2];
        if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
            return null; // Negative cycle exists
        }
    }
    return dist;
}

```

Floyd-Warshall (All Pairs)

```

int[][] floydWarshall(int[][] graph) {
    int n = graph.length;
    int[][] dist = new int[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = graph[i][j];
        }
    }
    for (int k = 0; k < n; k++) {

```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (dist[i][k] != Integer.MAX_VALUE && dist[k][j] != Integer.MAX_VALUE) {
            dist[i][j] = Math.min(dist[i][j], dist[i][k] + dist[k][j]);
        }
    }
}
return dist;
}

```

Prim's MST

```

int primMST(int[][] graph) {
    int n = graph.length;
    boolean[] inMST = new boolean[n];
    int[] key = new int[n];
    Arrays.fill(key, Integer.MAX_VALUE);
    key[0] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
    pq.offer(new int[]{0, 0});
    int mstWeight = 0;
    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int u = curr[0];
        if (inMST[u]) continue;
        inMST[u] = true;
        mstWeight += curr[1];
        for (int v = 0; v < n; v++) {
            if (graph[u][v] != 0 && !inMST[v] && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                pq.offer(new int[]{v, key[v]});
            }
        }
    }
    return mstWeight;
}

```

Kruskal's MST (Union-Find)

```

int kruskalMST(int n, int[][] edges) {
    Arrays.sort(edges, (a, b) -> a[2] - b[2]);
    UnionFind uf = new UnionFind(n);
    int mstWeight = 0, edgesUsed = 0;
    for (int[] edge : edges) {
        if (uf.union(edge[0], edge[1])) {
            mstWeight += edge[2];
        }
    }
}

```

```

        if (++edgesUsed == n - 1) break;
    }
}
return mstWeight;
}
class UnionFind {
    int[] parent, rank;
    UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false;
        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[px] > rank[py]) parent[py] = px;
        else { parent[py] = px; rank[px]++; }
        return true;
    }
}

```

Detect Cycle (Undirected)

```

boolean hasCycle(int n, int[][] edges) {
    List<Integer>[] graph = new ArrayList[n];
    for (int i = 0; i < n; i++) graph[i] = new ArrayList<>();
    for (int[] edge : edges) {
        graph[edge[0]].add(edge[1]);
        graph[edge[1]].add(edge[0]);
    }
    boolean[] visited = new boolean[n];
    for (int i = 0; i < n; i++) {
        if (!visited[i] && dfs(graph, i, -1, visited)) return true;
    }
    return false;
}
boolean dfs(List<Integer>[] graph, int node, int parent, boolean[] visited) {
    visited[node] = true;
    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            if (dfs(graph, neighbor, node, visited)) return true;
        } else if (neighbor != parent) return true;
    }
}

```

```

    return false;
}

```

Detect Cycle (Directed)

```

boolean hasCycle(int n, int[][] edges) {
    List<Integer>[] graph = new ArrayList[n];
    for (int i = 0; i < n; i++) graph[i] = new ArrayList<>();
    for (int[] edge : edges) graph[edge[0]].add(edge[1]);
    int[] visited = new int[n]; // 0: unvisited, 1: visiting, 2: visited
    for (int i = 0; i < n; i++) {
        if (dfs(graph, i, visited)) return true;
    }
    return false;
}

boolean dfs(List<Integer>[] graph, int node, int[] visited) {
    if (visited[node] == 1) return true;
    if (visited[node] == 2) return false;
    visited[node] = 1;
    for (int neighbor : graph[node]) {
        if (dfs(graph, neighbor, visited)) return true;
    }
    visited[node] = 2;
    return false;
}

```

Strongly Connected Components (Kosaraju)

```

List<List<Integer>> findSCCs(int n, int[][] edges) {
    List<Integer>[] graph = new ArrayList[n];
    List<Integer>[] reversed = new ArrayList[n];
    for (int i = 0; i < n; i++) {
        graph[i] = new ArrayList<>();
        reversed[i] = new ArrayList<>();
    }
    for (int[] edge : edges) {
        graph[edge[0]].add(edge[1]);
        reversed[edge[1]].add(edge[0]);
    }
    Stack<Integer> stack = new Stack<>();
    boolean[] visited = new boolean[n];
    for (int i = 0; i < n; i++) {
        if (!visited[i]) fillOrder(graph, i, visited, stack);
    }
    Arrays.fill(visited, false);
    List<List<Integer>> sccs = new ArrayList<>();
    while (!stack.isEmpty()) {

```



```

    int node = stack.pop();
    if (!visited[node]) {
        List<Integer> scc = new ArrayList<>();
        dfs(reversed, node, visited, scc);
        sccs.add(scc);
    }
}
return sccs;
}

void fillOrder(List<Integer>[] graph, int node, boolean[] visited, Stack<Integer> stack) {
    visited[node] = true;
    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) fillOrder(graph, neighbor, visited, stack);
    }
    stack.push(node);
}

void dfs(List<Integer>[] graph, int node, boolean[] visited, List<Integer> scc) {
    visited[node] = true;
    scc.add(node);
    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) dfs(graph, neighbor, visited, scc);
    }
}

```

9.3 Greedy Algorithms (10 problems)

Activity Selection

```

int maxActivities(int[] start, int[] end) {
    int[][] activities = new int[start.length][2];
    for (int i = 0; i < start.length; i++) {
        activities[i] = new int[]{start[i], end[i]};
    }
    Arrays.sort(activities, (a, b) -> a[1] - b[1]);
    int count = 1, lastEnd = activities[0][1];
    for (int i = 1; i < activities.length; i++) {
        if (activities[i][0] >= lastEnd) {
            count++;
            lastEnd = activities[i][1];
        }
    }
    return count;
}

```

Jump Game

```
boolean canJump(int[] nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > maxReach) return false;
        maxReach = Math.max(maxReach, i + nums[i]);
    }
    return true;
}
```

Jump Game II

```
int jump(int[] nums) {
    int jumps = 0, currentEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);
        if (i == currentEnd) {
            jumps++;
            currentEnd = farthest;
        }
    }
    return jumps;
}
```

Gas Station

```
int canCompleteCircuit(int[] gas, int[] cost) {
    int total = 0, tank = 0, start = 0;
    for (int i = 0; i < gas.length; i++) {
        total += gas[i] - cost[i];
        tank += gas[i] - cost[i];
        if (tank < 0) {
            start = i + 1;
            tank = 0;
        }
    }
    return total >= 0 ? start : -1;
}
```

Minimum Number of Arrows

```
int findMinArrowShots(int[][] points) {
    Arrays.sort(points, (a, b) -> Integer.compare(a[1], b[1]));
    int arrows = 1, end = points[0][1];
    for (int i = 1; i < points.length; i++) {

```

```

    if (points[i][0] > end) {
        arrows++;
        end = points[i][1];
    }
}
return arrows;
}

```

9.4 Backtracking (10 problems)

N-Queens

```

List<List<String>> solveNQueens(int n) {
    List<List<String>> result = new ArrayList<>();
    char[][] board = new char[n][n];
    for (char[] row : board) Arrays.fill(row, '.');
    backtrack(result, board, 0);
    return result;
}

void backtrack(List<List<String>> result, char[][] board, int row) {
    if (row == board.length) {
        result.add(construct(board));
        return;
    }
    for (int col = 0; col < board.length; col++) {
        if (isValid(board, row, col)) {
            board[row][col] = 'Q';
            backtrack(result, board, row + 1);
            board[row][col] = '.';
        }
    }
}

boolean isValid(char[][] board, int row, int col) {
    for (int i = 0; i < row; i++) if (board[i][col] == 'Q') return false;
    for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) if (board[i][j] == 'Q') return false;
    for (int i = row - 1, j = col + 1; i >= 0 && j < board.length; i--, j++) if (board[i][j] == 'Q') return false;
    return true;
}

List<String> construct(char[][] board) {
    List<String> result = new ArrayList<>();
    for (char[] row : board) result.add(new String(row));
    return result;
}

```

Sudoku Solver

```

void solveSudoku(char[][] board) {
    solve(board);
}

boolean solve(char[][] board) {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (board[i][j] == '.') {
                for (char c = '1'; c <= '9'; c++) {
                    if (isValid(board, i, j, c)) {
                        board[i][j] = c;
                        if (solve(board)) return true;
                        board[i][j] = '.';
                    }
                }
                return false;
            }
        }
    }
    return true;
}

boolean isValid(char[][] board, int row, int col, char c) {
    for (int i = 0; i < 9; i++) {
        if (board[row][i] == c || board[i][col] == c ||
            board[3*(row/3)+i/3][3*(col/3)+i%3] == c) return false;
    }
    return true;
}

```

9.5 Interview Questions (30+)

Q1: DP vs Greedy? **A:** DP: optimal substructure + overlapping subproblems. Greedy: local optimal → global optimal.

Q2: Memoization vs Tabulation? **A:** Memoization: top-down, recursive. Tabulation: bottom-up, iterative.

Q3: When to use DP? **A:** Overlapping subproblems, optimal substructure (knapsack, LCS, coin change).

Q4: Time complexity of Dijkstra? **A:** $O((V+E) \log V)$ with min-heap.

Q5: Dijkstra vs Bellman-Ford? **A:** Dijkstra: faster, no negative weights. Bellman-Ford: handles negative weights.

Q6: Floyd-Warshall use case? **A:** All pairs shortest path, $O(V^3)$.

Q7: Prim vs Kruskal? **A:** Prim: dense graphs, $O(E \log V)$. Kruskal: sparse graphs, $O(E \log E)$.

Q8: Union-Find time complexity? **A:** Nearly $O(1)$ with path compression and union by rank.

- Q9:** Topological sort requirement? **A:** DAG (Directed Acyclic Graph).
- Q10:** DFS vs BFS? **A:** DFS: stack/recursion, path finding. BFS: queue, shortest path in unweighted.
- Q11:** Dijkstra vs BFS? **A:** BFS: unweighted, $O(V+E)$. Dijkstra: weighted, $O((V+E) \log V)$.
- Q12:** Strongly Connected Components? **A:** Maximal subgraph where every vertex is reachable from every other. Kosaraju/Tarjan algorithm.
- Q13:** Articulation points? **A:** Vertices whose removal increases number of connected components.
- Q14:** Bridge in graph? **A:** Edge whose removal increases number of connected components.
- Q15:** Cycle detection methods? **A:** DFS (colors), Union-Find, topological sort (directed).
- Q16:** Bipartite graph check? **A:** BFS/DFS with 2-coloring.
- Q17:** Hamiltonian vs Eulerian? **A:** Hamiltonian: visits each vertex once (NP-complete). Eulerian: visits each edge once.
- Q18:** Eulerian path conditions? **A:** Connected, exactly 0 or 2 vertices with odd degree.
- Q19:** A* algorithm? **A:** Dijkstra + heuristic, $f(n) = g(n) + h(n)$.
- Q20:** Knapsack variants? **A:** 0/1, unbounded, fractional.
- Q21:** LCS vs LIS? **A:** LCS: common subsequence of two strings. LIS: increasing subsequence in one array.
- Q22:** Edit distance applications? **A:** Spell check, DNA sequencing, diff tools.
- Q23:** Kadane's algorithm? **A:** Maximum subarray sum, $O(n)$.
- Q24:** Two-pointer technique? **A:** Array/string problems, two sum, container with most water.
- Q25:** Sliding window? **A:** Subarray problems, longest substring, max sum subarray of size k.
- Q26:** Monotonic stack? **A:** Next greater element, largest rectangle in histogram.
- Q27:** Trie applications? **A:** Autocomplete, spell checker, IP routing.
- Q28:** Segment tree? **A:** Range queries (sum, min, max), $O(\log n)$ query/update.
- Q29:** Fenwick tree (BIT)? **A:** Prefix sums, $O(\log n)$ query/update.
- Q30:** Disjoint Set applications? **A:** Network connectivity, Kruskal's MST, cycle detection.

Thank You 🙌

All The Best For Your Interviews